Google

SWE Coding Questions - Last 30 days

22 questions candidates reported | by source site, each Hard, then Medium, then Easy

Collected 2026-10-20 | one optimal answer each, brute force as pseudo-code, follow-ups

3	19	0
HARD	MEDIUM	EASY

Optimal answers: 1 from the interview site, 5 from your app, 16 written here.
★ = top 75 Medium/Hard or top 10 Easy by Google frequency (1 in this PDF).
Study tiers by Google frequency on LeetCode: TYPE 6, TALK 7, READ 9 (in this PDF).
2 design-style questions are mixed in under their topics, last in each topic and marked DESIGN-STYLE.

Order: source site, recency (last 30 days, 90 days, ...), Hard, Medium, Easy, then strictly newest to oldest

- **1Point3Acres (20): Hard 3, Medium 17, Easy 0**
- **PracHub (1): Hard 0, Medium 1, Easy 0**
- **Exponent (1): Hard 0, Medium 1, Easy 0**

Date pills (colour = how recent): LAST 30 DAYS | LAST 90 DAYS | LAST 6 MONTHS

1Point3Acres: the date the question was last asked. PracHub: the day it was posted there. Exponent: ~ means an estimate from 'N months ago' (it is the last time any company was asked). HackTheRounds shows no per-question dates.

Topics (a tag on every dated question; headings only where there are no dates)

- BFS/DFS Graph and Tree Traversal and Shortest Paths (10)
- Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns (6)
- Dynamic Programming and State-Space Optimization (3)
- Expression Parsing and Evaluation (1)
- Other: Design, Heaps, Intervals and Misc (1)
- Trie and Prefix Indexing (1)

Sources

- hacktherounds.com/questions/google
- prachub.com Google SWE Coding & Algorithms, Technical Screen only
- aced.io/questions?company=google&role=swe&type=coding
- 1point3acres.com/interview/problems/company/google (SWE coding, no OA)

How to use

- Tap any title in the Contents to jump to it.
- Every page has < Prev | Contents | Next > at the bottom.

Google - Contents

22 questions | tap a title to jump to it

1POINT3ACRES

20 questions | last 30 days: 20 | last 90 days: 20 Go >

ASKED: LAST 30 DAYS (20) Go >

Hard (3) Go >

H	1. Median of Two Sorted Arrays (LC 4) with a Follo...	ARRAY / STRING	IN APP	Oct 9 '26	TYPE | LC 4 | 1Point3Acres
H	2. Friend Graph with Timeline (Friend / Unfriend Ev...	BFS / DFS / TREE		Sep 29 '26	TALK | 1Point3Acres
H	3. Maximum-Area Rectangle from Planar Points	ARRAY / STRING		Sep 29 '26	TALK | LC 963 | 1Point3Acres

Medium (17) Go >

M	4. Template Variable Expansion with Follow-Ups	BFS / DFS / TREE		Oct 17 '26	READ | 1Point3Acres
M	5. Bounded Knapsack Variant	DP		Oct 16 '26	TALK | LC 1710 | 1Point3Acres
M	6. URL Request Handler Router	TRIE		Oct 16 '26	READ | 1Point3Acres
M	7. Mahjong Winning Hand Detection (14 Tiles)	ARRAY / STRING		Oct 9 '26	READ | 1Point3Acres
M	8. Train / Flight Schedule Reachability	BFS / DFS / TREE		Oct 8 '26	TALK | LC 2092 | 1Point3Acres
M	9. Visible People in a Line (Last Person, Then Ever...	DESIGN / OTHER		Oct 7 '26	TALK | LC 1944 | 1Point3Acres
M	10. Course Scheduling and Dependency Variants	BFS / DFS / TREE	IN APP	Oct 4 '26	TALK | LC 210 | 1Point3Acres
M	11. Connected Crop Arrangement on a Grid	BFS / DFS / TREE		Oct 4 '26	READ | 1Point3Acres
M	12. Longest Non-Decreasing Subarray with One R...	DP		Oct 4 '26	READ | 1Point3Acres
★ M	13. Number of Islands	BFS / DFS / TREE	IN APP	Sep 30 '26	TYPE | LC 200 | 1Point3Acres
M	14. Path Minimizing the Maximum Cell (Swim in Ri...	BFS / DFS / TREE	IN APP	Sep 29 '26	TYPE | LC 778 | 1Point3Acres
M	15. Shortest Path Nodes and Escaping a Pursuer	BFS / DFS / TREE		Sep 29 '26	READ | 1Point3Acres
M	16. Meeting Rooms / Interval Scheduling Variants	ARRAY / STRING	IN APP	Sep 29 '26	TYPE | LC 253 | 1Point3Acres
M	17. Range Duplicate Check with Constant-Time Qu...	ARRAY / STRING		Sep 29 '26	READ | 1Point3Acres
M	18. Grid Path Count with Diagonal Moves and Che...	DP		Sep 29 '26	TYPE | LC 62 | 1Point3Acres
M	19. Expression Tree Single-Leaf Mutation	EXPRESSION		Sep 27 '26	READ | 1Point3Acres
M	20. Active Communication Top Users	ARRAY / STRING		Sep 22 '26	READ | 1Point3Acres

PRACHUB

1 questions | last 30 days: 1 | last 90 days: 1 Go >

POSTED: LAST 30 DAYS (1) Go >

Medium (1) Go >

M	21. Count Islands in a Grid and Track the Count as...	BFS / DFS / TREE	IN APP	Oct 2 '26	TALK | LC 305 | PracHub

EXPONENT

1 questions | last 30 days: 1 | last 90 days: 1 Go >

ASKED (ROUGH): LAST 30 DAYS (1) Go >

Medium (1) Go >

M	22. Find the shortest path in a graph.	BFS / DFS / TREE		~Sep '26	TYPE | LC 743 | Exponent

SIDE NOTE: marked problems that are not in this book

★ The top 75 Medium and Hard problems and the top 10 Easy problems by Google frequency are marked in this book. These 39 are marked too but are not questions here. Do them from your app. Rank is the position in the Medium and Hard list, or E1 to E10 for Easy.

Rank	LC	Problem	Lvl	Freq
3	3	Longest Substring Without Repeating	M	73/69.3
5	42	Trapping Rain Water	H	73.6/55.1
7	128	Longest Consecutive Sequence	M	70.3/58.3
8	15	3Sum	M	72.6/58.3
9	7	Reverse Integer	M	66.3/51.4
15	11	Container With Most Water	M	68.3/41.1
18	410	Split Array Largest Sum	H	57.7/46.9
21	17	Letter Combinations of a Phone Number	M	57.8/51.4
25	239	Sliding Window Maximum	H	54.9/46.9
26	31	Next Permutation	M	61/41.1
30	6	Zigzag Conversion	M	53.5/51.4
31	34	Find First and Last Position	M	53/46.9
32	54	Spiral Matrix	M	59.3/33
34	1944	Number of Visible People in a Queue	H	53.6/41.1
37	33	Search in Rotated Sorted Array	M	57.2/33
38	84	Largest Rectangle in Histogram	H	56.8/33
39	167	Two Sum II	M	49.8/46.9
42	155	Min Stack	M	50.8/41.1
43	19	Remove Nth Node From End of List	M	50.2/41.1
44	424	Longest Repeating Character Replacement	M	53.6/33
47	739	Daily Temperatures	M	47.8/41.1
48	78	Subsets	M	51.7/33
49	39	Combination Sum	M	47.6/41.1
52	16	3Sum Closest	M	50.6/33
53	131	Palindrome Partitioning	M	46/41.1
54	209	Minimum Size Subarray Sum	M	45.2/41.1
57	402	Remove K Digits	M	44.1/0
62	151	Reverse Words in a String	M	46/33
63	312	Burst Balloons	H	41.8/0
67	51	N-Queens	H	58.6/0
75	210	Course Schedule II	M	36.5/33
E1	20	Valid Parentheses	E	67.3/70.9
E3	14	Longest Common Prefix	E	73.6/55.1
E4	88	Merge Sorted Array	E	69.7/55.1
E5	26	Remove Duplicates from Sorted Array	E	65.7/55.1
E7	28	Find the Index of the First Occurrence	E	56.2/61
E8	485	Max Consecutive Ones	E	56/58.3
E9	169	Majority Element	E	66.5/33
E10	242	Valid Anagram	E	57.3/51.4

Freq is LeetCode's frequency for Google: all time / last 30 days.

1POINT3ACRES

20 questions | Hard 3, Medium 17, Easy 0 | asked in the last 30 days: 20, last 90 days: 20 (grouped by recency, newest first) | topics: BFS/DFS and trees first

ASKED: LAST 30 DAYS (20)

Hard (3)

HARD | IN APP LC 4 | 1Point3Acres | tech-screen | 1 of 22

Median of Two Sorted Arrays (LC 4) with a Follow-up Variant

ASKED 2026-10-09 | 11 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | ALL TIME 74.2 | 30 DAYS 61 | PRIORITY 4/228

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: your app's interview approach (LeetCode #4)

LeetCode match: LC 4 Median of Two Sorted Arrays

Frequency: Single report | Last asked: 2026-10-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/median-two-sorted-arrays-k-inputs

QUESTION

L5 R1 coding: LC 4 (Median of Two Sorted Arrays). Heading toward the LC 295 streaming two-heap approach drew a redirect from the interviewer. Follow-up: extend to many sorted inputs, in the spirit of LC 23 (Merge k Sorted Lists).

Per the candidate, the base question is LC 4 (Median of Two Sorted Arrays): given two sorted arrays, return the median of their combined elements.
Follow-up: the input becomes many sorted arrays instead of two, in the spirit of LC 23 (Merge k Sorted Lists).

APPROACH

Drill LC 4 until you can explain the partition invariant and the odd/even median cases without notes.
Drill LC 23 with a min-heap and with divide-and-conquer merging, and compare their costs as the number of inputs grows.
Practice stating, in under a minute, why a streaming two-heap approach is unnecessary when the inputs are static and sorted.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the median definition when the total is even (the mean of the two middle values), whether the arrays can be empty, whether duplicates are possible, and the sizes.

BRUTE FORCE TO SAY OUT LOUD

INTERVIEW-DAY PLAN

Merge and sort, then read the middle. O((m + n) log(m + n)); merging two sorted arrays with two pointers is O(m + n).

OPTIMAL IDEA

Binary search the cut in the shorter array: choose i in a and j = (m + n + 1) // 2 - i in b so the left parts hold half of the elements. The cut is valid when a[i-1] <= b[j] and b[j-1] <= a[i], with minus and plus infinity at the ends. The answer is the larger left value for an odd total, or the mean of the larger left and the smaller right value. O(log min(m, n)). For k arrays: binary search on the value and count elements at most x with a bisect per array, O(k log n log range), or pop from a min-heap until the middle, O(N log k).

DRY-RUN THESE CASES

One array empty, both of equal length, total odd and even, all elements equal, one array entirely smaller than the other, negative numbers.

LIKELY FOLLOW-UPS

Many sorted arrays (the stated follow-up), a stream (two heaps), or the k-th element instead of the median.

PITFALLS

Starting from the two-heap design of the streaming median (wrong family for static sorted input). Searching the longer array and running past its end. Off-by-one in the half size for odd totals.

HOW TO THINK ABOUT IT

Both inputs are already sorted, so the median is a partition problem, not a streaming one (the two-heap design of LeetCode 295 is the wrong family and the interviewer redirects it). Split the small array at i and the large one at j so that the left halves together hold half the elements; the split is right when every left value is at most every right value, which only needs two comparisons across the cut. That condition is monotone in i, so binary search it in O(log min(m, n)). For k arrays the same idea becomes: binary search on the value and count elements at most x with one bisect per array.

Basis: LeetCode 4, Median of Two Sorted Arrays; the follow-up to many sorted inputs is in the spirit of LeetCode 23, Merge k Sorted Lists.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def median(a, b):
    merged = sorted(a + b)
    n, mid = len(merged), len(merged) // 2
    if n % 2:
        return merged[mid]
    return (merged[mid - 1] + merged[mid]) / 2
```

TIME	SPACE
O((m + n) log(m + n))	O(m + n)

WHY IT IS SLOW

BRUTE FORCE

Sorting ignores that the inputs are sorted and costs O((m + n) log(m + n)) time and O(m + n) memory. A binary search for the partition reads only O(log min(m, n)) values.

OPTIMAL SOLUTION · Binary search on the partition

TIME	SPACE
O(log min(m, n))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Now there are k sorted arrays instead of two. How do you find the median?

Binary search on the answer value. For a candidate x, count the elements at most x with one bisect per array (O(k log n)); the median is the smallest x whose count reaches the middle rank. With integers the search range is the global min to max, so the cost is O(k log n log(range)). The heap alternative pops the smallest element from a k-way merge until it passes the middle, O((N / 2) log k), which is simpler but linear in the total size.

FOLLOW-UP 2 What is the heap version and when would you choose it?

A min-heap holds the front of every array. Pop (N + 1) / 2 times, remembering the one or two middle values. It is O(N log k) at worst, trivial to get right and needs no bound on the values, so choose it when the arrays are small or the values are not integers; choose the value search when N is huge and k is small.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Median of Two Sorted Arrays (LC 4) with a
# Follow-up Variant
#
# L5 R1 coding: LC 4 (Median of Two Sorted Arrays).
# Heading toward the LC 295 streaming two-heap approach
# drew a redirect from the interviewer. Follow-up: extend
# to many sorted inputs, in the spirit of LC 23 (Merge k
# Sorted Lists). Per the candidate, the base question is
# LC 4 (Median of Two Sorted Arrays): given two sorted
# arrays, return the median of their combined elements.
# Follow-up: the input becomes many sorted arrays instead
# of two, in the spirit of LC 23 (Merge k Sorted Lists).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Now there are k sorted arrays instead of two. How do
# you find the median?
# 2. What is the heap version and when would you choose
# it?
#
# INTERVIEW NOTES
# Both inputs are already sorted, so the median is a
# partition problem, not a streaming one (the two-heap
# design of LeetCode 295 is the wrong family and the
# interviewer redirects it). Split the small array at i
# and the large one at j so that the left halves together
# hold half the elements; the split is right when every
# left value is at most every right value, which only
# needs two comparisons across the cut. That condition is
# monotone in i, so binary search it in O(log min(m, n)).
# For k arrays the same idea becomes: binary search on the
# value and count elements at most x with one bisect per
# array.
#
# BRUTE FORCE (say it first): Merge everything and read
# the middle
# Concatenate, sort and take the middle one or two. It is
# the oracle and the baseline to say first. It ignores
# that the inputs are sorted, and it needs the whole
# merged array in memory. Time O((m + n) log(m + n)).
# Space O(m + n).


import bisect
import heapq


# BRUTE FORCE (say it first, it also checks the answers)
def median_brute(*arrays):
    # the oracle: merge, sort, read the middle
    merged = sorted(x for a in arrays for x in a)
    n, mid = len(merged), len(merged) // 2
    if n % 2:
        return merged[mid]
    return (merged[mid - 1] + merged[mid]) / 2


# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): Binary search on the partition
# Idea: Make a the shorter array. Binary search i in a,
# set j = half - i in b so the left side holds (m + n + 1)
# // 2 elements.
def median_two(a, b):
    """Both arrays are sorted: binary search the cut. The
    left parts must hold half of all the elements."""
    if len(a) > len(b):
        a, b = b, a
    m, n = len(a), len(b)
    if m + n == 0:
        raise ValueError("no elements")
    lo, hi, half = 0, m, (m + n + 1) // 2
    while lo <= hi:
        i = (lo + hi) // 2
        j = half - i
        # values just left and right of each cut
        a_left = a[i - 1] if i > 0 else -INF
        a_right = a[i] if i < m else INF
        b_left = b[j - 1] if j > 0 else -INF
        b_right = b[j] if j < n else INF
        if a_left <= b_right and b_left <= a_right:
            # a valid cut: read the median off its edges
            if (m + n) % 2:
                return max(a_left, b_left)
            return (max(a_left, b_left)
                    + min(a_right, b_right)) / 2
        if a_left > b_right:
            hi = i - 1
        else:
            lo = i + 1


# FOLLOW-UP 1: Now there are k sorted arrays instead of
# two. How do you find the median?
# Answer: Binary search on the answer value. For a
```

```python
# candidate x, count the elements at most x with one
# bisect per array (O(k log n)); the median is the
# smallest x whose count reaches the middle rank.
def median_k_value(arrays):
    """k arrays: binary search the value; one bisect per
    array counts how many elements are at most x."""
    arrays = [a for a in arrays if a]
    total = sum(len(a) for a in arrays)
    if total == 0:
        raise ValueError("no elements")

    def kth(rank):
        # smallest x with at least `rank` elements <= x
        lo = min(a[0] for a in arrays)
        hi = max(a[-1] for a in arrays)
        while lo < hi:
            mid = (lo + hi) // 2
            count = sum(bisect.bisect_right(a, mid)
                        for a in arrays)
            if count >= rank:
                hi = mid
            else:
                lo = mid + 1
        return lo

    if total % 2:
        return kth(total // 2 + 1)
    return (kth(total // 2) + kth(total // 2 + 1)) / 2


# FOLLOW-UP 2: What is the heap version and when would you
# choose it?
# Answer: A min-heap holds the front of every array. Pop
# (N + 1) / 2 times, remembering the one or two middle
# values.
def median_k_heap(arrays):
    """k arrays, simple version: pop from a k-way merge
    until the middle one or two values have been seen."""
    heap = [(a[0], i, 0)
            for i, a in enumerate(arrays) if a]
    heapq.heapify(heap)
    total = sum(len(a) for a in arrays)
    if total == 0:
        raise ValueError("no elements")
    low_rank, high_rank = (total - 1) // 2, total // 2
    seen, middle = 0, []
    while heap:
        value, i, j = heapq.heappop(heap)
        if seen >= low_rank:
            middle.append(value)
        if seen >= high_rank:
            break
        seen += 1
        if j + 1 < len(arrays[i]):
            nxt = (arrays[i][j + 1], i, j + 1)
            heapq.heappush(heap, nxt)
    if total % 2:
        return middle[0]
    return (middle[0] + middle[1]) / 2


if __name__ == "__main__":
    assert median_two([1, 3], [2]) == 2
    assert median_two([1, 2], [3, 4]) == 2.5
    assert median_two([], [5]) == 5
    assert median_k_value([[1, 3], [2], [4, 5, 6]]) == 3.5
    assert median_k_heap([[1, 3], [2], [4, 5, 6]]) == 3.5
    import random
    for _ in range(300):
        arrays = []
        for _ in range(random.randint(1, 5)):
            size = random.randint(0, 6)
            arrays.append(sorted(random.randint(-9, 9)
                                 for _ in range(size)))
        if not any(arrays):
            continue
        want = median_brute(*arrays)
        assert median_k_value(arrays) == want
        assert median_k_heap(arrays) == want
        if len(arrays) >= 2:
            rest = sum(arrays[1:], [])
            rest.sort()
            assert median_two(arrays[0], rest) == want
```

NOTES FROM CANDIDATES

Pitfalls: Starting from the LC 295 (Find Median from Data Stream) two-heap design is the wrong family for the base question; the interviewer stepped in with a hint. Both inputs are already sorted — lean on that.

Follow-ups: Many sorted inputs instead of two

PREPARATION

Drill LC 4 until you can explain the partition invariant and the odd/even median cases without notes.
Drill LC 23 with a min-heap and with divide-and-conquer merging, and compare their costs as the number of inputs grows.
Practice stating, in under a minute, why a streaming two-heap approach is unnecessary when the inputs are static and sorted.

HARD | 1Point3Acres | onsite-coding, phone-screen | 2 of 22

Friend Graph with Timeline (Friend / Unfriend Events)

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 186/228

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Frequency: Medium | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/friend-graph-with-timeline

QUESTION

EXAMPLES

```
Base: N=3, events [(1,A,B,friend), (5,B,C,friend)] ->
    answer 5.
Follow-up 1: after the above events, query (0, A, C)
    -> false, (5, A, C) -> true.
```

APPROACH

Have weighted/path-compressed union-find ready and warm. Practice the offline trick: segment tree over time intervals containing each edge, DFS the tree applying / rolling back unions.
Memorize the punchline for fully-dynamic: "online deletion breaks union-find; offline is OK; for online you need Holm–Lichtenberg or link-cut tree (O(polylog n))."

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the answer is for the base question (earliest time all N people are connected), what to return if it never happens, whether unfriend events exist, and whether queries ask about a past time.

BRUTE FORCE TO SAY OUT LOUD

For each query rebuild the graph from the events up to that time and run BFS. O(E + N) per query.

OPTIMAL IDEA

Base: sort events and union-find, return the time the component count hits 1. Query at a time t: union by size without path compression and record each union's time; two people are connected at t if the largest time on the path between them is at most t. Unfriend: offline, place each edge's lifetime into a segment tree over time and run DFS with a rollback union-find, checking connectivity at the leaves.

DRY-RUN THESE CASES

The example giving 5. A graph that never connects. Two events at the same time. Unfriend then refriend.

LIKELY FOLLOW-UPS

Online unfriend (Holm-de Lichtenberg-Thorup or link-cut trees), counting components over time.

PITFALLS

Using path compression where rollback is needed. Treating unfriend like union-find undo.

HOW TO THINK ABOUT IT

Everyone becomes connected at the first moment the graph has one component. If friendships only appear, sort by time and use union-find (components only merge). Unfriending breaks that: removing an edge can split a component and union-find cannot undo. With all events known up front, handle each edge's lifetime in a segment tree over time and use a rollback union-find.

Basis: LeetCode 1101, The Earliest Moment When Everyone Become Friends, plus connectivity queries over time and unfriend events (offline dynamic connectivity). Assumption: events are (time, a, b, 'friend'|'unfriend').

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, events):
    for t in sorted({e.time for e in events}):
        graph = build_graph(events, upto=t)
        if dfs_count(graph, start=0) == n:
            return t  # everyone reached
    return -1
```

TIME	SPACE
O(T * (N + E)) for T distinct times	**O(N + E)**

WHY IT IS SLOW

Each time point replays all events and searches the whole graph, O(T * (N + E)). A union-find over sorted events (friend-only) is near-linear; unfriend events need offline dynamic connectivity.

OPTIMAL SOLUTION · Base: earliest time everyone is connected (Union-Find on sorted events)

TIME	SPACE
O(E log E)	**O(N)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does unfriending break plain Union-Find?

A union merges two sets and forgets which edge caused it, so removing that edge cannot tell which members should separate. Online, fully dynamic connectivity needs heavy structures (Holm-de Lichtenberg-Thorup or link-cut trees, polylogarithmic time). When all events are known in advance, the offline segment-tree-over-time plus rollback DSU is the practical answer and the one to code in an interview.

FOLLOW-UP 2 What is the simple answer if the interviewer wants something quick?

Rebuild the graph from the events up to time t and run BFS or DFS for each query: O(E + N) per query. State the cost and then explain how the offline method amortizes it. A correct simple answer is accepted; naming the optimal one shows depth.

FOLLOW-UP 3 Report how many friend groups exist after each event (friends only).

Union-find with a component counter: decrement when a union merges two different roots; record the counter after each event. O(E alpha(N)).

FOLLOW-UP 4 Queries ask 'were A and B connected at time t?' with no unfriending.

Union by size without path compression, recording the time of each union on the edge it creates; two people are connected at t if the largest time on the path between them is at most t.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Friend Graph with Timeline (Friend /
# Unfriend Events)
#
# Examples: Base: N=3, events [(1,A,B,friend),
# (5,B,C,friend)] -> answer 5. Follow-up 1: after the abov
e
# events, query (0, A, C) -> false, (5, A, C) -> true.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does unfriending break plain Union-Find?
# 2. What is the simple answer if the interviewer wants
# something quick?
# 3. Report how many friend groups exist after each event
# (friends only).
# 4. Queries ask 'were A and B connected at time t?' with
# no unfriending.
#
# INTERVIEW NOTES
# Everyone becomes connected at the first moment the graph
# has one component. If friendships only appear, sort by
# time and use union-find (components only merge).
# Unfriending breaks that: removing an edge can split a
# component and union-find cannot undo. With all events
# known up front, handle each edge's lifetime in a segment
# tree over time and use a rollback union-find.
#
# BRUTE FORCE (say it first): Rebuild the graph at every
# event time and search it
# For each distinct time, replay all events up to it to
# get the current friend set and run a DFS from person 0
# to see whether everyone is reachable. Simple, correct
# for friend and unfriend events, and the oracle; it
# rebuilds the graph for every time. Time O(T * (N + E))
# for T distinct times. Space O(N + E).


from collections import defaultdict, deque


# BRUTE FORCE (say it first): Rebuild the graph at every
# event time and search it | time O(T * (N + E)) for T
# distinct times
def earliest_connected_rebuild(n, events):
    if n == 1:
        return 0
    for t in sorted({e[0] for e in events}):
        edges = set()
        for et, a, b, kind in sorted(
            events, key=lambda e: e[0]
        ):
            if et > t:
                break
            key = (min(a, b), max(a, b))
            (
                edges.add(key)
                if kind == "friend"
                else edges.discard(key)
            )
        seen, stack = {0}, [0]
        while stack:
            u = stack.pop()
            for a, b in edges:
                for x, y in ((a, b), (b, a)):
                    if x == u and y not in seen:
                        seen.add(y)
                        stack.append(y)
        if len(seen) == n:
            return t
    return -1


# HELPERS for the base question
def by_time(events):
    # events are (time, a, b, "friend" or "unfriend")
    return sorted(events, key=lambda e: e[0])


# FOLLOW-UP 4: Queries ask 'were A and B connected at time
# t?' with no unfriending.
# Answer: Union by size without path compression,
# recording the time of each union on the edge it creates;
# two people are connected at t if the largest time on the
# path between them is at most t.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class TimedUnionFind:
    """Union by size, no path compression, so every
    link keeps the time it was made."""

    def __init__(self, n):
        # every person starts alone in a group of one
        self.parent = list(range(n))
        # sizes decide which group hangs under which
        self.size = [1] * n
        # time a node was linked under its parent
        self.link_time = [0] * n
        # how many separate groups exist right now
        self.groups = n

    def find(self, x):
        # walk up to the leader of x's group
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def union(self, a, b, t):
        # find the leaders of both groups
        ra, rb = self.find(a), self.find(b)
        # already in one group: nothing changes
        if ra == rb:
            return False
        # make ra the bigger group
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        # hang the smaller group under the bigger one
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        # remember when this link appeared
        self.link_time[rb] = t
        # two groups became one
        self.groups -= 1
        return True

    def connected_at(self, a, b, t):
        # a person is always connected to themselves
        if a == b:
            return True
        # climb from a, noting the newest link
        seen, newest, x = {a: 0}, 0, a
```

```
        while self.parent[x] != x:
            newest = max(newest, self.link_time[x])
            x = self.parent[x]
            seen[x] = newest
        # climb from b until we reach an ancestor of a
        newest, y = 0, b
        while y not in seen:
```

```
            # b's leader reached without meeting a
            if self.parent[y] == y:
                return False
            newest = max(newest, self.link_time[y])
            y = self.parent[y]
        # connected at t if all path links are <= t
        return max(newest, seen[y]) <= t


# BASE QUESTION (optimal): earliest time everyone is
# connected (Union-Find on sorted events)
# Idea: Sort by time and union the two people for each
# friend event. The first time the component count reaches
# 1 is the answer; -1 if it never does.
def earliest_all_connected(n, events):
    # zero or one person is connected from the start
    if n <= 1:
        return 0
    uf = TimedUnionFind(n)
    # handle friendships in time order
    for t, a, b, kind in by_time(events):
        # the base question has friend events only
        if kind == "friend":
            uf.union(a, b, t)
            # one group left: everyone is connected now
            if uf.groups == 1:
                return t
    # the groups never merged into one
    return -1


# FOLLOW-UP 1: Why does unfriending break plain Union-
# Find?
# Answer: A union merges two sets and forgets which edge
# caused it, so removing that edge cannot tell which
# members should separate.
# FOLLOW-UP 2: What is the simple answer if the
# interviewer wants something quick?
# Answer: Rebuild the graph from the events up to time t
# and run BFS or DFS for each query: O(E + N) per query.
# State the cost and then explain how the offline method
# amortizes it.
def connected_with_unfriends(n, events, a, b, t):
    # replay every event up to time t on a set of edges
    edges = set()
```

```
    for time, x, y, kind in by_time(events):
        if time > t:
            break
        pair = (min(x, y), max(x, y))
        # friend adds the edge, unfriend removes it
        if kind == "friend":
            edges.add(pair)
        else:
            edges.discard(pair)
    # build neighbour lists from the surviving edges
    adj = defaultdict(list)
    for x, y in edges:
        adj[x].append(y)
        adj[y].append(x)
    # breadth-first search from a looking for b
    seen, queue = {a}, deque([a])
    while queue:
        u = queue.popleft()
        if u == b:
```

```
            return True
        for v in adj[u]:
            if v not in seen:
                seen.add(v)
                queue.append(v)
    return False


# FOLLOW-UP 3: Report how many friend groups exist after
# each event (friends only).
# Answer: Union-find with a component counter: decrement
# when a union merges two different roots; record the
# counter after each event. O(E alpha(N)).
def groups_after_each_event(n, events):
    uf = TimedUnionFind(n)
    counts = []
    # replay friend events in time order
    for t, a, b, kind in by_time(events):
        if kind == "friend":
            uf.union(a, b, t)
            # record the group count after this event
            counts.append(uf.groups)
    return counts


if __name__ == "__main__":
```

```
    # people A, B, C are 0, 1, 2
    ev = [(1, 0, 1, "friend"), (5, 1, 2, "friend")]
    # everyone is connected at time 5
    assert earliest_all_connected(3, ev) == 5
    # a missing friendship means never connected
    assert earliest_all_connected(3, ev[:1]) == -1
    assert earliest_all_connected(1, []) == 0
    # groups: 3 -> 2 -> 1
    assert groups_after_each_event(3, ev) == [2, 1]
    # A and C are apart at time 0, together at time 5
    uf = TimedUnionFind(3)
    for t, a, b, k in ev:
        uf.union(a, b, t)
    assert not uf.connected_at(0, 2, 4)
    assert uf.connected_at(0, 2, 5)
    # an unfriend breaks the link again
    ev2 = ev + [(7, 1, 2, "unfriend")]
    assert connected_with_unfriends(3, ev2, 0, 2, 6)
    assert not connected_with_unfriends(3, ev2, 0, 2, 8)
```

NOTES FROM CANDIDATES

The base problem looks innocent; the follow-up about unfriending is the actual signal. In practice, interviewers accept "rebuild graph from scratch per query + BFS" as a working answer if you also articulate why the optimal algorithm is fully dynamic connectivity.
The same family shows up as "friends form a chain over time" or "chess pieces R / L that can move plus walls" — the connectivity backbone is identical.
The follow-up the interviewer is fishing for is consistently a segment tree built over the event timeline; have the offline algorithm ready.
Two algorithmic families converge here: (1) offline — sort all events by time, run union-find, snapshot connectivity at each query T; (2) online with unfriend support — vanilla union-find doesn't handle deletion, so either rebuild per query, use link-cut trees, or apply offline reverse-time processing (treat unfriends as adds in reverse). Clarify with the interviewer which they want.
In an L4 R1 variant, the interviewer requested a graph-and-DFS implementation for the unfriend extension, then continued asking for an optimization that reused the earlier union-find work. Completing the rebuild approach did not end that follow-up.

PREPARATION

Have weighted/path-compressed union-find ready and warm.
Practice the offline trick: segment tree over time intervals containing each edge, DFS the tree applying / rolling back unions.
Memorize the punchline for fully-dynamic: "online deletion breaks union-find; offline is OK; for online you need Holm–Lichtenberg or link-cut tree (O(polylog n))."

HARD | 1Point3Acres | onsite-coding | 3 of 22

Maximum-Area Rectangle from Planar Points

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 177/228

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #963 Minimum Area Rectangle II

LeetCode match: LC 963 Minimum Area Free Rectangle (related)

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/max-area-rectangle-from-points

QUESTION

Given 2D points, find the maximum area of a rectangle whose four corners are input points; sides need not be parallel to the axes. Candidates compare it to LC 963 (Minimum Area Rectangle II) but maximizing. Interviewers ask to improve beyond an O(N^3) approach.

Input: a list of 2D coordinate points.
Return the maximum area of any rectangle whose four vertices are all input points.
Rectangle sides do not need to be parallel to the coordinate axes.
Follow-up pressure: start from an O(N^3) idea, then optimize toward a better complexity.

APPROACH

Practice the diagonal-grouping idea: two point pairs form a rectangle when they share the same midpoint and the same length, so bucket all pairs by (midpoint, squared length) and compare pairs within each bucket.
Write the area computation from two adjacent side vectors and verify it on both an axis-aligned and a 45-degree rotated rectangle.
Prepare the complexity argument: O(N^2) pairs to bucket, with the worst case driven by bucket sizes.

HOW TO THINK ABOUT IT

Four points form a rectangle exactly when two of its diagonals share the same midpoint and have the same length. So you never need to pick four points: bucket every pair of points by (sum of the two points, squared distance). Two different pairs in one bucket are the two diagonals of a rectangle, and the area is the cross product of two adjacent sides.

Basis: LeetCode 963, Minimum Area Free Rectangle, with the objective flipped to maximum area.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(points):
    best = 0
    for p, q, r in permutations(points, 3):
        if dot(q - p, r - p) != 0:
            continue  # no right angle at p
        s = q + r - p  # the fourth corner
        if s in points:
            area = abs(cross(q - p, r - p))
            best = max(best, area)
    return best
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

BRUTE FORCE

Three nested loops over the points, O(n^3). Bucketing pairs by their diagonal midpoint and length finds rectangles from O(n^2) pairs.

OPTIMAL SOLUTION · Bucket pairs by diagonal midpoint and length

TIME	SPACE
O(n^2) pairs (worst case O(n^3) if every pair shares a bucket)	**O(n^2)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Can you do better than O(n^2) pairs?

Not in general: the answer can depend on all pairs, and even deciding whether any rectangle exists is believed to need about n^2 work. The practical gain is in the buckets: most buckets hold one pair, so the work after bucketing is small. State the worst case honestly (points on a circle put many pairs in few buckets).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Maximum-Area Rectangle from Planar Points
#
# Given 2D points, find the maximum area of a rectangle
# whose four corners are input points; sides need not be
# parallel to the axes. Candidates compare it to LC 963
# (Minimum Area Rectangle II) but maximizing. Interviewers
# ask to improve beyond an O(N^3) approach. Input: a list
# of 2D coordinate points. Return the maximum area of any
# rectangle whose four vertices are all input points.
# Rectangle sides do not need to be parallel to the
# coordinate axes. Follow-up pressure: start from an
# O(N^3) idea, then optimize toward a better complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Can you do better than O(n^2) pairs?
#
# INTERVIEW NOTES
# Four points form a rectangle exactly when two of its
# diagonals share the same midpoint and have the same
# length. So you never need to pick four points: bucket
# every pair of points by (sum of the two points, squared
# distance). Two different pairs in one bucket are the two
# diagonals of a rectangle, and the area is the cross
# product of two adjacent sides.
#
# BRUTE FORCE (say it first): Try every triple with a
# right angle
# Pick a corner p and two neighbours q and r with (q - p)
# . (r - p) = 0, then check that the fourth point q + r -
# p exists. Correct and the first thing to say; cubic is
# too slow for thousands of points. Time O(n^3). Space
# O(n).


from collections import defaultdict
from itertools import combinations
```

```
# HELPERS for the brute force
def cross(o, p, q):
    # twice the signed area of triangle o, p, q
    ax, ay = p[0] - o[0], p[1] - o[1]
    bx, by = q[0] - o[0], q[1] - o[1]
    return ax * by - ay * bx
```

```
# BRUTE FORCE (say it first)
def max_rect_area_cubic(points):
    # first idea to say aloud: try every corner triple
    pts = set(map(tuple, points))
    best = 0
    for p in pts:
        for q in pts:
            for r in pts:
                if len({p, q, r}) < 3:
                    continue
                # right angle at p between p->q and p->r
                dot = ((q[0] - p[0]) * (r[0] - p[0])
                       + (q[1] - p[1]) * (r[1] - p[1]))
                # the fourth corner must also exist
                s = (q[0] + r[0] - p[0],
                     q[1] + r[1] - p[1])
                if dot == 0 and s in pts:
                    area = abs(cross(p, q, r))
                    best = max(best, area)
    return best


# HELPERS for the base question
# FOLLOW-UP 1: Can you do better than O(n^2) pairs?
# Answer: Not in general: the answer can depend on all
# pairs, and even deciding whether any rectangle exists is
# believed to need about n^2 work.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def all_rectangle_areas(points):
    # duplicates cannot help, so drop them
    pts = list(set(map(tuple, points)))
    buckets = defaultdict(list)
    # every pair is a possible diagonal
    for p, q in combinations(pts, 2):
        # same centre (sum of corners) and same length
        key = (p[0] + q[0], p[1] + q[1],
               (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)
        buckets[key].append((p, q))
    # two diagonals in one bucket form a rectangle
    for pairs in buckets.values():
        for (p, q), (r, s) in combinations(pairs, 2):
            # area = |cross| of two sides leaving p
            yield abs(cross(p, r, s))
```

```
# BASE QUESTION (optimal): Bucket pairs by diagonal
# midpoint and length
# Idea: For every pair (p, q) use the key (p.x + q.x, p.y
# + q.y, squared length). All pairs with the same key are
# diagonals of one circle around the same centre, so any
# two of them form a rectangle.
def max_rect_area(points):
    # best area, or 0 when no rectangle exists
    return max(all_rectangle_areas(points), default=0)


# RELATED VARIANT: smallest area (LeetCode 963)
def min_rect_area(points):
    # LeetCode 963 asks for the smallest instead
    return min(all_rectangle_areas(points), default=0)
```

```python
if __name__ == "__main__":
    # a tilted square of area 2
    sq = [(0, 0), (1, 1), (0, 2), (-1, 1)]
    assert max_rect_area(sq) == 2
    assert max_rect_area_cubic(sq) == 2
    # axis-aligned 2 x 1 rectangle plus a stray point
    rect = [(0, 0), (2, 0), (0, 1), (2, 1), (5, 5)]
    assert max_rect_area(rect) == 2
    # collinear points make no rectangle
    assert max_rect_area([(0, 0), (1, 1), (2, 2)]) == 0
    # the minimum version picks the small one
    two = rect + [(0, 3), (2, 3)]
    assert min_rect_area(two) == 2
    assert max_rect_area(two) == 6
```

NOTES FROM CANDIDATES

Candidates describe it as LC 963 (Minimum Area Rectangle II) with the objective flipped to maximum area.
Asked as a standalone L4 onsite coding round; in another L4 onsite it was the second problem after a CPU scheduling question, and the interviewer explicitly expected a better-than-cubic solution.
Clarify up front: whether duplicate points can appear, what to return when no rectangle exists (commonly 0), and whether integer or floating-point coordinates are used (area precision matters for rotated rectangles).

PREPARATION

Practice the diagonal-grouping idea: two point pairs form a rectangle when they share the same midpoint and the same length, so bucket all pairs by (midpoint, squared length) and compare pairs within each bucket.
Write the area computation from two adjacent side vectors and verify it on both an axis-aligned and a 45-degree rotated rectangle.
Prepare the complexity argument: O(N^2) pairs to bucket, with the worst case driven by bucket sizes.

Medium (17)

MEDIUM | 1Point3Acres | onsite-coding | 4 of 22

Template Variable Expansion with Follow-Ups

ASKED 2026-10-17 | 3 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 214/228

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-17 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/template-variable-expansion-dependencies

QUESTION

Replace **%KEY%** placeholders using a key-value map whose values may contain other placeholders. Clarify malformed delimiters and literal-percent escaping, flatten nested values, and detect cyclic dependencies before expansion.

Input: a key-value map and a string containing placeholders delimited by %, such as %USER%. Replace every placeholder with the corresponding value.
Map values can themselves contain placeholders, so resolve or flatten dependencies before expanding the final string.
Define behavior for malformed delimiters such as unmatched percent signs, for a literal % in output, and for nested-looking forms before coding; the prompt intentionally leaves these contracts open.
Follow-up: detect cyclic dependencies among map values.
Model keys as a dependency graph and use indegrees plus a queue to determine whether a complete topological resolution is possible.

EXAMPLES

```
Given {USER: "admin", HOME: "/%USER%/home"}:
```

Input: I am %USER% My home is %HOME% Output: I am admin My home is /admin/home

APPROACH

Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

HOW TO THINK ABOUT IT

Agree the grammar first: %NAME% is a placeholder, %% is a literal percent sign, an unmatched % is an error and an unknown name is an error. Values may contain placeholders themselves, so expand a value by expanding its own placeholders first. A cycle (A uses B, B uses A) would recurse forever, so detect it with a visiting set or, equivalently, with indegrees and a queue.

Basis: Written for this question; resolving values that contain other placeholders is a dependency graph problem (as in LeetCode 210, Course Schedule II).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def expand(text, depth=0):
    if depth > LIMIT:
        raise Error("too deep: cycle?")
    out = ""
    for token in tokens(text):
        if token == "%%":
            out += "%"
        elif is_key(token):
            value = values[name_of(token)]
            out += expand(value, depth + 1)
        else:
            out += token
    return out
```

TIME	SPACE
exponential in the worst ... exponential in the worst case	**O(depth)**

WHY IT IS SLOW

Re-expands shared values every time and finds cycles only by running out of depth. Memoising each value and keeping a visiting set expands each key once and reports a cycle at once.

OPTIMAL SOLUTION · Depth-first expansion with memo and a visiting set

TIME	SPACE
O(total length of the expanded text + keys)	**O(keys)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Detect cycles before expanding.

Model each key as a vertex with an edge to every key its value mentions, count indegrees and run Kahn's algorithm: if fewer vertices than keys come out, some keys sit on a cycle and cannot be resolved. This reports the problem up front instead of failing halfway through a long document.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Template Variable Expansion with Follow-
# Ups
#
# Replace %KEY% placeholders using a key-value map whose
# values may contain other placeholders. Clarify malformed
# delimiters and literal-percent escaping, flatten nested
# values, and detect cyclic dependencies before expansion.
# Input: a key-value map and a string containing
# placeholders delimited by %, such as %USER%. Replace
# every placeholder with the corresponding value. Map
# values can themselves contain placeholders, so resolve
# or flatten dependencies before expanding the final
# string. Define behavior for malformed delimiters such as
# unmatched percent signs, for a literal % in output, and
# for nested-looking forms before coding; the prompt
# intentionally leaves these contracts open. Follow-up:
# detect cyclic dependencies among map values. Model keys
# as a dependency graph and use indegrees plus a queue to
# determine whether a complete topological resolution is
# possible.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Detect cycles before expanding.
#
# INTERVIEW NOTES
# Agree the grammar first: %NAME% is a placeholder, %% is
# a literal percent sign, an unmatched % is an error and
# an unknown name is an error. Values may contain
# placeholders themselves, so expand a value by expanding
# its own placeholders first. A cycle (A uses B, B uses A)
# would recurse forever, so detect it with a visiting set
# or, equivalently, with indegrees and a queue.
#
# BRUTE FORCE (say it first): Recursive expansion without
# memo or cycle check, capped by depth
# Recursively substitute placeholders and stop with an
# error after a fixed depth. Correct for acyclic maps;
# repeats shared work and detects cycles only by running
# out of depth. Time exponential in the worst case. Space
# O(depth).


from collections import defaultdict, deque
```

```
# BRUTE FORCE (say it first): Recursive expansion without
# memo or cycle check, capped by depth | time exponential
# in the worst case
def expand_brute(text, mapping, depth=0):
    if depth > 30:
        raise ValueError("cycle")
    out, i = [], 0
    while i < len(text):
        if text[i] != "%":
            out.append(text[i])
            i += 1
        elif text[i : i + 2] == "%%":
            out.append("%")
            i += 2
        else:
            j = text.find("%", i + 1)
            if j < 0:
                raise ValueError("unmatched %")
            out.append(
                expand_brute(
                    mapping[text[i + 1 : j]],
                    mapping,
                    depth + 1,
                )
            )
            i = j + 1
    return "".join(out)


# HELPERS for the base question
def tokens(s):
    # split text into ("text", char) and ("var", name)
    i = 0
    while i < len(s):
        if s[i] != "%":
            yield "text", s[i]
            i += 1
        elif s[i:i + 2] == "%%":
            # %% is a literal percent sign
            yield "text", "%"
            i += 2
        else:
            j = s.find("%", i + 1)
```

```
            if j < 0:
                raise ValueError("unmatched %")
```

```
            yield "var", s[i + 1:j]
            i = j + 1


# BASE QUESTION (optimal): Depth-first expansion with memo
# and a visiting set
# Idea: expand(text) scans the text once: %% becomes %,
# %NAME% becomes the resolved value of NAME.
def expand_template(text, mapping):
    done, visiting = {}, set()

    def resolve(name):
        # values may hold placeholders: expand them first
        if name in done:
            return done[name]
        if name in visiting:
            raise ValueError("cycle at " + name)
        if name not in mapping:
            raise KeyError(name)
        visiting.add(name)
        done[name] = scan(mapping[name])
        visiting.discard(name)
        return done[name]

    def scan(s):
        return "".join(resolve(x) if kind == "var" else x
                       for kind, x in tokens(s))

    return scan(text)


# FOLLOW-UP 1: Detect cycles before expanding.
# Answer: Model each key as a vertex with an edge to every
# key its value mentions, count indegrees and run Kahn's
# algorithm: if fewer vertices than keys come out, some
# keys sit on a cycle and cannot be resolved.
def has_cycle(mapping):
    # Kahn's algorithm over key -> keys it mentions
    deps = {}
    for k, v in mapping.items():
        deps[k] = {x for kind, x in tokens(v)
                   if kind == "var"}
    users, indeg = defaultdict(list), {}
    for k, d in deps.items():
        known = d & mapping.keys()
```

```
        indeg[k] = len(known)
        for x in known:
            users[x].append(k)
    queue = deque(k for k, n in indeg.items() if n == 0)
    resolved = 0
    while queue:
        k = queue.popleft()
        resolved += 1
        for u in users[k]:
            indeg[u] -= 1
            if indeg[u] == 0:
                queue.append(u)
    # keys left over sit on a cycle
    return resolved != len(mapping)


if __name__ == "__main__":
    m = {"USER": "admin", "HOME": "/%USER%/home"}
    text = "I am %USER% My home is %HOME%"
    assert expand_template(text, m) == (
        "I am admin My home is /admin/home")
    sure = expand_template("100%% sure", {})
    assert sure == "100% sure"
    assert not has_cycle(m)
```

```
    loop = {"A": "%B%", "B": "%A%"}
    assert has_cycle(loop)
    for bad, error in (("a % b", ValueError),
                       ("%NOPE%", KeyError)):
        try:
            expand_template(bad, {})
            raise AssertionError("should have failed")
        except error:
            pass
    try:
        expand_template("%A%", loop)
        raise AssertionError("cycle not caught")
    except ValueError:
        pass
```

NOTES FROM CANDIDATES

The interviewer spent most of the round probing the delimiter and escaping contract before allowing implementation. State a compact grammar and get explicit agreement; otherwise a nominally easy parser can lose the coding budget to unresolved edge cases.

PREPARATION

Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

MEDIUM | 1Point3Acres | tech-screen | 5 of 22

Bounded Knapsack Variant

ASKED 2026-10-16 | 4 DAYS AGO | LAST 30 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 180/228

ABOUT THIS QUESTION

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1710 Maximum Units on a Truck

Frequency: Single report | Last asked: 2026-10-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/unit-weight-bounded-knapsack

QUESTION

Each item type has a value and an available count; every item weighs 1. Given capacity W, maximize total value. DP or DFS + memo works at O(n·W), greedy by value after sorting is O(n log n), and the best answer uses quickselect for O(n).

QUESTION

Input: for each item type, its value and the number of copies available. Every item has weight 1.
Input: a maximum total weight W.
Output: the maximum total value achievable without exceeding W.

APPROACH

Write the sort-based greedy and argue its optimality in two sentences (unit weights mean every slot is interchangeable). Implement quickselect that partitions item types by value and tracks the total count on each side to locate where W runs out. Compare the O(n × W), O(n log n), and O(n) versions out loud with their complexities.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether every item really weighs 1, whether values can be zero or negative (assume non-negative), whether W can exceed the total number of items, and how large the counts and W can get.

BRUTE FORCE TO SAY OUT LOUD

A knapsack table over the capacity, one pass per copy. O(total copies * W), correct for any weights.

OPTIMAL IDEA

With unit weights every slot is interchangeable, so sort the item types by value, highest first, and take min(copies, slots left) of each until the bag is full. O(n log n). To reach O(n), run quickselect on the value: split the types into higher, equal and lower, recurse into the side that contains the W-th slot, and keep a running total of what was taken.

DRY-RUN THESE CASES

W = 0, W larger than all copies together, one type, equal values, a type with a huge count, all values equal.

LIKELY FOLLOW-UPS

Different weights (bounded knapsack with binary splitting), a huge W (greedy still works), or negative values (never take them).

PITFALLS

Reaching for DP because the word knapsack appears. Forgetting that a type may only be partly taken. Letting the remaining capacity go negative.

HOW TO THINK ABOUT IT

The word knapsack points at DP, but the statement says every item weighs 1. With equal weights every slot in the bag is interchangeable, so an item that is worth more is never worse than one that is worth less: swapping a cheaper item for a dearer one always helps. Take item types by value, highest first, until W slots are used. You do not even need the full sort: quickselect finds where the W-th slot falls in O(n).

HOW TO THINK ABOUT IT

Basis: LeetCode 1710, Maximum Units on a Truck: every item weighs 1, so the knapsack collapses to 'take the most valuable items first'.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def knapsack(items, W):
    best = [0] * (W + 1)
    for value, count in items:
        for _ in range(count):
            for w in range(W, 0, -1):
                take = best[w - 1] + value
                best[w] = max(best[w], take)
    return best[W]
```

TIME	SPACE
O(total copies * W)	O(W)

WHY IT IS SLOW

Every copy is its own pass over the table, O(total copies * W). With unit weights the most valuable items are always the right ones, so sorting by value and taking greedily is O(n log n).

OPTIMAL SOLUTION · Sort by value and take greedily

TIME	SPACE
O(n log n)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is greedy correct here but not for a general knapsack?

With unit weights each chosen item uses exactly one slot, so the only thing that matters is value per slot, which is just the value. If an optimal bag held a cheaper item while a dearer one was available, swapping them keeps the weight and raises the value, so the optimal bag can always be rearranged into the greedy one. With different weights a cheap item can leave room for several others, so value per weight and the leftover space interact and greedy fails.

FOLLOW-UP 2 What if the weights are not all 1?

Then it is a real bounded knapsack and greedy no longer works; use DP over the capacity. Splitting each count into 1, 2, 4, ... copies keeps the number of 0/1 items at about log(count) per type, so the cost is O(n * log(max count) * W) instead of O(total copies * W).

FOLLOW-UP 3 W is up to 10^9. Which version still works?

Only the greedy ones: sorting is O(n log n) and quickselect O(n), and neither depends on W, while any table over the capacity needs O(W) memory. The loop simply stops when the remaining capacity reaches zero.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Bounded Knapsack Variant
#
# Each item type has a value and an available count; every
# item weighs 1. Given capacity W, maximize total value.
# DP or DFS + memo works at O(n·W), greedy by value after
# sorting is O(n log n), and the best answer uses
# quickselect for O(n). Input: for each item type, its
# value and the number of copies available. Every item has
# weight 1. Input: a maximum total weight W. Output: the
# maximum total value achievable without exceeding W.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is greedy correct here but not for a general
# knapsack?
# 2. What if the weights are not all 1?
# 3. W is up to 10^9. Which version still works?
#
# INTERVIEW NOTES
# The word knapsack points at DP, but the statement says
# every item weighs 1. With equal weights every slot in
# the bag is interchangeable, so an item that is worth
# more is never worse than one that is worth less:
# swapping a cheaper item for a dearer one always helps.
# Take item types by value, highest first, until W slots
# are used. You do not even need the full sort:
# quickselect finds where the W-th slot falls in O(n).
#
# BRUTE FORCE (say it first): DP over the capacity
# Treat every copy as its own 0/1 item and run the classic
# 1D knapsack table. It is correct for any weights, so it
# is the oracle, but it is far too slow when the counts or
# W are large. Time O(total copies * W). Space O(W).


import random


# BRUTE FORCE (say it first, it also checks the answers)
def knapsack_dp(items, W):
    # the oracle: every copy is its own 0/1 item
    best = [0] * (W + 1)
    for value, count in items:
        for _ in range(count):
            for w in range(W, 0, -1):
                take = best[w - 1] + value
                best[w] = max(best[w], take)
    return best[W]


# BASE QUESTION (optimal): Sort by value and take greedily
# Idea: Sort the item types by value, highest first. For
# each type take min(copies, slots left) and add value
# times that. Stop when the bag is full.
def max_value_sorted(items, W):
    """Unit weights: every slot is worth the same, so the
    best value always goes in first."""
    total = 0
    for value, count in sorted(items, reverse=True):
        # take as many copies as still fit in the bag
        take = min(count, W)
        total += value * take
        W -= take
        if W == 0:
            break
    return total


# SAME answer in O(n): quickselect
def max_value_select(items, W):
    """Same answer in O(n) expected: quickselect finds
    where the W-th slot falls without a full sort."""
    total, pool = 0, list(items)
    while pool and W > 0:
        pivot = random.choice(pool)[0]
        high = [it for it in pool if it[0] > pivot]
        mid = [it for it in pool if it[0] == pivot]
        low = [it for it in pool if it[0] < pivot]
        high_count = sum(c for _, c in high)
        if high_count >= W:
            # the bag fills up inside the dearer types
            pool = high
            continue
        # every dearer copy fits, then fill from the pivot
        total += sum(v * c for v, c in high)
        W -= high_count
        take = min(W, sum(c for _, c in mid))
        total += pivot * take
        W -= take
        pool = low
    return total


# FOLLOW-UP 2: What if the weights are not all 1?
# Answer: Then it is a real bounded knapsack and greedy no
# longer works; use DP over the capacity. Splitting each
# count into 1, 2, 4, ...
def knapsack_bounded(items, W):
    """Weights differ: greedy fails, so use DP. Splitting
    each count into 1, 2, 4, ... copies keeps it small."""
    best = [0] * (W + 1)
    for value, weight, count in items:
        chunk = 1
        while count > 0:
            take = min(chunk, count)
            count -= take
            cost, gain = take * weight, take * value
            for w in range(W, cost - 1, -1):
                best[w] = max(best[w],
                              best[w - cost] + gain)
            chunk *= 2
    return best[W]


if __name__ == "__main__":
    demo = [(3, 2), (5, 1), (4, 4)]
    assert max_value_sorted(demo, 4) == 17
    assert max_value_select(demo, 4) == 17
    assert max_value_sorted([], 5) == 0
    assert max_value_sorted([(7, 3)], 0) == 0
    assert max_value_sorted([(7, 3)], 10) == 21
    mixed = [(10, 5, 1), (6, 3, 2)]
    assert knapsack_bounded(mixed, 6) == 12
    # greedy is only valid when every weight is 1
    unit = [(3, 1, 2), (5, 1, 1), (4, 1, 4)]
    assert knapsack_bounded(unit, 4) == 17
    for _ in range(300):
        n = random.randint(0, 6)
        rnd = random.randint
        items = [(rnd(1, 9), rnd(1, 4)) for _ in range(n)]
```

```
        W = random.randint(0, 12)
        want = knapsack_dp(items, W)
        assert max_value_sorted(items, W) == want
        assert max_value_select(items, W) == want
```

NOTES FROM CANDIDATES

Pitfalls: Recognizing "knapsack" and reaching for DP is the trap; with unit weights the greedy argument is simpler and faster.

PREPARATION

Write the sort-based greedy and argue its optimality in two sentences (unit weights mean every slot is interchangeable). Implement quickselect that partitions item types by value and tracks the total count on each side to locate where W runs out. Compare the O(n × W), O(n log n), and O(n) versions out loud with their complexities.

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | phone-screen, onsite-coding | 6 of 22

URL Request Handler Router

ASKED 2026-10-16 | 4 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 222/228**

ABOUT THIS QUESTION

Topic: Trie and Prefix Indexing

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/url-request-handler-router

QUESTION

EXAMPLES

```
Register /users/{userId}/photo/{photoId}. A lookup
    for /users/123/photo/456 matches that registered
    handler.
```

APPROACH

In 15 minutes, implement the scan baseline and check the existing example, segment-count mismatches, and no-match cases under an explicitly chosen return convention.
In 25 minutes, build the segment trie and compare its complete match set with the scan on small generated inputs. Include a literal branch that fails later and patterns with different placeholder names but the same structure.
In 10 minutes, write a contract checklist for precedence, duplicate registration, trailing slashes, query strings, and URL decoding; separate agreed rules from assumptions in your tests.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how to choose between several matching patterns (literal over placeholder?), what to return on no match or duplicate registration, and how to treat trailing slashes, query strings and case.

BRUTE FORCE TO SAY OUT LOUD

Store the patterns in a list and compare segment by segment. O(H * L) per lookup. It is a correct baseline and gives a reference to test the trie against.

OPTIMAL IDEA

Segment trie: each node has literal children and one wildcard child, with the original pattern stored at the end node. Lookup explores the literal child first and the wildcard child on failure, because a literal branch can die later. Accept only when all segments are consumed and a handler is stored.

DRY-RUN THESE CASES

An exact route, a placeholder route, both literal and placeholder matching the same URL, segment-count mismatch, no match.

LIKELY FOLLOW-UPS

Methods (GET and POST), wildcard at the end, priority rules, and many routes.

PITFALLS

Picking the literal branch greedily without falling back. Overwriting patterns that differ only in placeholder names.

HOW TO THINK ABOUT IT

Matching a path against many patterns is a prefix problem, so share the prefixes in a trie of segments. The catch is the wildcard: a literal branch can fail later, so you cannot pick it greedily. Try the literal child first and fall back to the wildcard child. That also encodes the precedence rule for free.

Basis: Written. Assumed rules (confirm them): patterns look like /users/{id}/photo; {name} matches exactly one segment; a literal beats a placeholder at the first segment where they differ; no match returns None; trailing slashes are ignored.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def lookup(url):
    parts = url.split("/")
    matches = []
    for pattern in patterns:
        p = pattern.split("/")
        if len(p) != len(parts):
            continue
        if all(is_param(x) or x == y
               for x, y in zip(p, parts)):
            matches.append(pattern)
    return best_by_precedence(matches)
```

TIME	SPACE
O(H * L)	**O(1)**
per lookup	extra

BRUTE FORCE

WHY IT IS SLOW

Each lookup compares against every registered pattern, O(patterns * segments). A trie of segments with a literal child and a wildcard child follows only the compatible branches.

OPTIMAL SOLUTION · Segment trie with literal-first fallback

TIME	SPACE
O(L * branching) per lookup, O(total pattern length) to build	**O(total pattern length)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Different HTTP methods per route.

Make the method the first segment of the path (a tiny trick) or keep a dict method to trie. Lookup uses the method as the first step.

FOLLOW-UP 2 A wildcard at the end that matches the rest of the path (/static/*).

Give each trie node a 'rest' handler. During lookup, remember the deepest node that has one and use it if the exact walk fails. Precedence: exact, then placeholder, then rest-wildcard.

FOLLOW-UP 3 Return the extracted parameters (id = 7).

Carry a list of (name, segment) while walking; append when you take a wildcard edge and drop it when you backtrack. Store the placeholder names with the pattern, not in the trie.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: URL Request Handler Router
#
# Examples: Register /users/{userId}/photo/{photoId}. A
# lookup for /users/123/photo/456 matches that registered
# handler.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Different HTTP methods per route.
# 2. A wildcard at the end that matches the rest of the
# path (/static/*).
# 3. Return the extracted parameters (id = 7).
#
# INTERVIEW NOTES
# Matching a path against many patterns is a prefix
# problem, so share the prefixes in a trie of segments.
# The catch is the wildcard: a literal branch can fail
# later, so you cannot pick it greedily. Try the literal
# child first and fall back to the wildcard child. That
# also encodes the precedence rule for free.
#
# BRUTE FORCE (say it first): Scan every registered
# pattern
# Compare the URL's segments with each pattern's segments
# (same count, each one equal or a placeholder), collect
# the matches and keep the best by specificity. It is the
# correctness baseline, and a good oracle for the trie.
# Time O(H * L) per lookup. Space O(1) extra.


# HELPERS for the brute force
def segments(path):
    # "/users/7/photo" -> ["users", "7", "photo"]
    return [s for s in path.split("/") if s]


# BRUTE FORCE (say it first): try every route in a list
def fits(pat, segs):
    # the placeholder values if the route fits, else None
    params = {}
    for i, p in enumerate(pat):
        if p == "*":
            return params if len(segs) > i else None
        if i >= len(segs):
            return None
        if p.startswith("{") and p.endswith("}"):
            params[p[1:-1]] = segs[i]
        elif p != segs[i]:
            return None
    return params if len(pat) == len(segs) else None


class RouterBrute:
    def __init__(self):
        self.routes = []

    def add(self, pattern, handler=None, method="GET"):
        pat = [method] + segments(pattern)
        self.routes.append((pat, handler or pattern))

    def match(self, url, method="GET"):
        segs = [method] + segments(url)
        best = None
        for pat, handler in self.routes:
            params = fits(pat, segs)
            if params is None:
                continue
            # literal beats placeholder beats trailing *
            score = [2 if p == s else 1 if p[0] == "{"
                     else 0 for p, s in zip(pat, segs)]
            if best is None or score >= best[0]:
                best = (score, handler, params)
        return (best[1], best[2]) if best else (None, {})


# HELPERS for the base question
class Node:
    def __init__(self):
        # fixed segment -> child node
        self.literal = {}
        # child for a {placeholder} segment
        self.wild = None
        self.wild_name = None
        # handler when the path ends here
        self.handler = None
        # handler for a trailing "*" (rest of the path)
        self.rest = None


# BASE QUESTION (optimal): Segment trie with literal-first
# fallback
# Idea: Each trie node has literal children and one
# wildcard child; the pattern is stored at its last node.
# Lookup tries the literal child first, and if that whole
```

```
# branch fails it backs up and tries the wildcard child.
# FOLLOW-UP 1: Different HTTP methods per route.
# Answer: Make the method the first segment of the path (a
# tiny trick) or keep a dict method to trie. Lookup uses
# the method as the first step.
class Router:
    def __init__(self):
        self.root = Node()

    def add(self, pattern, handler=None, method="GET"):
        node = self.root
        # FOLLOW-UP 1: the HTTP method is the first
        # segment
        # the method is simply the first segment
        for seg in [method] + segments(pattern):
            # FOLLOW-UP 2: a trailing * matches the rest
            # of the path
            if seg == "*":
                # all deeper paths use this handler
                node.rest = handler or pattern
                return
            if seg.startswith("{") and seg.endswith("}"):
                node.wild = node.wild or Node()
                node.wild_name = seg[1:-1]
                node = node.wild
            else:
                lit = node.literal
                node = lit.setdefault(seg, Node())
        node.handler = handler or pattern

    def match(self, url, method="GET"):
        segs = [method] + segments(url)
        params = {}

        def walk(node, i):
            if i == len(segs):
                return node.handler
            # 1) exact segment first
            child = node.literal.get(segs[i])
            if child is not None:
                found = walk(child, i + 1)
                if found is not None:
```

```
                    return found
            # 2) then a placeholder, remembering its value
            if node.wild is not None:
                # FOLLOW-UP 3: remember the value (id = 7)
                params[node.wild_name] = segs[i]
                found = walk(node.wild, i + 1)
                if found is not None:
                    return found
                # undo the value when backtracking
                params.pop(node.wild_name, None)
            # 3) finally a trailing wildcard
            return node.rest

        handler = walk(self.root, 0)
        return handler, dict(params)


if __name__ == "__main__":
    r = Router()
    r.add("/users/{userId}/photo/{photoId}", "photo")
    r.add("/users/me", "me")
    r.add("/static/*", "static")
    r.add("/orders/{id}", "create", method="POST")
    hit = r.match("/users/123/photo/456")
    ids = {"userId": "123", "photoId": "456"}
    assert hit == ("photo", ids)
    # a literal beats a placeholder
    assert r.match("/users/me")[0] == "me"
    assert r.match("/users/me/photo/9")[0] == "photo"
    # trailing wildcard takes the rest
```

```
    assert r.match("/static/css/app.css")[0] == "static"
    # different method, different route
    post = r.match("/orders/5", "POST")
    assert post == ("create", {"id": "5"})
    assert r.match("/orders/5")[0] is None
    # the brute force agrees with the trie
    import random
    words = ["a", "b", "{}", "*"]
    for _ in range(300):
        fast, slow = Router(), RouterBrute()
        for k in range(random.randint(1, 5)):
            n = random.randint(1, 3)
            parts = [random.choice(words)
                     for _ in range(n)]
            if "*" in parts:
```

```
                parts = parts[:parts.index("*") + 1]
            # a placeholder is named after its position
            parts = ["{p%d}" % i if p == "{}" else p
                     for i, p in enumerate(parts)]
            pattern = "/" + "/".join(parts)
            fast.add(pattern, f"h{k}")
            slow.add(pattern, f"h{k}")
        for _ in range(8):
            n = random.randint(1, 3)
            url = "/" + "/".join(random.choice("abc")
                                 for _ in range(n))
            assert fast.match(url) == slow.match(url), url
```

NOTES FROM CANDIDATES

Clarify how to choose between multiple matching handlers, including a literal segment competing with a placeholder. The precedence rule is unspecified.

The no-match result, duplicate-registration behavior, and URL normalization rules are also unspecified; do not assume a canonical behavior before discussing them.

Matching approach

Start with a scan of registered patterns: compare segment counts, then require each position to contain either the same literal or a placeholder. This is a correctness baseline that can collect all matches before applying the agreed precedence rule.

A segment trie shares prefixes. Keep literal children and a separate wildcard child; at each URL segment explore both compatible branches. A literal branch can fail later, so choosing it greedily can miss a valid wildcard match.

Store the original registered patterns at terminal nodes. Patterns with different placeholder names can share the same trie path; retain each original rather than overwriting it. Accept terminals only after consuming the entire URL.

For tokenized paths of L segments and H handlers, the scan costs O(HL) segment comparisons in the worst case. A wildcard trie can branch, so lookup is not necessarily O(L): account for visited states, matching candidates, and string-processing costs separately. Keep normalization

and winner selection tied to the clarified contract.

PREPARATION

In 15 minutes, implement the scan baseline and check the existing example, segment-count mismatches, and no-match cases under an explicitly chosen return convention.
In 25 minutes, build the segment trie and compare its complete match set with the scan on small generated inputs. Include a literal branch that fails later and patterns with different placeholder names but the same structure.
In 10 minutes, write a contract checklist for precedence, duplicate registration, trailing slashes, query strings, and URL decoding; separate agreed rules from assumptions in your tests.

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 7 of 22

Mahjong Winning Hand Detection (14 Tiles)

ASKED 2026-10-09 | 11 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 210/228

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/mahjong-winning-hand-check

QUESTION

Mountain View onsite coding round 1: given a hand of 14 tiles, decide whether it is a winning mahjong hand. The candidate must clarify the rule set (typically: 4 melds of three identical tiles or three consecutive ranks + 1 pair).

Input: array of 14 integers representing tile values (clarify suit / range during the interview).
A winning hand consists of:

One pair (two identical tiles), and
Four melds, each being either:

A triplet (3 identical tiles), or
A run (3 consecutive ranks in the same suit).

Minority variant: A newer version instead specifies one pair plus two melds while retaining a 14-tile input. Because that accounts for only eight tiles, clarify the intended meld count and how the remaining six tiles should be handled before coding.

Output: boolean.

APPROACH

Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

HOW TO THINK ABOUT IT

A winning hand is one pair plus four melds (a triplet of identical tiles or a run of three consecutive ranks in one suit). Try every tile as the pair, then repeatedly take the smallest remaining tile: it can only be covered by a triplet of itself or by a run that starts at it, so there are at most two choices at each step and the search is tiny for 14 tiles.

Basis: Written for this question; the pair-plus-melds check is the same backtracking as LeetCode 846, Hand of Straights, plus a triplet option.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_win(tiles):
    for t in set(tiles):
        if tiles.count(t) >= 2:
            rest = remove(tiles, [t, t])
            if all_melds(rest):
                return True
    return False
def all_melds(tiles):
    if not tiles:
        return True
    first = min(tiles)
    for meld in (triple(first), run(first)):
        if contains(tiles, meld):
            if all_melds(remove(tiles, meld)):
                return True
    return False
```

TIME	SPACE
roughly C(12,3) * C(9,3) ... roughly C(12,3) * C(9,3) * C(6,3) per pair	**O(1)**

WHY IT IS SLOW

Tries every set of three tiles at every step. The smallest remaining tile can only be in a triplet of itself or a run starting at it, so at most two choices remain.

OPTIMAL SOLUTION · Try each pair, then greedy-smallest with two options

TIME	SPACE
O(1) for 14 tiles (about 2^4 branches per pair)	**O(distinct tiles)**

OPTIMAL SOLUTION · Try each pair, then greedy-smallest with two options

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A variant needs only two melds plus a pair (8 tiles of 14).

Ask what happens to the other six tiles before coding: if they are free, check whether any 8 tiles form a pair and two melds with the same search over subsets of the 14. The rule set is intentionally vague, so confirm melds, honours and flowers first.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Mahjong Winning Hand Detection (14 Tiles)
#
# Mountain View onsite coding round 1: given a hand of 14
# tiles, decide whether it is a winning mahjong hand. The
# candidate must clarify the rule set (typically: 4 melds
# of three identical tiles or three consecutive ranks + 1
# pair). Input: array of 14 integers representing tile
# values (clarify suit / range during the interview). A
# winning hand consists of: One pair (two identical
# tiles), and Four melds, each being either: A triplet (3
# identical tiles), or A run (3 consecutive ranks in the
# same suit). Minority variant: A newer version instead
# specifies one pair plus two melds while retaining a
# 14-tile input. Because that accounts for only eight
# tiles, clarify the intended meld count and how the
# remaining six tiles should be handled before coding.
# Output: boolean.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A variant needs only two melds plus a pair (8 tiles
# of 14).
#
# INTERVIEW NOTES
# A winning hand is one pair plus four melds (a triplet of
# identical tiles or a run of three consecutive ranks in
# one suit). Try every tile as the pair, then repeatedly
# take the smallest remaining tile: it can only be covered
# by a triplet of itself or by a run that starts at it, so
# there are at most two choices at each step and the
# search is tiny for 14 tiles.
#
# BRUTE FORCE (say it first): Choose any three tiles for
# every meld
# For each pair, try every set of three remaining tiles
# that forms a meld and recurse. Correct and the oracle;
# it ignores that the smallest tile must be used first.
# Time roughly C(12,3) * C(9,3) * C(6,3) per pair. Space
# O(1).


import itertools
from collections import Counter


# BRUTE FORCE (say it first): Choose any three tiles for
# every meld | time roughly C(12,3) * C(9,3) * C(6,3) per
# pair
def mahjong_brute(hand):
    def is_meld(t):
        t = sorted(t)
        same = t[0] == t[1] == t[2]
        run = (
            t[0] // 10 == t[1] // 10 == t[2] // 10 < 3
            and t[1] == t[0] + 1
            and t[2] == t[0] + 2
        )
        return same or run

    def melds(tiles):
        if not tiles:
            return True
        for idx in itertools.combinations(
            range(len(tiles)), 3
        ):
            if is_meld(
                [tiles[i] for i in idx]
            ) and melds(
                [
                    t
                    for i, t in enumerate(tiles)
                    if i not in idx
                ]
            ):
                return True
        return False

    for i, j in itertools.combinations(
        range(len(hand)), 2
    ):
        if hand[i] == hand[j] and melds(
            [
                t
                for k, t in enumerate(hand)
                if k not in (i, j)
            ]
        ):
            return True
    return False
```

```
# HELPERS for the base question
def is_run_start(tile, count):
    # numbered suits are 0..2 with ranks 1..9; the tile
    # id is suit * 10 + rank. Honours (30+) never run.
    ok = tile < 30 and tile % 10 <= 7
    nxt, nxt2 = count.get(tile + 1), count.get(tile + 2)
    return ok and nxt and nxt2


def melds_left(count, need):
    # can the tiles in `count` form exactly `need` melds?
    live = [t for t in sorted(count) if count[t]]
    tile = live[0] if live else None
    if tile is None:
        return need == 0
    if need == 0:
        return False
    # the smallest tile is either in a triplet ...
    if count[tile] >= 3:
        count[tile] -= 3
        ok = melds_left(count, need - 1)
        count[tile] += 3
        if ok:
            return True
    # ... or starts a run of three
    if is_run_start(tile, count):
        for t in (tile, tile + 1, tile + 2):
            count[t] -= 1
        ok = melds_left(count, need - 1)
        for t in (tile, tile + 1, tile + 2):
            count[t] += 1
        return ok
```

```
    return False


# BASE QUESTION (optimal): Try each pair, then greedy-
# smallest with two options
# Idea: Tiles are encoded as suit * 10 + rank for the
# numbered suits (ranks 1 to 9, suits 0 to 2) and 30 and
# above for honours, which never form runs.
# FOLLOW-UP 1: A variant needs only two melds plus a pair
# (8 tiles of 14).
# Answer: Ask what happens to the other six tiles before
# coding: if they are free, check whether any 8 tiles form
# a pair and two melds with the same search over subsets
```

```
# of the 14.
def is_winning_hand(hand, melds=4):
    # FOLLOW-UP 1: melds=2 handles the 8-tile variant
    # one pair plus `melds` melds uses every tile
    if len(hand) != 2 + 3 * melds:
        return False
    count = Counter(hand)
    # try every tile value as the pair
    for tile in list(count):
        if count[tile] >= 2:
            count[tile] -= 2
            ok = melds_left(count, melds)
            count[tile] += 2
            if ok:
                return True
    return False


if __name__ == "__main__":
    win = [1, 1, 1, 2, 3, 4, 11, 12, 13,
           21, 22, 23, 31, 31]
    assert is_winning_hand(win)
    # 2, 3, 5 is not a run
    lose = [1, 1, 1, 2, 3, 5, 11, 12, 13,
            21, 22, 23, 31, 31]
    assert not is_winning_hand(lose)
    # honours only make triplets or the pair
    honours = [30, 30, 31, 31, 31, 32, 32, 32, 33, 33, 33,
               34, 34, 34]
    assert is_winning_hand(honours)
    # the smaller variant: a pair and two melds (8 tiles)
    assert is_winning_hand([5, 5, 1, 2, 3, 7, 7, 7], 2)
    # wrong tile count is never a win
    assert not is_winning_hand(win[:13])
```

NOTES FROM CANDIDATES

Clarify the rules — the prompt is intentionally sparse. Confirm: are honor tiles (winds/dragons) included? Do flowers count? Multi-suit?
Greedy + backtracking is the standard approach:

Try every possible pair (tile, tile); remove the pair.
On the remaining 12 tiles, repeatedly take the smallest remaining tile and try to form a triplet of it or a run starting at it; backtrack on failure.

For 14 tiles this runs in well under 1 ms; the interviewer cares more about the case analysis and clean recursion than micro-optimizations.
The same algorithm extends to special hands (seven pairs / ????) if the interviewer adds them as a follow-up.

PREPARATION

Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 8 of 22

Train / Flight Schedule Reachability

ASKED 2026-10-08 | 12 DAYS AGO | LAST 30 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 182/228

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2092 Find All People With Secret (related)

Frequency: Low | Last asked: 2026-10-08 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/train-schedule-reachability

QUESTION

Given trains (or flights) as (from, to, departure, arrival), decide whether you can travel from a source to a destination, boarding a later leg only if it departs after you arrive. Graph reachability with time state, solved with BFS / DFS.

Input: a schedule of legs, each with a departure station, an arrival station, a departure time, and an arrival time; plus a source and a destination.
Output: whether the destination is reachable from the source, where a leg can only be taken if it departs after you arrive at its departure station.
The same prompt appears as a flight schedule between airports in new-grad R1.

EXAMPLES

```
trains = [
  ("A", "Y", dep=1, arr=20),
  ("A", "Y", dep=2, arr=5),
  ("Y", "B", dep=6, arr=9)
]
source = "A", destination = "B"

Output: true — the first A->Y train arrives too late
    for the Y->B connection, but the second one makes
    it.
```

APPROACH

Implement BFS / DFS over (station, arrival time) states that only follows legs departing after the current arrival.
Hand-trace the example, including the late A->Y leg that must be rejected.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether a connection with departure equal to arrival is allowed, whether a leg can arrive at the same time it leaves, whether times are integers, and whether the answer is yes or no or the route.

BRUTE FORCE TO SAY OUT LOUD

Search over (station, arrival time) states following any leg that departs later. O(legs^2).

OPTIMAL IDEA

Sort the legs by departure time and keep earliest[station]. The source starts at minus infinity. For each leg, if earliest[origin] is before (or equal to, if allowed) the departure, set earliest[destination] = min(earliest[destination], arrival). The destination is reachable when it has an entry. O(L log L); the table also gives the earliest arrival time, and a parent leg per station gives the route.

DRY-RUN THESE CASES

The example with the late first A to Y leg, source equals destination, no legs, a connection with arrival exactly equal to departure, parallel legs, a cycle in the stations.

LIKELY FOLLOW-UPS

Return the route or the earliest arrival, many queries on one schedule (group by origin and use a time-ordered heap), or minimum number of legs.

PITFALLS

Ignoring time and treating it as station reachability (the stated trap). Mixing up strict and non-strict connection times. Processing legs in input order instead of departure order.

HOW TO THINK ABOUT IT

Plain station-to-station reachability is wrong because a leg is only usable if you are already at its departure station before it leaves. Time is part of the state, so ask what you need to know about each station: the earliest time you can be there. Process the legs in order of departure time; a leg is usable if the earliest arrival at its origin is not later than its departure, and then it can improve the earliest arrival at its destination. One sort and one pass answer reachability, and the same table gives the earliest arrival time.

Basis: LeetCode 2092, Find All People With Secret, and LeetCode 787, Cheapest Flights Within K Stops: reachability where time is part of the state.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def reachable(legs, src, dst):
    stack, seen = [(src, -INF)], set()
    while stack:
        station, now = stack.pop()
        if station == dst:
            return True
        if (station, now) in seen:
            continue
        seen.add((station, now))
        for a, b, dep, arr in legs:
            if a == station and dep > now:
                stack.append((b, arr))
    return False
```

TIME	SPACE
O(legs^2)	O(legs)

WHY IT IS SLOW

Every state rescans all legs and the same station is reached at many times, O(legs^2). Sorting the legs by departure and keeping only the earliest arrival per station answers in O(L log L).

OPTIMAL SOLUTION · Sort by departure and keep earliest arrivals

TIME	SPACE
O(L log L)	O(S)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the earliest arrival time at the destination.

The table already holds it: earliest_arrivals(legs, source)[dest], or infinity when the station is unreachable. That is the same one pass, so it costs nothing extra.

FOLLOW-UP 2 Print the route, not just yes or no.

Remember the leg that last improved each station's earliest arrival, then walk back from the destination to the source and reverse. Because only a strictly better arrival replaces the stored leg, the stored legs form a valid chain in time.

FOLLOW-UP 3 Many queries on the same schedule: what changes in the graph representation?

Group the legs by origin station and sort each list by departure time. For a query the earliest arrival is relaxed with a heap ordered by time (Dijkstra on time), and the first leg you may board at a station is found with bisect on its departure list, so each query costs O(L log S) instead of re-sorting every time. If the source is fixed and the queries only differ by destination, keep the earliest-arrival table and answer each in O(1).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Train / Flight Schedule Reachability
#
# Given trains (or flights) as (from, to, departure,
# arrival), decide whether you can travel from a source to
# a destination, boarding a later leg only if it departs
# after you arrive. Graph reachability with time state,
# solved with BFS / DFS. Input: a schedule of legs, each
# with a departure station, an arrival station, a
# departure time, and an arrival time; plus a source and a
# destination. Output: whether the destination is
# reachable from the source, where a leg can only be taken
# if it departs after you arrive at its departure station.
# The same prompt appears as a flight schedule between
# airports in new-grad R1. Examples: trains = [ ("A", "Y",
# dep=1, arr=20), ("A", "Y", dep=2, arr=5), ("Y", "B",
# dep=6, arr=9) ] source = "A", destination = "B" Output:
# true — the first A->Y train arrives too late for the Y->
B
# connection, but the second one makes it.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the earliest arrival time at the destination.
# 2. Print the route, not just yes or no.
# 3. Many queries on the same schedule: what changes in
# the graph representation?
#
# INTERVIEW NOTES
# Plain station-to-station reachability is wrong because a
# leg is only usable if you are already at its departure
# station before it leaves. Time is part of the state, so
# ask what you need to know about each station: the
# earliest time you can be there. Process the legs in
# order of departure time; a leg is usable if the earliest
# arrival at its origin is not later than its departure,
# and then it can improve the earliest arrival at its
# destination. One sort and one pass answer reachability,
# and the same table gives the earliest arrival time.
#
# BRUTE FORCE (say it first): Explore every (station,
# arrival time) state
# Search from the source over states (station, time you
# arrived), following every leg that departs after the
# current arrival. Correct and the obvious first answer,
# but each state rescans all legs. Time O(legs^2). Space
# O(legs).
```

```


# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first, it also checks the answers)
def reachable_brute(legs, source, dest, strict=True):
    # the oracle: search over (station, arrival time)
    # states
    seen, stack = set(), [(source, -INF)]
    while stack:
        station, now = stack.pop()
        if station == dest:
            return True
        if (station, now) in seen:
            continue
        seen.add((station, now))
        for frm, to, dep, arr in legs:
            ok = dep > now if strict else dep >= now
            if frm == station and ok:
                stack.append((to, arr))
    return False


# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
# Idea: Sort the legs by departure time. Keep
# earliest[station] = earliest known arrival, with the
# source at minus infinity.
def earliest_arrivals(legs, source, strict=True):
    """Time is part of the state: keep the earliest time
    we can be at each station. Legs go in departure
    order, so every leg that could feed this one is
    already done."""
    earliest = {source: -INF}
    by_departure = sorted(legs, key=lambda x: x[2])
    for frm, to, dep, arr in by_departure:
        have = earliest.get(frm, INF)
        # we must already be there before it leaves
        if have < dep or (not strict and have == dep):
            if arr < earliest.get(to, INF):
                earliest[to] = arr
    return earliest
```

```
# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
# Idea: Sort the legs by departure time. Keep
# earliest[station] = earliest known arrival, with the
# source at minus infinity.
def reachable(legs, source, dest, strict=True):
    return dest in earliest_arrivals(legs, source, strict)


# FOLLOW-UP 2: Print the route, not just yes or no.
# Answer: Remember the leg that last improved each
# station's earliest arrival, then walk back from the
# destination to the source and reverse.
def itinerary(legs, source, dest, strict=True):
    """Follow-up: the route. Remember the leg that last
    improved each station, then walk back from the end."""
    earliest, came = {source: -INF}, {}
    for leg in sorted(legs, key=lambda x: x[2]):
        frm, to, dep, arr = leg
        have = earliest.get(frm, INF)
        usable = have < dep or (not strict
                                and have == dep)
        if usable and arr < earliest.get(to, INF):
            earliest[to], came[to] = arr, leg
    if dest not in earliest:
        return None
    route = []
    while dest != source:
        leg = came[dest]
        route.append(leg)
        dest = leg[0]
    return route[::-1]


if __name__ == "__main__":
    legs = [("A", "Y", 1, 20), ("A", "Y", 2, 5),
            ("Y", "B", 6, 9)]
    assert reachable(legs, "A", "B")
    # without the early A to Y train the connection is
    # lost
    assert not reachable(legs[:1] + legs[2:], "A", "B")
    assert earliest_arrivals(legs, "A")["B"] == 9
    assert itinerary(legs, "A", "B") == [legs[1], legs[2]]
    assert itinerary(legs, "B", "A") is None
    assert reachable(legs, "A", "A")
```

```
    touch = [("A", "B", 1, 5), ("B", "C", 5, 8)]
    assert not reachable(touch, "A", "C")
```

```
assert reachable(touch, "A", "C", strict=False)
import random
for _ in range(300):
    many = []
    for _ in range(random.randint(0, 8)):
        dep = random.randint(0, 9)
        many.append((random.choice("ABCD"),
                     random.choice("ABCD"), dep,
                     dep + random.randint(1, 4)))
    src = random.choice("ABCD")
    dst = random.choice("ABCD")
    for strict in (True, False):
        want = reachable_brute(many, src, dst, strict)
        got = reachable(many, src, dst, strict)
        assert got == want
```

NOTES FROM CANDIDATES

Pitfalls: Plain station-level reachability is wrong: in the example, A->Y->B exists as a path, but only one of the two A->Y legs makes the connection.
Clarify whether a connection with departure exactly equal to arrival is allowed.

Follow-ups: Follow-ups push on graph representation

PREPARATION

Implement BFS / DFS over (station, arrival time) states that only follows legs departing after the current arrival.
Hand-trace the example, including the late A->Y leg that must be rejected.

MEDIUM | 1Point3Acres | tech-screen | 9 of 22

Visible People in a Line (Last Person, Then Everyone)

ASKED 2026-10-07 | 13 DAYS AGO | LAST 30 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 181/228

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1944 Number of Visible People in a Queue

LeetCode match: LC 1944 Number of Visible People in a Queue (related)

Frequency: Single report | Last asked: 2026-10-07 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/visible-people-in-line

QUESTION

Given heights of people in a line, A can see B if nobody between them is taller than both. Part 1: count how many people the last person can see. Part 2: return that count for every person. Written in a plain editor with no compilation.

QUESTION

Input: an array of heights representing a line of people.
Visibility rule: person A can see person B if no one standing between them is taller than both A and B.
Part 1: return how many people the last person in line can see.
Part 2: for the same array, return, for every person in the line, how many people they can see.

EXAMPLES

```
[1, 10, 6, 7, 9, 8, 2, 4, 3, 5] -> 6 (the last person,
    height 5, can see 3, 4, 2, 8, 9, 10).
```

APPROACH

Write a brute-force Part 1 that checks every earlier person against the rule, then look for a single-pass version before attempting Part 2.
Extend the Part 1 solution to Part 2 and state its complexity before coding.
Hand-trace the example above without running code, including a tie case you define yourself.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what blocks the view (someone strictly taller than both), how ties are treated, which directions count in Part 2, and whether the code runs (it does not: trace by hand).

BRUTE FORCE TO SAY OUT LOUD

Check every pair against the people between them. O(n^2) for the last person, O(n^3) for everyone.

OPTIMAL IDEA

Part 1: walk from the person before the last towards the front, keeping the maximum height between the last person and the current one; the current person is visible when that maximum is at most the larger of the two heights. O(n). Part 2: for each person, everyone up to the first strictly taller person on the right is visible, then only people at least as tall as everyone from there on; a monotonic stack gives the next at-least-as-tall and next strictly-taller indexes and the chain length in O(n). Run it on the reversed line for the left side and add.

DRY-RUN THESE CASES

A single person, all equal heights, strictly increasing and decreasing lines, the example from the statement (the last person sees 6), a tie case you invent.

LIKELY FOLLOW-UPS

Only looking in one direction, blocking on equal heights, or returning who they see instead of how many.

PITFALLS

Using a strict comparison where the rule says taller than both. Forgetting that the first taller person is itself visible. Rereading the example without tracing it by hand.

HOW TO THINK ABOUT IT

Person A sees person B when nobody strictly between them is taller than both, which means the tallest person between them is at most as tall as the taller of A and B. Walking away from the last person and keeping the running maximum of everyone passed lets you test each person in O(1). For everyone at once the visible people to the right form a chain: everything up to the first taller person, then only the people who are at least as tall as everyone before them. A monotonic stack builds those chains for all positions in O(n).

Basis: LeetCode 1944, Number of Visible People in a Queue, with the rule that a person is hidden only by someone taller than both.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def sees(a, i, j):
    lo, hi = min(i, j), max(i, j)
    for k in range(lo + 1, hi):
        if a[k] > max(a[i], a[j]):
            return False
    return True
def count_for(a, i):
    return sum(1 for j in range(len(a))
               if j != i and sees(a, i, j))
```

TIME	SPACE
O(n^3) for everyone, O(n^2) for the last person	**O(1)**

WHY IT IS SLOW

Each pair rescans the people between it, O(n^3) for everyone and O(n^2) for the last person. One pass with a running maximum answers the last person in O(n), and a monotonic stack answers everyone in O(n).

OPTIMAL SOLUTION · One pass with a running maximum (last person)

TIME	SPACE
O(n)	**O(1)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Part 2: return the number of people each person can see.

Run the right-looking count for every position with a monotonic stack, run it again on the reversed line for the looking-left count, and add the two. The stack makes both passes O(n), instead of the O(n^2) you would get by repeating the last-person scan for every position.

FOLLOW-UP 2 How do ties and the direction of sight change the answer?

Under the stated rule only someone strictly taller than both blocks the view, so equal heights never block: in [5, 5, 5] the first person sees both others. If the interviewer wants 'taller or equal blocks', replace the non-strict comparisons with strict ones. If a person only looks towards the front, drop the reversed pass and use see_right on the reversed line alone.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Visible People in a Line (Last Person,
# Then Everyone)
#
# Given heights of people in a line, A can see B if nobody
# between them is taller than both. Part 1: count how many
# people the last person can see. Part 2: return that
# count for every person. Written in a plain editor with
# no compilation. Input: an array of heights representing
# a line of people. Visibility rule: person A can see
# person B if no one standing between them is taller than
# both A and B. Part 1: return how many people the last
# person in line can see. Part 2: for the same array,
# return, for every person in the line, how many people
# they can see. Examples: [1, 10, 6, 7, 9, 8, 2, 4, 3, 5]
# -> 6 (the last person, height 5, can see 3, 4, 2, 8, 9,
# 10).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Part 2: return the number of people each person can
# see.
# 2. How do ties and the direction of sight change the
# answer?
#
# INTERVIEW NOTES
# Person A sees person B when nobody strictly between them
# is taller than both, which means the tallest person
# between them is at most as tall as the taller of A and
# B. Walking away from the last person and keeping the
# running maximum of everyone passed lets you test each
# person in O(1). For everyone at once the visible people
# to the right form a chain: everything up to the first
# taller person, then only the people who are at least as
# tall as everyone before them. A monotonic stack builds
# those chains for all positions in O(n).
#
# BRUTE FORCE (say it first): Test every pair
# For a pair (i, j) look at everyone between them and
# check that none is taller than both. Counting over all
# pairs is O(n^3). Fine as the oracle and for hand-tracing
# the example. Time O(n^3) for everyone, O(n^2) for the
# last person. Space O(1).


# BRUTE FORCE (say it first, it also checks the answers)
def sees(a, i, j):
    # the oracle: nobody strictly between is taller than
    # both
    lo, hi = min(i, j), max(i, j)
    return all(a[k] <= max(a[i], a[j])
               for k in range(lo + 1, hi))


# BRUTE FORCE (say it first, it also checks the answers)
```

```
def counts_brute(a):
    return [sum(1 for j in range(len(a))
                if j != i and sees(a, i, j))
            for i in range(len(a))]


# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): One pass with a running maximum
# (last person)
# Idea: Walk from the person next to the last one towards
# the front, keeping the largest height seen strictly
# between the last person and the current one.
def last_sees(a):
    """Part 1: walk away from the last person keeping the
    tallest height seen strictly between us and them."""
    if not a:
        return 0
    last, between, count = a[-1], -INF, 0
    for k in range(len(a) - 2, -1, -1):
        # nobody between is taller than both of us
        if between <= max(last, a[k]):
            count += 1
        between = max(between, a[k])
    return count


# FOLLOW-UP 1: Part 2: return the number of people each
# person can see.
# Answer: Run the right-looking count for every position
# with a monotonic stack, run it again on the reversed
# line for the looking-left count, and add the two.
def see_right(a):
    """Part 2, looking right: everyone up to the first
    strictly taller person is visible, then only people at
    least as tall as everyone from that person on."""
    n = len(a)
    next_ge, next_gt = [None] * n, [None] * n
    stack = []
    for k in range(n - 1, -1, -1):
        # next person at least as tall as person k
        while stack and a[stack[-1]] < a[k]:
            stack.pop()
        next_ge[k] = stack[-1] if stack else None
        stack.append(k)
    stack = []
    for k in range(n - 1, -1, -1):
        # next person strictly taller than person k
        while stack and a[stack[-1]] <= a[k]:
            stack.pop()
        next_gt[k] = stack[-1] if stack else None
        stack.append(k)
    chain = [0] * n
    for k in range(n - 1, -1, -1):
        # how many "at least as tall" steps follow k
        if next_ge[k] is not None:
            chain[k] = 1 + chain[next_ge[k]]
    out = []
    for i in range(n):
        t = next_gt[i]
        if t is None:
            out.append(n - 1 - i)
        else:
            out.append(t - i + chain[t])
    return out


# FOLLOW-UP 1: Part 2: return the number of people each
# person can see.
# Answer: Run the right-looking count for every position
# with a monotonic stack, run it again on the reversed
# line for the looking-left count, and add the two.
def see_all(a):
    # looking left is looking right on the reversed line
    left = see_right(a[::-1])[::-1]
    return [x + y for x, y in zip(left, see_right(a))]


if __name__ == "__main__":
    line = [1, 10, 6, 7, 9, 8, 2, 4, 3, 5]
    assert last_sees(line) == 6
    assert see_all(line)[-1] == 6
    assert last_sees([]) == 0 and last_sees([7]) == 0
    # equal heights never block each other
    assert last_sees([5, 5, 5]) == 2
    assert see_all([5, 5, 5]) == [2, 2, 2]
    import random
    for _ in range(300):
        a = [random.randint(1, 6)
             for _ in range(random.randint(0, 9))]
        assert see_all(a) == counts_brute(a)
        if a:
            assert last_sees(a) == counts_brute(a)[-1]
```

NOTES FROM CANDIDATES

Pitfalls: The code is written in a plain text editor that does not compile or run; trace the example by hand before declaring the solution done.
Confirm the direction of "see" for Part 2 (toward the front only, or both directions) and how equal heights are treated before coding; the example does not exercise ties.

Follow-ups: Generalize from the last person to every person in the line (Part 2).

PREPARATION

Write a brute-force Part 1 that checks every earlier person against the rule, then look for a single-pass version before attempting Part 2.
Extend the Part 1 solution to Part 2 and state its complexity before coding.
Hand-trace the example above without running code, including a tie case you define yourself.

MEDIUM | IN APP LC 210 | 1Point3Acres | onsite-coding | 10 of 22

Course Scheduling and Dependency Variants

ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 179/228

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: your app's interview approach (LeetCode #210)

LeetCode match: LC 210 Course Schedule II

Frequency: Medium | Last asked: 2026-10-04

Original: https://www.1point3acres.com/interview/problems/company/google/course-schedule-dependency-variants

QUESTION

Topological-ordering family. Variants: course ordering with validity checks and a nested-list extension; merging two arrays whose internal order defines precedence (return false on a conflict cycle); and a damaged-node dependency-path variant that minimizes broken nodes on a route.

Course-ordering variant

Work through the course-ordering problem, including validity checks, complexity analysis, and test cases.
Extend the result to a nested-list format. The grouping rule is unspecified; clarify what each inner list represents before implementing the extension.

Damaged-node and path variant

Remove damaged nodes and their downstream effects, then find a valid path.
A follow-up minimizes broken nodes on a required route. Clarify the graph direction, path endpoints, and meaning of broken nodes before choosing an algorithm.

Merge-two-ordered-sequences variant

Input: two integer arrays a and b. The order of elements inside each array is an ordering constraint — a = [1, 3, 5] means 1 must come before 3, and 3 before 5.
Determine whether the two arrays can be merged into one sequence that preserves the relative order of both. Return false if the constraints conflict; otherwise return true (or any valid merged sequence).

EXAMPLES

Merge-two-ordered-sequences

```
a = [1, 2, 3]
b = [2, 4, 5]
-> true   (one valid merged order: [1, 2, 3, 4, 5])

a = [1, 6, 4]
b = [4, 1]
-> false  (a requires 1 -> 6 -> 4, b requires 4 -> 1:
    contradiction)
```

APPROACH

In 20 minutes, implement prerequisite ordering with cycle detection; check isolated courses, independent choices, a chain, and a directed cycle.
In 10 minutes, write an independent result checker for vertex coverage and every prerequisite constraint. Accept different valid orders.
In 10 minutes, list the missing grouping and path-cost rules, and explain which algorithm decisions depend on each answer.

HOW TO THINK ABOUT IT

Every 'A must come before B' rule is an edge. A valid ordering exists exactly when the graph has no cycle, and Kahn's algorithm finds one: repeatedly take a vertex with no unmet prerequisites. The two-array merge is the same problem: each adjacent pair inside an array is an edge and equal values are the same vertex; the merge is possible only if the combined graph has no cycle.

Basis: LeetCode 210, Course Schedule II, plus a merge-two-ordered-sequences variant (the same graph in disguise).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, rules):
    for order in permutations(range(n)):
        pos = {v: i for i, v in enumerate(order)}
        ok = all(pos[a] < pos[b]
                 for a, b in rules)
        if ok:
            return True
    return False
```

TIME	SPACE
O(V! * E)	O(V)

WHY IT IS SLOW

V! orderings. Kahn's algorithm repeatedly takes a vertex with no unmet prerequisites and finishes in O(V + E).

OPTIMAL SOLUTION · Kahn's algorithm (course order, or can two sequences merge)

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you check that an order is valid?

Record each vertex's position in the order, check that every vertex appears exactly once, then check that for every edge (u, v) the position of u is smaller than the position of v. Accept any valid order, not one fixed answer, because independent vertices can swap.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Course Scheduling and Dependency Variants
#
# Topological-ordering family. Variants: course ordering
# with validity checks and a nested-list extension;
# merging two arrays whose internal order defines
# precedence (return false on a conflict cycle); and a
# damaged-node dependency-path variant that minimizes
# broken nodes on a route. Course-ordering variant Work
# through the course-ordering problem, including validity
# checks, complexity analysis, and test cases. Extend the
# result to a nested-list format. The grouping rule is
# unspecified; clarify what each inner list represents
# before implementing the extension. Damaged-node and path
# variant Remove damaged nodes and their downstream
# effects, then find a valid path. A follow-up minimizes
# broken nodes on a required route. Clarify the graph
# direction, path endpoints, and meaning of broken nodes
# before choosing an algorithm. Merge-two-ordered-
# sequences variant Input: two integer arrays a and b.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you check that an order is valid?
#
# INTERVIEW NOTES
# Every 'A must come before B' rule is an edge. A valid
# ordering exists exactly when the graph has no cycle, and
# Kahn's algorithm finds one: repeatedly take a vertex
# with no unmet prerequisites. The two-array merge is the
# same problem: each adjacent pair inside an array is an
# edge and equal values are the same vertex; the merge is
# possible only if the combined graph has no cycle.
#
# BRUTE FORCE (say it first): Try every permutation of the
# vertices
# Generate permutations and check every constraint.
# Correct and a good oracle for tiny inputs; factorial
# time is unusable beyond about ten vertices. Time O(V! *
# E). Space O(V).


import itertools
from collections import defaultdict, deque


# BRUTE FORCE (say it first): Try every permutation of the
# vertices | time O(V! * E)
def merge_orders_brute(a, b):
    vs = sorted(set(a) | set(b))
    edges = [
        (x[i], x[i + 1])
        for x in (a, b)
        for i in range(len(x) - 1)
    ]
    for perm in itertools.permutations(vs):
        pos = {v: i for i, v in enumerate(perm)}
        if all(pos[u] < pos[v] for u, v in edges):
            return True
    return False


# HELPERS for the base question
def edges_from_sequences(*sequences):
    # each neighbouring pair in a sequence is a rule
    return [(s[i], s[i + 1]) for s in sequences
            for i in range(len(s) - 1)]


def merge_two_orders(a, b):
    # shared values are the same vertex
    edges = edges_from_sequences(a, b)
    # a cycle (conflict) gives None
    return topological_order(set(a) | set(b), edges)


# BASE QUESTION (optimal): Kahn's algorithm (course order,
# or can two sequences merge)
# Idea: Build the graph (prerequisite -> dependent,
# including isolated vertices), count indegrees and keep a
# queue of vertices with indegree 0. Pop one, append it to
# the order and lower the indegree of its dependents.
def topological_order(vertices, edges):
    # edge (u, v) means u must come before v
    adj = defaultdict(list)
    indeg = {v: 0 for v in vertices}
    for u, v in edges:
        adj[u].append(v)
        # count unmet prerequisites of v
        indeg[v] = indeg.get(v, 0) + 1
        indeg.setdefault(u, 0)
    # start with everything that has no prerequisite
    queue = deque(v for v, d in indeg.items() if d == 0)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        # finishing u frees its dependents
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                queue.append(v)
    # fewer than all vertices means a cycle: no order
    return order if len(order) == len(indeg) else None


# BASE QUESTION (optimal): Kahn's algorithm (course order,
# or can two sequences merge)
# Idea: Build the graph (prerequisite -> dependent,
# including isolated vertices), count indegrees and keep a
# queue of vertices with indegree 0. Pop one, append it to
# the order and lower the indegree of its dependents.
def can_merge(a, b):
    # True when some order keeps both sequences in order
    return merge_two_orders(a, b) is not None


# FOLLOW-UP 1: How do you check that an order is valid?
# Answer: Record each vertex's position in the order,
# check that every vertex appears exactly once, then check
# that for every edge (u, v) the position of u is smaller
# than the position of v.
def is_valid_order(order, vertices, edges):
    # every vertex must appear exactly once
    if len(order) != len(set(vertices)):
        return False
    if set(order) != set(vertices):
        return False
    pos = {v: i for i, v in enumerate(order)}
    # every prerequisite must come before its dependent
    return all(pos[u] < pos[v] for u, v in edges)


if __name__ == "__main__":
    # course order with an isolated course 3
    courses, rules = range(4), [(0, 1), (1, 2)]
    order = topological_order(courses, rules)
```

```
    assert is_valid_order(order, courses, rules)
    # a swapped order breaks a rule
    swapped = [1, 0, 2, 3]
    assert not is_valid_order(swapped, courses, rules)
    # a cycle has no order
    loop = [(0, 1), (1, 0)]
    assert topological_order(range(2), loop) is None
    # merge examples from the question
    assert can_merge([1, 2, 3], [2, 4, 5])
    merged = merge_two_orders([1, 2, 3], [2, 4, 5])
    assert merged is not None
    assert not can_merge([1, 6, 4], [4, 1])
    # a repeated value in one array is a self conflict
    assert not can_merge([1, 2, 1], [])
```

NOTES FROM CANDIDATES

For standard prerequisite-ordering practice, represent each course as a vertex and each prerequisite as an edge from prerequisite to dependent. Include isolated courses. The result orders every course after its prerequisites; a cycle prevents a complete ordering.

Maintain indegrees and a queue of zero-indegree vertices. Emit a vertex, decrement its outgoing neighbors, and enqueue newly ready vertices. If fewer than V vertices are emitted, report failure under the chosen return convention. This takes O(V + E) time and space; check each edge against positions in the output to validate an ordering.

This ordering algorithm does not determine the nested-list grouping rule or solve the damaged-node path objective. Keep those contracts separate rather than assuming they are ordinary topological sorting.

The merge-two-sequences variant is the same ordering engine in disguise: shared values are the same node, and each adjacent pair inside an array is an edge. A cycle means the merge is impossible. It was asked as the final onsite round of an early-career L3 loop, and the candidate nearly ran out of time — recognizing it as a graph problem early is the main hurdle.

PREPARATION

In 20 minutes, implement prerequisite ordering with cycle detection; check isolated courses, independent choices, a chain, and a directed cycle.
In 10 minutes, write an independent result checker for vertex coverage and every prerequisite constraint. Accept different valid orders.
In 10 minutes, list the missing grouping and path-cost rules, and explain which algorithm decisions depend on each answer.

MEDIUM | 1Point3Acres | phone-screen | 11 of 22

Connected Crop Arrangement on a Grid

ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 207/228

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-10-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/connected-crop-arrangement-grid

QUESTION

Given an N x M grid and K crop counts summing to N*M, fill every cell so each crop's cells form one 4-connected region; return any valid arrangement. Follow-up: two adjacent square gardens (N x N and M x M) with counts summing to N^2 + M^2.

A garden is an N x M grid. There are K crop types labeled 1..K; crop i must occupy exactly count[i] cells.
Guaranteed: sum(count) = N * M, so every cell is used.
For every crop type, all of its cells must form one connected component under 4-directional adjacency (up, down, left, right; diagonals do not count).
Different crops have no relationship constraints with each other.
Return any valid arrangement.

Follow-up: two square gardens

Instead of one rectangle, there are two square gardens placed directly next to each other: garden A is N x N, garden B is M x M; together they form the planting area.
Now sum(count) = N^2 + M^2. The same single-connected-region rule applies across the combined shape.

EXAMPLES

Follow-up shapes for N = 2, M = 3 (digits mark garden A = 1, garden B = 2):

```
11222
11222
  222
```

or

```
11
11
222
222
222
```

APPROACH

Write a traversal over an N x M grid in which every consecutive pair of visited cells shares an edge, then fill crops in order along it; argue why every crop's cells stay connected.
Extend the same traversal to an L-shaped region made of two squares, handling the transition cell between gardens.
Test with K = 1, many crops of size 1, a single row or column, and both follow-up layouts above.

HOW TO THINK ABOUT IT

Walk the grid in a snake (left to right on one row, right to left on the next) so that every cell is next to the one before it. If you give crop 1 the first count[1] cells of that walk, crop 2 the next count[2] cells and so on, each crop is a contiguous stretch of the walk, so its cells are connected. No search or backtracking is needed.

Basis: Written for this question: a constructive fill along a Hamiltonian path of the grid (a snake), no LeetCode problem matches it exactly.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def fill(crop):
    if crop == len(counts):
        return True
    size = counts[crop]
    for cells in combinations(free_cells, size):
        if is_connected(cells):
            place(cells, crop)
            if fill(crop + 1):
                return True
            remove(cells)
    return False
```

TIME	SPACE
exponential	O(n * m)

WHY IT IS SLOW

Exponential in the number of cells. Filling the crops along a snake path through the grid is O(n * m) and every crop stays connected by construction.

OPTIMAL SOLUTION · Fill crops along a snake path

TIME	SPACE
O(n * m)	O(n * m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Two adjacent square gardens (N x N next to M x M).

Use a different Hamiltonian path of the combined L-shaped area: snake down and up the columns of the first garden so that the last column ends in the top row, step into the second garden there and snake through it. Hand out the crop counts along that path exactly as before. The code checks that this path visits every cell once with each step adjacent, for every size from 1 to 6.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Connected Crop Arrangement on a Grid
#
# Given an N x M grid and K crop counts summing to N*M,
# fill every cell so each crop's cells form one
# 4-connected region; return any valid arrangement.
# Follow-up: two adjacent square gardens (N x N and M x M)
# with counts summing to N^2 + M^2. A garden is an N x M
# grid. There are K crop types labeled 1..K; crop i must
# occupy exactly count[i] cells. Guaranteed: sum(count) =
# N * M, so every cell is used. For every crop type, all
# of its cells must form one connected component under
# 4-directional adjacency (up, down, left, right;
# diagonals do not count). Different crops have no
# relationship constraints with each other. Return any
# valid arrangement. Follow-up: two square gardens Instead
# of one rectangle, there are two square gardens placed
# directly next to each other: garden A is N x N, garden B
# is M x M; together they form the planting area. Now
# sum(count) = N^2 + M^2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Two adjacent square gardens (N x N next to M x M).
#
# INTERVIEW NOTES
# Walk the grid in a snake (left to right on one row,
# right to left on the next) so that every cell is next to
# the one before it. If you give crop 1 the first count[1]
# cells of that walk, crop 2 the next count[2] cells and
# so on, each crop is a contiguous stretch of the walk, so
# its cells are connected. No search or backtracking is
# needed.
#
# BRUTE FORCE (say it first): Backtrack over every
# assignment
# Try every way to place the counts and test connectivity.
# Correct and only possible on tiny grids, which makes it
# a good oracle for the constructive answer. Time
# exponential. Space O(n * m).


import itertools
from collections import defaultdict


# BRUTE FORCE (say it first): Backtrack over every
```

```
# assignment | time exponential
def crops_brute_possible(n, m, counts):
    cells = [(i, j) for i in range(n) for j in range(m)]

    def connected(group):
        group = set(group)
        if not group:
            return True
```

```
        stack, seen = [next(iter(group))], set()
        while stack:
            r, c = stack.pop()
            if (r, c) in seen:
                continue
            seen.add((r, c))
            stack += [
                x
                for x in (
                    (r + 1, c),
                    (r - 1, c),
                    (r, c + 1),
                    (r, c - 1),
                )
                if x in group
            ]
        return seen == group

    def go(idx, remaining):
        if idx == len(counts):
            return True
        for pick in itertools.combinations(
            remaining, counts[idx]
        ):
            if connected(pick) and go(
                idx + 1,
                [c for c in remaining if c not in pick],
            ):
                return True
        return False

    return go(0, cells)


# HELPERS for the base question
def snake_cells(n, m):
    # row 0 left to right, row 1 right to left, ...
    return [(i, j if i % 2 == 0 else m - 1 - j)
            for i in range(n) for j in range(m)]


def fill_along(grid, path, counts):
    # hand out the path to crop 1, crop 2, ... in turn
    k = 0
    for crop, cnt in enumerate(counts, 1):
        for _ in range(cnt):
            r, c = path[k]
            grid[r][c] = crop
            k += 1
    return grid


# HELPERS for follow-up 1
def two_gardens_path(n, m):
    # garden A is n x n; garden B is m x m on its right
    path = []
    # snake down and up the columns of garden A
    for c in range(n):
        # the last column must end in row 0
        down = (c % 2 == 0) == (n % 2 == 0)
        rows = range(n)
        if not down:
            rows = range(n - 1, -1, -1)
        path += [(r, c) for r in rows]
    # step into garden B at row 0 and snake through it
    for r in range(m):
        cols = range(n, n + m) if r % 2 == 0 \
            else range(n + m - 1, n - 1, -1)
        path += [(r, c) for c in cols]
    return path


# BASE QUESTION (optimal): Fill crops along a snake path
# Idea: Generate the snake order of the cells and hand out
# count[0] cells to crop 1, the next count[1] to crop 2,
# and so on. Consecutive cells of the snake share an edge,
# so each crop is connected.
def arrange_crops(n, m, counts):
    # every cell is used exactly once
    assert sum(counts) == n * m
    grid = [[0] * m for _ in range(n)]
    # touching path cells keep each crop connected
    return fill_along(grid, snake_cells(n, m), counts)


# FOLLOW-UP 1: Two adjacent square gardens (N x N next to
# M x M).
# Answer: Use a different Hamiltonian path of the combined
# L-shaped area: snake down and up the columns of the
# first garden so that the last column ends in the top
# row, step into the second garden there and snake through
# it.
def arrange_two_gardens(n, m, counts):
    assert sum(counts) == n * n + m * m
    # the grid is wide enough for both gardens
    grid = [[0] * (n + m) for _ in range(max(n, m))]
    path = two_gardens_path(n, m)
    return fill_along(grid, path, counts)


# HELPERS for the checks below
def crops_valid(grid, counts):
    cells = defaultdict(set)
    for i, row in enumerate(grid):
        for j, v in enumerate(row):
            if v:
                cells[v].add((i, j))
    for crop, cnt in enumerate(counts, 1):
        group = cells[crop]
        # the right number of cells
        if len(group) != cnt:
            return False
        # flood fill from one cell must reach them all
        stack, seen = [next(iter(group))], set()
        while stack:
            r, c = stack.pop()
            if (r, c) in seen:
                continue
            seen.add((r, c))
            for x in ((r + 1, c), (r - 1, c),
                      (r, c + 1), (r, c - 1)):
                if x in group:
                    stack.append(x)
        if seen != group:
            return False
    return True


if __name__ == "__main__":
    # a 2 x 3 field with two crops
    g = arrange_crops(2, 3, [2, 4])
    assert crops_valid(g, [2, 4])
    # many tiny crops still connect
    ones = [1] * 9
    assert crops_valid(arrange_crops(3, 3, ones), ones)
    # two gardens: 2 x 2 next to 3 x 3 (13 cells)
    counts = [5, 4, 4]
    g2 = arrange_two_gardens(2, 3, counts)
    assert crops_valid(g2, counts)
    # the path visits every cell once, step by step
    p = two_gardens_path(2, 3)
    assert len(set(p)) == 13
    assert all(abs(a[0] - b[0]) + abs(a[1] - b[1]) == 1
               for a, b in zip(p, p[1:]))
```

NOTES FROM CANDIDATES

Asked as the first-round online coding interview of an early-career L3 loop.
Because any valid arrangement is accepted, look for a constructive traversal order that keeps consecutive cells adjacent, rather than backtracking.
For the follow-up, the two gardens can be stacked side by side or one above the other; clarify how they are joined (which edges touch and the offset) before designing the traversal.

PREPARATION

Write a traversal over an N x M grid in which every consecutive pair of visited cells shares an edge, then fill crops in order along it; argue why every crop's cells stay connected.
Extend the same traversal to an L-shaped region made of two squares, handling the transition cell between gardens.
Test with K = 1, many crops of size 1, a single row or column, and both follow-up layouts above.

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 12 of 22

Longest Non-Decreasing Subarray with One Replacement

ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 209/228

ABOUT THIS QUESTION

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/longest-non-decreasing-subarray-one-replacement

QUESTION

Find the maximum length of a contiguous non-decreasing subarray if at most one element may be changed to any value. Some rounds start from the plain longest-increasing-subarray question and add the one-replacement follow-up; those versions ask for an increasing run, so clarify strict vs non-strict.

Input: an integer array nums.
You may change at most one element to any value.
Return the maximum length of a contiguous subarray that is non-decreasing (nums[i] <= nums[i + 1] for every adjacent pair inside it) after the change.

Variant: other rounds ask for a strictly increasing run and open with the no-replacement version (longest increasing subarray) before adding the one-replacement follow-up — clarify strict vs non-strict before coding.

QUESTION

Expect to discuss and compare the complexity of several approaches; a brute-force first pass is usually followed by a request to optimize.

EXAMPLES

```
nums = [1, 0, 3, 4, 5, 2, 3]
Output: 5
Change nums[1] from 0 to 1 -> [1, 1, 3, 4, 5, 2, 3]
[1, 1, 3, 4, 5] is a non-decreasing subarray of
    length 5.
```

APPROACH

Write the plain longest-run scan, then extend it with left-run / right-run arrays (or a two-state DP: "no change used" / "change used") so you can explain the jump from one to the other.
Test the bridge case explicitly: joining the run ending at i - 1 with the run starting at i + 1 is only valid when nums[i - 1] <= nums[i + 1]; in the strict variant there must be room for a value strictly between the neighbors, i.e. nums[i + 1] - nums[i - 1] >= 2 when the replacement must be an integer (plain < only if any real value is allowed). Otherwise the change only extends one side by one.
Repeat the exercise with the strict-increasing variant to make sure your comparisons flip correctly.

HOW TO THINK ABOUT IT

Precompute left[i], the length of the non-decreasing run ending at i, and right[i], the length of the run starting at i. If you change element i to any value, it can join the run on its left and the run on its right when a[i-1] <= a[i+1] (a value between them exists); otherwise it can join only the longer of the two. Take the best over all i.

Basis: Written for this question (the one-change version of LeetCode 674, Longest Continuous Increasing Subsequence).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    best = longest_non_decreasing(a)
    for i in range(len(a)):
        for v in candidate_values(a):
            b = a[:i] + [v] + a[i + 1:]
            run = longest_non_decreasing(b)
            best = max(best, run)
    return best
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

O(n^3). The run lengths on both sides of each position, computed once, answer every change in O(1).

OPTIMAL SOLUTION · Left and right run lengths

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The run must be strictly increasing.

Use < in the run lengths, and when the replaced value must fit between the neighbours check a[i+1] - a[i-1] >= 2 (an integer strictly between them exists) instead of a[i-1] <= a[i+1]. Clarify strict versus non-strict before coding.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Longest Non-Decreasing Subarray with One
# Replacement
#
# Find the maximum length of a contiguous non-decreasing
# subarray if at most one element may be changed to any
# value. Some rounds start from the plain longest-
# increasing-subarray question and add the one-replacement
# follow-up; those versions ask for an increasing run, so
# clarify strict vs non-strict. Input: an integer array
# nums. You may change at most one element to any value.
# Return the maximum length of a contiguous subarray that
# is non-decreasing (nums[i] <= nums[i + 1] for every
# adjacent pair inside it) after the change. Variant:
# other rounds ask for a strictly increasing run and open
# with the no-replacement version (longest increasing
# subarray) before adding the one-replacement follow-up —
# clarify strict vs non-strict before coding. Expect to
# discuss and compare the complexity of several
# approaches; a brute-force first pass is usually followed
# by a request to optimize.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The run must be strictly increasing.
#
# INTERVIEW NOTES
# Precompute left[i], the length of the non-decreasing run
# ending at i, and right[i], the length of the run
# starting at i. If you change element i to any value, it
# can join the run on its left and the run on its right
# when a[i-1] <= a[i+1] (a value between them exists);
# otherwise it can join only the longer of the two. Take
# the best over all i.
#
# BRUTE FORCE (say it first): Try every change and re-scan
# For every position and every candidate value, rewrite
# that element and find the longest non-decreasing run.
# Correct and a good oracle; cubic. Time O(n^3). Space
# O(n).


# BRUTE FORCE (say it first): Try every change and re-scan
# | time O(n^3)
def one_change_brute(a):
    best = 0
    cands = set(a) | {0}
    for i in range(len(a)):
        for v in cands | {a[i]}:
            b = a[:i] + [v] + a[i + 1 :]
            run = 0
            for k in range(len(b)):
                run = (
                    run + 1
                    if k and b[k - 1] <= b[k]
                    else 1
                )
                best = max(best, run)
    return best


# HELPERS for the base question
def run_lengths(a, ok):
    n = len(a)
    # left[i]: run ending at i, right[i]: starting at i
    left, right = [1] * n, [1] * n
    for i in range(1, n):
        if ok(a[i - 1], a[i]):
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if ok(a[i], a[i + 1]):
            right[i] = right[i + 1] + 1
    return left, right


# BASE QUESTION (optimal): Left and right run lengths
# Idea: left[i] and right[i] are the non-decreasing run
# lengths ending and starting at i. Changing a[i] gives
# left[i-1] + 1 + right[i+1] if a[i-1] <= a[i+1],
# otherwise max(left[i-1], right[i+1]) + 1.
# FOLLOW-UP 1: The run must be strictly increasing.
# Answer: Use < in the run lengths, and when the replaced
# value must fit between the neighbours check a[i+1] -
# a[i-1] >= 2 (an integer strictly between them exists)
# instead of a[i-1] <= a[i+1]. Clarify strict versus non-
# strict before coding.
def longest_with_one_change(a, strict=False):
    n = len(a)
    # nothing to fix in arrays of length 0, 1 or 2
    if n <= 2:
        return n
    # FOLLOW-UP 1: strict=True means strictly increasing
    # non-decreasing uses <=, strictly increasing uses <
    def ok(x, y):
        return x < y if strict else x <= y

    # can a value fit between the two neighbours?
    gap = 2 if strict else 0
    left, right = run_lengths(a, ok)
    best = 1
    # try changing each element a[i]
    for i in range(n):
        L = left[i - 1] if i else 0
        R = right[i + 1] if i + 1 < n else 0
        # at an end, or when a value fits between the
        # neighbours, the change joins both runs
        if not L or not R or a[i + 1] - a[i - 1] >= gap:
            best = max(best, L + 1 + R)
        # otherwise it joins only the longer run
        else:
            best = max(best, max(L, R) + 1)
    return best


# WARM-UP: no replacement allowed
def longest_without_change(a):
    # the plain longest increasing subarray warm-up
    best = run = 0
    for i, x in enumerate(a):
        run = run + 1 if i and a[i - 1] < x else 1
```

```
        best = max(best, run)
    return best


if __name__ == "__main__":
    # change 0 to 1: [1, 1, 3, 4, 5] has length 5
    nums = [1, 0, 3, 4, 5, 2, 3]
    assert longest_with_one_change(nums) == 5
    # tiny arrays
    assert longest_with_one_change([]) == 0
    assert longest_with_one_change([7]) == 1
    # strict version needs room between neighbours
    strict = longest_with_one_change
    assert strict([1, 2, 2, 3, 4], True) == 4
    assert strict([1, 1, 1, 1], True) == 2
    # the warm-up without any change
    assert longest_without_change([1, 3, 5, 4, 7]) == 3
```

NOTES FROM CANDIDATES

Asked as an onsite coding round in an early-career L3 loop, and as the first technical round of new-grad VOs.
Candidates who wrote a slow version first were asked to optimize, leaving little time for anything else — the problem is short, so interviewers expect the linear approach to arrive quickly.
Edge cases worth confirming: arrays of length 0–2, where the replaced element is the first or last of the run, and whether the replaced value must fit between both neighbors or only one.

PREPARATION

Write the plain longest-run scan, then extend it with left-run / right-run arrays (or a two-state DP: "no change used" / "change used") so you can explain the jump from one to the other.
Test the bridge case explicitly: joining the run ending at i - 1 with the run starting at i + 1 is only valid when nums[i - 1] <= nums[i + 1]; in the strict variant there must be room for a value strictly between the neighbors, i.e. nums[i + 1] - nums[i - 1] >= 2 when the replacement must be an integer (plain < only if any real value is allowed). Otherwise the change only extends one side by one.
Repeat the exercise with the strict-increasing variant to make sure your comparisons flip correctly.

MEDIUM | IN APP LC 200 | 1Point3Acres | onsite-coding, phone-screen | 13 of 22

★ TOP 75: rank 17 in the Medium and Hard priority list

★ Number of Islands

ASKED 2026-09-30 | 20 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | **ALL TIME 64.8** | **30 DAYS 33** | **PRIORITY 19/228**

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: your app's interview approach (LeetCode #200)

LeetCode match: LC 200 Number of Islands

Frequency: Medium | Last asked: 2026-09-30 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/number-of-islands

QUESTION

EXAMPLES

```
[[1,1,0,0],[1,1,0,0],[0,0,1,1],[0,0,1,1]] -> 2.
Streaming: [[0,0],[0,1],[1,1]] adds -> counts [1, 1,
    1] (last addition merges the two land cells).
```

APPROACH

Write both the static count and the streaming addLand version from memory in under 15 min total.
Be ready to switch between DFS / BFS / Union-Find on demand.

HOW TO THINK ABOUT IT

Each island is a connected group of land cells, so count how many times you start a fresh search at an unvisited land cell. The search itself (DFS or BFS) just marks everything reachable. If land is added over time, do not rerun the search: Union-Find keeps the count up to date.

Basis: LeetCode 200, Number of Islands, and LeetCode 305, Number of Islands II (streaming additions with Union-Find).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    seen, count = set(), 0
    for r, c in all_cells(grid):
        if grid[r][c] == 0 or (r, c) in seen:
            continue
        count += 1
        dfs(r, c)  # marks the whole island
    return count
def dfs(r, c):
    if out_of_grid(r, c) or grid[r][c] == 0:
        return
    if (r, c) in seen:
        return
    seen.add((r, c))
    for dr, dc in DIRECTIONS:
        dfs(r + dr, c + dc)
```

BRUTE FORCE

TIME	SPACE
O(m * n)	O(m * n) recursion

WHY IT IS SLOW

Time is already O(m * n); the weakness is the recursion depth, which can reach m * n on a snake-shaped island and overflow the stack. An explicit queue or stack avoids that.

OPTIMAL SOLUTION · Iterative DFS flood fill

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if land can also be removed?

Union-Find cannot split sets. Either rebuild the affected component with a local BFS from each former neighbour, or process the whole timeline offline with a segment tree over time and union-find with rollback (the offline dynamic connectivity technique).

FOLLOW-UP 2 How do you handle a grid too large for memory?

Process it in horizontal strips: label islands within the strip, keep only the boundary row's labels, and merge labels across strips with a union-find over label ids. Memory is O(width + number of labels on the boundary).

FOLLOW-UP 3 Find the area of the largest island.

Count the cells during each search and keep the maximum. Same traversal, one counter.

FOLLOW-UP 4 The grid is huge and does not fit in memory.

Process it in strips of rows, label islands in each strip, and merge labels across strip boundaries with a Union-Find over the labels on the boundary row. **Cost:** O(m * n) time overall and O(width + labels on the boundary row) memory per strip.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Number of Islands
#
# Examples: [[1,1,0,0],[1,1,0,0],[0,0,1,1],[0,0,1,1]] -> 2
.
# Streaming: [[0,0],[0,1],[1,1]] adds -> counts [1, 1, 1]
# (last addition merges the two land cells).
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if land can also be removed?
# 2. How do you handle a grid too large for memory?
# 3. Find the area of the largest island.
# 4. The grid is huge and does not fit in memory.
#
# INTERVIEW NOTES
# Each island is a connected group of land cells, so count
# how many times you start a fresh search at an unvisited
# land cell. The search itself (DFS or BFS) just marks
# everything reachable. If land is added over time, do not
# rerun the search: Union-Find keeps the count up to date.
#
# BRUTE FORCE (say it first): Recursive flood fill
# Scan; at each unvisited land cell count one island and
# recurse in four directions to mark it. The textbook
# answer and the oracle; deep recursion can overflow on a
# large snake-shaped island. Time O(m * n). Space O(m * n)
# recursion.


# BRUTE FORCE (say it first): Recursive flood fill | time
# O(m * n)
def islands_flood(grid):
    if not grid:
        return 0
    m, n = len(grid), len(grid[0])
    seen = set()

    def fill(r, c):
        if (
            not (0 <= r < m and 0 <= c < n)
            or grid[r][c] != 1
            or (r, c) in seen
        ):
            return
        seen.add((r, c))
        for dr, dc in (
            (1, 0),
            (-1, 0),
            (0, 1),
            (0, -1),
        ):
            fill(r + dr, c + dc)

    count = 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] == 1 and (r, c) not in seen:
                count += 1
                fill(r, c)
    return count


# HELPERS for the base question
DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def neighbors(r, c, rows, cols):
    # the cells beside (r, c) that are inside the grid
    for dr, dc in DIRS:
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            yield nr, nc


def flood(grid, start, seen):
    # visit all land joined to start; return the count
    rows, cols = len(grid), len(grid[0])
    seen.add(start)
    stack, size = [start], 0
    # a stack instead of recursion: no recursion limit
    while stack:
```

```
        r, c = stack.pop()
        size += 1
        for nb in neighbors(r, c, rows, cols):
            if grid[nb[0]][nb[1]] == 1 and nb not in seen:
                seen.add(nb)
                stack.append(nb)
    return size


def island_sizes(grid):
    if not grid:
        return []
    seen, sizes = set(), []
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            # an island starts at land not visited yet
            if grid[r][c] == 1 and (r, c) not in seen:
                sizes.append(flood(grid, (r, c), seen))
    return sizes


# BASE QUESTION (optimal): Iterative DFS flood fill
# Idea: Scan every cell; each unvisited land cell starts a
# new island, and an explicit stack marks all connected
# land as visited. Iterative DFS avoids Python's recursion
# limit on large grids.
def num_islands(grid):
    # one entry per island
    return len(island_sizes(grid))


# VARIANT: land cells arrive one at a time
class IslandCounter:
    """Follow-up: land cells arrive one at a time."""

    def __init__(self, rows, cols):
        self.rows, self.cols = rows, cols
        # land cell -> parent cell (union-find)
        self.parent = {}
        self.count = 0

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add_land(self, r, c):
        # adding the same cell twice changes nothing
        if (r, c) in self.parent:
            return self.count
        self.parent[(r, c)] = (r, c)
        self.count += 1
        # join with every land neighbour
        for nb in neighbors(r, c, self.rows, self.cols):
            if nb in self.parent:
                a, b = self.find((r, c)), self.find(nb)
                if a != b:
                    self.parent[a] = b
                    # two islands merged into one
                    self.count -= 1
        return self.count


# FOLLOW-UP 3: Find the area of the largest island.
# Answer: Count the cells during each search and keep the
# maximum. Same traversal, one counter.
def largest_island(grid):
    # follow-up: the area of the biggest island
    return max(island_sizes(grid), default=0)


if __name__ == "__main__":
    g = [[1, 1, 0, 0], [1, 1, 0, 0],
         [0, 0, 1, 1], [0, 0, 1, 1]]
    assert num_islands(g) == 2
    assert largest_island(g) == 4
    assert num_islands([]) == 0
    # streaming adds: counts stay 1, 1, 1
    ic = IslandCounter(2, 2)
    adds = [(0, 0), (0, 1), (1, 1)]
    counts = [ic.add_land(r, c) for r, c in adds]
    assert counts == [1, 1, 1]
    # two separate cells make two islands
    ic = IslandCounter(3, 3)
    assert ic.add_land(0, 0) == 1
    assert ic.add_land(2, 2) == 2
    assert ic.add_land(1, 1) == 3
```

NOTES FROM CANDIDATES

For streaming, Union-Find with path compression is the natural choice — each addLand is amortized alpha(n).
In one onsite, the follow-up arrived after a working BFS solution and explicitly required a Union-Find rewrite; retrofitting the original traversal consumed the remaining time.
Common edge cases: diagonal connectivity (some prompts use 8-connectivity), grid with no land.

PREPARATION

Write both the static count and the streaming addLand version from memory in under 15 min total.
Be ready to switch between DFS / BFS / Union-Find on demand.

MEDIUM | IN APP LC 778 | 1Point3Acres | onsite-coding | 14 of 22

Path Minimizing the Maximum Cell (Swim in Rising Water)

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | ALL TIME 40 | 30 DAYS 33 | PRIORITY 55/228

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: your app's interview approach (LeetCode #778)

LeetCode match: LC 778 Swim in Rising Water

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/path-min-max-grid-dijkstra

QUESTION

A bottleneck-path family: minimize the maximum value encountered on a grid path, then generalize the same reasoning to a weighted graph whose edges carry safety values and an admissibility threshold.

QUESTION

Grid variant

Input is a grid of heights and start/end cells; movement is through four-neighbors.
Cost of a path is the maximum cell value along it.
Return the minimum possible path cost.

Weighted-graph safety variant

Input is a graph whose edges carry safety values, a start node, an end node, and a safety threshold.
Find the lowest safety value required by any valid start-to-end path while every traversed edge remains under the supplied threshold.
Return -1 when no admissible path exists.
Clarify whether the threshold comparison is strict or inclusive before coding.

Mouse-and-cat safest-path variant (L4 phone screen)

Warm-up: an N x N grid with a start S, a target T, and impassable water cells; with 4-directional moves, decide whether any path from S to T exists.
Main question: the grid also contains a cat at C. The mouse moves only on land, 4-directionally, from S to the cheese at T. Find a path that maximizes the minimum Manhattan distance to the cat over all cells on the path.
This flips the objective (maximize a minimum instead of minimize a maximum), but it is the same bottleneck-path structure once each cell's distance to the cat is known.

Follow-up: Network Delay Time variant

In a directed graph with edge weights, return the maximum shortest-path distance from a source to all nodes, or -1 if any node is unreachable.

EXAMPLES

Grid

```
[ 0  1  2  3  4 ]
[24 23 22 21  5 ]
[12 13 14 15 16 ]
[11 17 18 19 20 ]
[10  9  8  7  6 ]
```

Start=(0,0), End=(4,0) in (row, col) — the bottom-left cell with value 10. Answer: 16 (the path's max cell is 16).

APPROACH

Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation.
Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

HOW TO THINK ABOUT IT

The cost of a path is its largest cell, not the sum. That cost can only grow as the path grows, so Dijkstra still works if you replace 'distance + weight' by max(distance, weight): the first time a cell leaves the heap its cost is final. The cat version flips the objective (maximise a minimum), which is the same shape with a max-heap and each cell scored by its distance to the cat.

Basis: LeetCode 778, Swim in Rising Water (bottleneck path), plus the 'safest path from a cat' flip: maximise the minimum Manhattan distance.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid, start, end):
    for t in sorted(set(all_values(grid))):
        if bfs_reaches(grid, start, end, t):
            return t  # smallest limit
```

TIME	SPACE
O(V^2)	O(V)

WHY IT IS SLOW

One full BFS per distinct value, O(V^2). Dijkstra on the running maximum finds the answer in one pass, O(V log V).

OPTIMAL SOLUTION · Dijkstra on the running maximum

TIME	SPACE
O(V log V)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Maximise the minimum distance to a cat instead.

Same bottleneck shape with the objective flipped: score each cell by its Manhattan distance to the cat, run Dijkstra with a max-heap on the running minimum, and skip water. The same monotonicity argument holds (extending a path can only lower its minimum).

FOLLOW-UP 2 Why is Dijkstra correct for a max-along-path cost?

The cost max(path) never decreases when a path is extended, so a vertex's cheapest cost cannot be improved by going through a vertex that is popped later. That is the only property Dijkstra needs from its cost function (monotone, non-negative extension).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Path Minimizing the Maximum Cell (Swim in
# Rising Water)
#
# A bottleneck-path family: minimize the maximum value
```

```
# encountered on a grid path, then generalize the same
# reasoning to a weighted graph whose edges carry safety
# values and an admissibility threshold. Grid variant
# Input is a grid of heights and start/end cells; movement
# is through four-neighbors. Cost of a path is the maximum
# cell value along it. Return the minimum possible path
# cost. Weighted-graph safety variant Input is a graph
# whose edges carry safety values, a start node, an end
# node, and a safety threshold. Find the lowest safety
# value required by any valid start-to-end path while
# every traversed edge remains under the supplied
# threshold. Return -1 when no admissible path exists.
# Clarify whether the threshold comparison is strict or
# inclusive before coding.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Maximise the minimum distance to a cat instead.
# 2. Why is Dijkstra correct for a max-along-path cost?
#
# INTERVIEW NOTES
# The cost of a path is its largest cell, not the sum.
# That cost can only grow as the path grows, so Dijkstra
# still works if you replace 'distance + weight' by
# max(distance, weight): the first time a cell leaves the
# heap its cost is final. The cat version flips the
# objective (maximise a minimum), which is the same shape
# with a max-heap and each cell scored by its distance to
# the cat.
#
# BRUTE FORCE (say it first): Try every threshold and test
# reachability
# For each candidate value t in increasing order run a BFS
# using only cells <= t and stop at the first t that
# connects start to end. Correct, but it repeats a full
# search per value. Time O(V^2). Space O(V).


from collections import deque
import heapq
```

```
# BRUTE FORCE (say it first): Try every threshold and test
# reachability | time O(V^2)
def bottleneck_brute(g, start, end):
    n, m = len(g), len(g[0])
    for t in sorted({x for row in g for x in row}):
        if (
            g[start[0]][start[1]] > t
            or g[end[0]][end[1]] > t
        ):
            continue
        seen, q = {start}, deque([start])
        while q:
            r, c = q.popleft()
            if (r, c) == end:
                return t
            for a, b in (
                (r + 1, c),
                (r - 1, c),
                (r, c + 1),
                (r, c - 1),
            ):
                if (
                    0 <= a < n
                    and 0 <= b < m
                    and (a, b) not in seen
                    and g[a][b] <= t
                ):
                    seen.add((a, b))
                    q.append((a, b))
    return None
```

```
# HELPERS for the base question
INF = float("inf")


def around(r, c, n, m):
    # four-neighbour cells inside an n x m grid
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < n and 0 <= b < m:
            yield a, b


# BASE QUESTION (optimal): Dijkstra on the running maximum
```

```
# Idea: Heap entries are (cost so far, cell). Popping a
# cell fixes its cost; each neighbour gets max(cost,
# neighbour value). Stop when the end cell is popped. Skip
# stale heap entries.
def min_max_path(g, start, end):
    n, m = len(g), len(g[0])
    # best[cell] = smallest possible "highest cell so far"
    best = {start: g[start[0]][start[1]]}
    heap = [(best[start], start)]
    while heap:
        cost, (r, c) = heapq.heappop(heap)
        # skip outdated heap entries
        if cost > best[(r, c)]:
            continue
        if (r, c) == end:
            return cost
        for a, b in around(r, c, n, m):
            # path cost = highest cell so far or this cell
            nxt = max(cost, g[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, (a, b)))
    return None


# VARIANT: weighted graph with a safety threshold
def min_bottleneck_edge(graph, s, t, limit):
    # graph[u] = [(v, safety)]; every edge must be < limit
    best = {s: 0}
    heap = [(0, s)]
    while heap:
        cost, u = heapq.heappop(heap)
        if cost > best[u]:
            continue
        if u == t:
            return cost
        for v, w in graph.get(u, []):
            # edges at or above the limit are not allowed
            if w >= limit:
                continue
            nxt = max(cost, w)
            if nxt < best.get(v, INF):
                best[v] = nxt
                heapq.heappush(heap, (nxt, v))
    # no admissible path
```

```
    return -1


# RELATED: network delay (plain Dijkstra)
def network_delay(n, edges, src):
    # plain Dijkstra: largest shortest distance, or -1
    graph = {}
    for u, v, w in edges:
        graph.setdefault(u, []).append((v, w))
    dist = {src: 0}
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
```

```
        if d > dist[u]:
            continue
        for v, w in graph.get(u, []):
            if d + w < dist.get(v, INF):
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    # unreachable nodes make the answer -1
    return max(dist.values()) if len(dist) == n else -1


# FOLLOW-UP 1: Maximise the minimum distance to a cat
# instead.
# Answer: Same bottleneck shape with the objective
# flipped: score each cell by its Manhattan distance to
# the cat, run Dijkstra with a max-heap on the running
# minimum, and skip water.
def safest_path_score(grid, start, target, cat):
    # maximise the smallest cat distance ('#' is water)
    n, m = len(grid), len(grid[0])

    def dist(r, c):
        return abs(r - cat[0]) + abs(c - cat[1])

    best = {start: dist(*start)}
    heap = [(-best[start], start)]
    while heap:
        neg, (r, c) = heapq.heappop(heap)
        if -neg < best[(r, c)]:
            continue
        if (r, c) == target:
            return -neg
        for a, b in around(r, c, n, m):
```

```
            if grid[a][b] == "#":
                continue
            # the path is only as safe as its closest cell
            score = min(-neg, dist(a, b))
            if score > best.get((a, b), -1):
                best[(a, b)] = score
                heapq.heappush(heap, (-score, (a, b)))
    return None


if __name__ == "__main__":
    g = [[0, 1, 2, 3, 4], [24, 23, 22, 21, 5],
         [12, 13, 14, 15, 16], [11, 17, 18, 19, 20],
         [10, 9, 8, 7, 6]]
    assert min_max_path(g, (0, 0), (4, 0)) == 16
    # start equals end: just that cell
    assert min_max_path(g, (2, 2), (2, 2)) == 14
    gr = {0: [(1, 5), (2, 9)], 1: [(2, 3)]}
    assert min_bottleneck_edge(gr, 0, 2, 10) == 5
    assert min_bottleneck_edge(gr, 0, 2, 5) == -1
    cat_grid = ["...", ".#.", "..."]
    cat, a, b = (0, 2), (0, 0), (2, 2)
    score = safest_path_score(cat_grid, a, b, cat)
    assert score == 2
    chain = [(0, 1, 2), (1, 2, 3)]
    assert network_delay(3, chain, 0) == 5
    assert network_delay(3, [(0, 1, 2)], 0) == -1
```

NOTES FROM CANDIDATES

The clean solution is a Dijkstra variant: priority queue keyed on the running maximum, relax with new = max(dist[cur], height[nxt]). The same skeleton handles the safety variant with edge values in place of cell heights, plus the admissibility filter.

Also mentionable: binary-search the answer + BFS/DFS reachability check — but Dijkstra is the cleaner one-pass solution.

NOTES FROM CANDIDATES

The grid prompt minimizes a maximum rather than a sum. Be ready to justify why Dijkstra still works: the path cost is monotone as a path grows, so extending a settled vertex never lowers its committed cost.

The safety version preserves the bottleneck-path shape but changes cells to weighted edges and adds an explicit admissibility threshold.

For the plain shortest-path follow-up, do standard Dijkstra with a min-heap; skip stale priority-queue entries (if curDist != dist[u]: continue) and account for unreachable nodes.

In the mouse-and-cat variant, each cell's score is a direct Manhattan distance to a single cat, so it can be computed per cell; clarify whether water cells block only the mouse or also affect distance, and whether S or T themselves count toward the minimum.

PREPARATION

Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation.
Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

MEDIUM | 1Point3Acres | onsite-coding | 15 of 22

Shortest Path Nodes and Escaping a Pursuer

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 212/228

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/shortest-path-nodes-and-pursuer-escape

QUESTION

Unweighted undirected graph: shortest path from Alice's node to a target (BFS). Follow-up 1: all nodes that lie on at least one shortest path. Follow-up 2: Bob starts at another node at the same time; decide whether Alice can follow some shortest path to the target without being caught.

QUESTION

Input: an undirected, unweighted graph; Alice starts at one node and must reach a target node.
Return the shortest path (length) from Alice to the target.
Follow-up 1: find all nodes that can appear on some shortest path from Alice to the target.
Follow-up 2: Bob is at another node. Alice picks one shortest path and starts moving; Bob starts moving at the same time. Decide whether Alice can reach the target safely without being caught by Bob.

APPROACH

Run BFS from both the start and the target; a node v is on some shortest path exactly when distA[v] + distT[v] == distA[target]. For the pursuer follow-up, add a BFS from Bob and compare arrival times node by node along candidate shortest paths; practice turning "exists a safe shortest path" into a search over only those on-path nodes where Alice arrives strictly before Bob.
Test with disconnected targets, Bob starting on the target, and multiple equally short routes where only some are safe.

HOW TO THINK ABOUT IT

BFS from Alice gives dA and BFS from the target gives dT. A node is on some shortest path exactly when dA[v] + dT[v] equals the shortest length. For the pursuer, BFS from Bob gives dB. State the rule you assume: Alice is caught at a node v if Bob can be there no later than she is (dB[v] <= dA[v]). Alice is safe when a shortest path exists whose every node has dB[v] > dA[v].

Basis: Written for this question (BFS layers, as in LeetCode 1976, Number of Ways to Arrive at Destination).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(graph, alice, bob, target):
    dA = bfs_dist(graph, alice)
    dB = bfs_dist(graph, bob)
    paths = all_paths(graph, alice, target)
    for path in paths:
        if len(path) - 1 != dA[target]:
            continue  # not a shortest path
        if all(dB[v] > dA[v] for v in path):
            return True  # Bob is always late
    return False
```

TIME	SPACE
exponential in the worst ... exponential in the worst case	**O(paths)**

WHY IT IS SLOW

The number of shortest paths can be exponential. Three BFS runs (from Alice, the target and Bob) decide every node in O(V + E).

OPTIMAL SOLUTION · Three BFS runs and a safe shortest-path DAG

TIME	SPACE
O(V + E)	**O(V)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if Bob can also meet Alice in the middle of an edge?

Then arriving at the same node at the same time is not the only way to be caught: Alice moving u -> v while Bob moves v -> u swaps places. Add that case by also rejecting an edge (u, v) when dB[v] == dA[u] and dB[u] == dA[v]. Always clarify what 'caught' means first.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Shortest Path Nodes and Escaping a
# Pursuer
#
# Unweighted undirected graph: shortest path from Alice's
# node to a target (BFS). Follow-up 1: all nodes that lie
# on at least one shortest path. Follow-up 2: Bob starts
# at another node at the same time; decide whether Alice
# can follow some shortest path to the target without
# being caught. Input: an undirected, unweighted graph;
# Alice starts at one node and must reach a target node.
# Return the shortest path (length) from Alice to the
# target. Follow-up 1: find all nodes that can appear on
# some shortest path from Alice to the target. Follow-up
# 2: Bob is at another node. Alice picks one shortest path
# and starts moving; Bob starts moving at the same time.
# Decide whether Alice can reach the target safely without
# being caught by Bob.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if Bob can also meet Alice in the middle of an
# edge?
#
# INTERVIEW NOTES
# BFS from Alice gives dA and BFS from the target gives
# dT. A node is on some shortest path exactly when dA[v] +
# dT[v] equals the shortest length. For the pursuer, BFS
# from Bob gives dB. State the rule you assume: Alice is
# caught at a node v if Bob can be there no later than she
# is (dB[v] <= dA[v]). Alice is safe when a shortest path
# exists whose every node has dB[v] > dA[v].
#
# BRUTE FORCE (say it first): Enumerate every shortest
# path
# Depth-first search all simple paths of the shortest
# length from Alice to the target and check each node
# against Bob's arrival time. Correct and the oracle; the
# number of shortest paths can be huge. Time exponential
# in the worst case. Space O(paths).


from collections import deque


# HELPERS for the brute force
```

```
def bfs(adj, s):
```

```
    # distance from s to every reachable node
    d = {s: 0}
    q = deque([s])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if v not in d:
                d[v] = d[u] + 1
                q.append(v)
    return d


# BRUTE FORCE (say it first): Enumerate every shortest
# path | time exponential in the worst case
def escape_brute(adj, a, t, b):
    def bfs(s):
        d = {s: 0}
        q = deque([s])
        while q:
            u = q.popleft()
            for v in adj[u]:
                if v not in d:
                    d[v] = d[u] + 1
                    q.append(v)
        return d

    da, db = bfs(a), bfs(b)
    if t not in da:
        return None, set(), False
    nodes, safe = set(), False

    def go(u, path):
        nonlocal safe
        if len(path) - 1 > da[t]:
            return
        if u == t:
            if len(path) - 1 == da[t]:
                nodes.update(path)
                if all(
                    db.get(v, 10**9) > i
                    for i, v in enumerate(path)
                ):
                    safe = True
            return
        for v in adj[u]:
```

```
            if v not in path:
                go(v, path + [v])

    go(a, [a])
    return da[t], nodes, safe


# HELPERS for the base question
FAR = 10 ** 9


# BASE QUESTION (optimal): Three BFS runs and a safe
# shortest-path DAG
# Idea: dA, dT and dB come from BFS. The nodes on some
# shortest path are those with dA[v] + dT[v] == dA[t].
def shortest_length(adj, a, t):
    # the base question: edges on the shortest path
    return bfs(adj, a).get(t)


# QUESTION FOLLOW-UP 1: all nodes on a shortest path
def nodes_on_shortest_paths(adj, a, t):
    da, dt = bfs(adj, a), bfs(adj, t)
    if t not in da:
        return set()
```

```
    # a node is on some shortest path if the two
    # distances add up to the full length
    return {v for v in da if v in dt
            and da[v] + dt[v] == da[t]}


# QUESTION FOLLOW-UP 2: can Alice escape Bob?
def can_escape(adj, a, t, b, meet_on_edge=False):
    da, db = bfs(adj, a), bfs(adj, b)
    if t not in da:
        return False
    on_path = nodes_on_shortest_paths(adj, a, t)

    def safe(u, v):
        # Bob must reach v strictly after Alice does
        if db.get(v, FAR) <= da[v]:
            return False
        # FOLLOW-UP 1: meet_on_edge=True: they meet inside
        # an edge
        # optional: they swap places on the edge u-v
```

```
        swap = db.get(v, FAR) == da[u]
        swap = swap and db.get(u, FAR) == da[v]
        return not (meet_on_edge and swap)

    # Bob standing on Alice's start catches her at once
    if db.get(a, FAR) <= 0:
        return False
    # walk the shortest-path layers using safe nodes only
    frontier, seen = [a], {a}
    while frontier:
        nxt = []
        for u in frontier:
            for v in adj[u]:
                if v in on_path and da[v] == da[u] + 1 \
                        and v not in seen and safe(u, v):
                    seen.add(v)
                    nxt.append(v)
        frontier = nxt
    return t in seen


if __name__ == "__main__":
    # a square 0-1-3 and 0-2-3 plus a tail 3-4
    adj = {0: [1, 2], 1: [0, 3], 2: [0, 3],
           3: [1, 2, 4], 4: [3]}
    assert shortest_length(adj, 0, 4) == 3
    on_path = nodes_on_shortest_paths(adj, 0, 4)
    assert on_path == {0, 1, 2, 3, 4}
    # Bob waiting at the target catches Alice there
    assert not can_escape(adj, 0, 4, 4)
    chain = {0: [1], 1: [0, 2], 2: [1, 3], 3: [2]}
    # Bob starts at the target and reaches it first
    assert not can_escape(chain, 0, 3, 3)
    # Bob starts behind Alice: she stays ahead
    assert can_escape(chain, 1, 3, 0)
```

NOTES FROM CANDIDATES

Asked as an L4 onsite coding round.
Follow-up 2 has an open definition of "caught": clarify whether it means arriving at the same node at the same time, Bob arriving first, or also meeting on an edge; also clarify whether Alice may choose her shortest path knowing Bob's position.

PREPARATION

Run BFS from both the start and the target; a node v is on some shortest path exactly when distA[v] + distT[v] == distA[target].
For the pursuer follow-up, add a BFS from Bob and compare arrival times node by node along candidate shortest paths; practice turning "exists a safe shortest path" into a search over only those on-path nodes where Alice arrives strictly before Bob.
Test with disconnected targets, Bob starting on the target, and multiple equally short routes where only some are safe.

MEDIUM | IN APP LC 253 | 1Point3Acres | phone-screen, tech-screen, onsite-coding | 16 of 22

Meeting Rooms / Interval Scheduling Variants

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE **ALL TIME 58** **30 DAYS 0** **PRIORITY 44/228**

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: your app's interview approach (LeetCode #253)

LeetCode match: LC 253 Meeting Rooms II

Frequency: Very high | Last asked: 2026-09-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/meeting-rooms-variants

QUESTION

LC 253 (Meeting Rooms II) appears in many forms across phone screen and onsite coding rounds — minimum number of rooms, room assignment, cyclic time, daily server jobs, gate entry/exit. Common follow-ups push toward sweep line, min-heap, or distributed processing.

The base task in every variant: given a list of intervals (start, end) — or pickup/return pairs, meeting time pairs, server job runs — answer one or more of:

Min rooms / cars / servers needed to honor all intervals concurrently (the canonical min-meeting-rooms problem).
Assignment: which interval goes to which room/car, so the caller can use the rental record.
Single-interval overlap test between two intervals.

Variant: parking-lot capacity

Arrival and departure intervals represent cars occupying a parking lot. Compute the peak simultaneous occupancy using the same interval-overlap model.

Variant: cyclic time (24h wrap)

Intervals wrap around midnight; if an interval ends after the day boundary, it carries into the next day.
Follow-up: "if N is huge, how do you optimize?" — expect a sweep-line discussion.

QUESTION

Variant: daily-recurring server jobs

Same set of jobs runs every day. If a job can't finish in one day, truncate at midnight and resume next day.
A server can run multiple identical jobs that overlap on the same machine; different jobs need separate servers.
Follow-up: a single job that runs many days needs its own dedicated server.

Variant: on-call rotation consolidation

Input: on-call rotations {name, start, end} using half-open intervals.
Output maximal contiguous segments during which the active on-call set does not change; omit gaps with nobody on call.
Segments are sorted by start, and names in each segment are the active set for that time range.
Clean approach: sort starts, keep a min-heap by end time, and emit a segment whenever the next start or next end changes the active set.

Variant: gate entry / exit with priority rules

Array of (timestamp, action) where action in {enter, exit}; each index is a different person.
Return for each person what time they actually enter/exit, applying tie-break rules:

If the prior tick had an enter, enter wins on a tie.
If the prior tick had an exit, exit wins on a tie.
If the prior tick had no activity, exit wins.
Within the winning class, lower index wins.

Optimization: instead of stepping time tick-by-tick, jump through the sorted input timestamps to reach O(n).

Variant: Google Calendar free-time intersection

Input: requested attendees plus each person's busy calendar intervals.
Return every time range where all requested attendees are free.
Normalize each person's busy blocks into free gaps, then intersect the candidate ranges across attendees.

Variant: car-rental assignment

Input: rental orders containing an order ID, pickup time, and return time.
Assign every order to a car so that no assigned intervals overlap, minimize the number of cars, and return each car's complete order list. Model each car explicitly with its own rental record.
A current output variant instead returns a dictionary from each reservation ID to its assigned car ID; the follow-up asks for the final fleet size.
A return at time t and another pickup at time t may reuse the same car.
Follow-up: if the fleet is capped at K cars and all requests cannot be served, define how the allocator detects and handles the failure.

Variant: exact-concurrency intervals

QUESTION

Compute the maximum number of simultaneously active meetings.
Given a target concurrency X, also return every maximal time range during which exactly X meetings are active.
Define the event ordering when one meeting starts at the same timestamp another ends; this choice controls whether a zero-width overlap is counted.

Variant: maximum-profit job scheduling

Each job has startTime, endTime, and profit. Choose a set of non-overlapping jobs maximizing total profit. The candidate identified it as LC 1235 (Maximum Profit in Job Scheduling). Asked as the first question of an L4 onsite round, followed by an unrelated range-query problem in the same slot.

On-call employee metadata variant

Each duty record includes employee ID, employee name, start time, and end time. Return the employee count and identifying information for each time segment.

EXAMPLES

```
Canonical min-meeting-rooms example:
    [[0,30],[5,10],[15,20]] -> 2.
On-call consolidation: A[10,50], B[20,60], C[30,40],
    D[30,40] -> [10,20]: A, [20,30]: A,B, [30,40]:
    A,B,C,D, [40,50]: A,B, [50,60]: B.
Single-interval overlap:
    intervalsOverlap([1,5],[4,7]) -> true.
```

Long-running job: a 36-hour daily job is impossible under the truncate rule; reserve a separate server.

APPROACH

Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

HOW TO THINK ABOUT IT

The number of rooms needed is the largest number of intervals overlapping at any moment. Turn each interval into +1 at its start and -1 at its end, sort the events (ends before starts at the same time) and keep a running count. To say which room each meeting gets, keep a min-heap of (end time, room) and reuse the room that frees up first.

Basis: LeetCode 253, Meeting Rooms II, with the common variants: room assignment and cyclic (24-hour) time.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(intervals):
    best = 0
    for start, _ in intervals:
        busy = 0
        for s, e in intervals:
            if s <= start < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Every interval is compared with every other, O(n^2). Sorting start and end events and keeping a running count is O(n log n).

OPTIMAL SOLUTION · Sweep line over start and end events

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Meetings wrap around midnight (cyclic time).

Split every wrapped meeting [s, e) with s >= e into [s, 24h) and [0, e) and run the same sweep: the overlap at any clock time is unchanged. A meeting that is exactly 24 hours long covers the whole day.

FOLLOW-UP 2 The input is huge and spread over many machines.

Each shard sorts its events and produces a compressed timeline of (time, delta) changes; merge the sorted timelines with a k-way merge and take the running maximum, or add the per-shard counts at fixed time buckets. The answer is a max over time of a sum over shards.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Meeting Rooms / Interval Scheduling
# Variants
#
# LC 253 (Meeting Rooms II) appears in many forms across
# phone screen and onsite coding rounds — minimum number
# of rooms, room assignment, cyclic time, daily server
# jobs, gate entry/exit. Common follow-ups push toward
# sweep line, min-heap, or distributed processing. The
# base task in every variant: given a list of intervals
# (start, end) — or pickup/return pairs, meeting time
# pairs, server job runs — answer one or more of: Min
```

```python
# rooms / cars / servers needed to honor all intervals
# concurrently (the canonical min-meeting-rooms problem).
# Assignment: which interval goes to which room/car, so
# the caller can use the rental record. Single-interval
# overlap test between two intervals. Variant: parking-lot
# capacity Arrival and departure intervals represent cars
# occupying a parking lot. Compute the peak simultaneous
# occupancy using the same interval-overlap model.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Meetings wrap around midnight (cyclic time).
# 2. The input is huge and spread over many machines.
#
# INTERVIEW NOTES
# The number of rooms needed is the largest number of
# intervals overlapping at any moment. Turn each interval
# into +1 at its start and -1 at its end, sort the events
# (ends before starts at the same time) and keep a running
# count. To say which room each meeting gets, keep a min-
# heap of (end time, room) and reuse the room that frees
# up first.
#
# BRUTE FORCE (say it first): Count the overlap at every
# start time
# For every interval count how many intervals contain its
# start point and take the maximum. Correct and simple;
# quadratic. Time O(n^2). Space O(1).


import heapq
from collections import Counter


# BRUTE FORCE (say it first): Count the overlap at every
```

```python
# start time | time O(n^2)
def min_rooms_brute(intervals):
    return max(
        (
            sum(
                1
                for s2, e2 in intervals
                if s2 <= s < e2
            )
            for s, e in intervals
        ),
        default=0,
    )


# BASE QUESTION (optimal): Sweep line over start and end
# events
# Idea: Create (time, +1) for each start and (time, -1)
# for each end, sort so that an end at time t comes before
# a start at t (rooms free up the moment a meeting ends)
# and track the running maximum.
def min_rooms(intervals):
    # +1 at every start, -1 at every end
    events = []
    for s, e in intervals:
        events.append((s, 1))
        events.append((e, -1))
    # at equal times ends (-1) sort before starts (+1)
    events.sort()
    cur = best = 0
    for _, delta in events:
        cur += delta
        best = max(best, cur)
    return best


# VARIANT: which room for each meeting
def assign_rooms(intervals):
```

```python
    # which room does each meeting get?
    order = sorted(range(len(intervals)),
                   key=lambda i: intervals[i])
    heap, rooms, used = [], [0] * len(intervals), 0
    for i in order:
        start, end = intervals[i]
        # reuse the room that frees first, if it is free
```

```python
        if heap and heap[0][0] <= start:
            _, room = heapq.heappop(heap)
        else:
            room, used = used, used + 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms


# VARIANT: on-call consolidation
def oncall_segments(rotations):
    # rotations: (name, start, end), half-open intervals
    delta = {}
    for name, s, e in rotations:
        delta.setdefault(s, []).append((name, 1))
        delta.setdefault(e, []).append((name, -1))
    active, out, times = Counter(), [], sorted(delta)
    for t, nxt in zip(times, times[1:]):
        # apply every change that happens at time t
        for name, d in delta[t]:
            active[name] += d
        who = frozenset(n for n in active if active[n])
        # skip gaps; merge with the previous equal segment
        if not who:
            continue
        if out and out[-1][1] == t and out[-1][2] == who:
            out[-1] = (out[-1][0], nxt, who)
        else:
            out.append((t, nxt, who))
    return out


# FOLLOW-UP 1: Meetings wrap around midnight (cyclic
# time).
# Answer: Split every wrapped meeting [s, e) with s >= e
# into [s, 24h) and [0, e) and run the same sweep: the
# overlap at any clock time is unchanged. A meeting that
# is exactly 24 hours long covers the whole day.
def min_rooms_cyclic(intervals, day=24 * 60):
    parts = []
    for s, e in intervals:
        if s < e:
            parts.append((s, e))
        else:
            # wraps past midnight: split at the boundary
```

```python
            parts += [(s, day), (0, e)]
    return min_rooms(parts)


if __name__ == "__main__":
    base = [(0, 30), (5, 10), (15, 20)]
    assert min_rooms(base) == 2
    # touching meetings share a room
    assert min_rooms([(5, 8), (8, 9)]) == 1
    rooms = assign_rooms(base)
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]
    wrap = [(22 * 60, 2 * 60), (23 * 60, 60), (0, 3 * 60)]
    assert min_rooms_cyclic(wrap) == 3
    rot = [("A", 1, 5), ("B", 3, 8)]
    assert oncall_segments(rot) == [
        (1, 3, frozenset("A")),
        (3, 5, frozenset("AB")),
        (5, 8, frozenset("B"))]
    # a gap with nobody on call is left out
```

```
gap = [("A", 1, 2), ("B", 4, 5)]
assert len(oncall_segments(gap)) == 2
```

NOTES FROM CANDIDATES

Asked in Round 1 coding, Round 2 onsite coding, phone screen, and as a warm-up in the FDE loop. Almost every other coding loop contains some flavor of this family.
Interviewer commonly chains two follow-ups: (1) reconstruct assignment, (2) scale out to many shards / regions.
For the assignment follow-up, min-heap keyed on end time is the clean answer: pop the earliest-ending room, push the new interval.
For "distributed across shards", expect open-ended discussion: per-shard sweep, cross-shard merge, reconciliation; the interviewer often acknowledges the question is intentionally vague.

The on-call metadata variant includes detailed complexity follow-ups: account for heap operations, stored employee information, and output size separately, and explain each symbol in the complexity expression.

PREPARATION

Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

MEDIUM | 1Point3Acres | onsite-coding | 17 of 22

Range Duplicate Check with Constant-Time Queries

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 211/228

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/range-duplicate-check-constant-time-queries

QUESTION

Decide whether the elements in a range of an array are all unique. Progression: set-based duplicate check -> sliding window -> many range queries where reads dominate, requiring O(1) per query after linear-time preprocessing / updates with linear space.

QUESTION

Base: given an array and a range [l, r], determine whether the elements in that range are all unique.
The problem is staged:

Easy: duplicate detection with a set.
Sliding-window duplicate checking.
Many queries over different ranges: optimize for a read-heavy, write-light workload.

Target for the final stage: O(1) per query, preprocessing (and updates, in one version) in linear time, linear space.

APPROACH

Precompute, for each index i, the previous index holding the same value (prev[i]) with a single hashmap pass.
Turn that into an O(1) query: a range [l, r] is duplicate-free exactly when every prev[j] < l for j in the range — precompute a running "furthest start that keeps [start, j] unique" array so each query is one comparison.
Explain the cost of an update to one element (which prev entries change) and why it stays linear.

HOW TO THINK ABOUT IT

A range [l, r] has a duplicate exactly when some index i in it has its next equal element at an index <= r. Let next[i] be the next index holding the same value and reach[l] the smallest next[i] over all i >= l. Then the range is all-unique iff reach[l] > r, a single comparison per query after one linear pass.

Basis: Written for this question (a precomputed next-occurrence array, similar in spirit to LeetCode 3, Longest Substring Without Repeating Characters).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def all_unique(a, l, r):
    seen = set()
    for x in a[l:r + 1]:
        if x in seen:
            return False
        seen.add(x)
    return True
```

TIME	SPACE
O(r - l) per query	O(r - l)

WHY IT IS SLOW

Each query costs the length of the range. A next-occurrence array with suffix minimums answers each query with one comparison.

OPTIMAL SOLUTION · Next-occurrence array with suffix minimum

TIME	SPACE
O(n) preprocessing, O(1) per query	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The array changes between queries.

Rebuild next and reach in O(n) after a batch of changes when reads dominate. If single-element updates are frequent, keep per-value sorted index lists and a segment tree of next-occurrence minima so an update and a query cost O(log n) instead.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Range Duplicate Check with Constant-Time
# Queries
#
# Decide whether the elements in a range of an array are
# all unique. Progression: set-based duplicate check ->
# sliding window -> many range queries where reads
# dominate, requiring O(1) per query after linear-time
# preprocessing / updates with linear space. Base: given
# an array and a range [l, r], determine whether the
# elements in that range are all unique. The problem is
# staged: Easy: duplicate detection with a set. Sliding-
# window duplicate checking. Many queries over different
# ranges: optimize for a read-heavy, write-light workload.
# Target for the final stage: O(1) per query,
# preprocessing (and updates, in one version) in linear
# time, linear space.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The array changes between queries.
#
# INTERVIEW NOTES
# A range [l, r] has a duplicate exactly when some index i
# in it has its next equal element at an index <= r. Let
# next[i] be the next index holding the same value and
# reach[l] the smallest next[i] over all i >= l. Then the
# range is all-unique iff reach[l] > r, a single
# comparison per query after one linear pass.
#
# BRUTE FORCE (say it first): Put the range in a set
# Insert the elements of the range into a set and compare
# sizes. Correct; every query costs the length of the
# range. Time O(r - l) per query. Space O(r - l).


from collections import Counter


# BRUTE FORCE (say it first): Put the range in a set |
# time O(r - l) per query
def range_unique_brute(a, l, r):
    return len(set(a[l : r + 1])) == r - l + 1


# HELPERS for the base question
```

```
INF = float("inf")
```

```


# EASY VERSION (say it first)
def unique_with_set(a, l, r):
    # easy version: put the range in a set
    return len(set(a[l:r + 1])) == r - l + 1


# STAGE 2: sliding window
def longest_unique_window(a):
    # stage 2: sliding window with a counter
    seen, left, best = Counter(), 0, 0
    for right, x in enumerate(a):
        seen[x] += 1
        # shrink until x occurs only once
        while seen[x] > 1:
            seen[a[left]] -= 1
            left += 1
        best = max(best, right - left + 1)
    return best


# BASE QUESTION (optimal): Next-occurrence array with
# suffix minimum
# Idea: Scan right to left with a map from value to its
# latest index to fill next[i] (infinity if none). Then
# reach[i] = min(next[i], reach[i + 1]). Query(l, r) is
# reach[l] > r.
# FOLLOW-UP 1: The array changes between queries.
# Answer: Rebuild next and reach in O(n) after a batch of
# changes when reads dominate.
class RangeUnique:
    """O(n) build, then O(1) per query."""

    def __init__(self, a):
        self.a = list(a)
        self.rebuild()

    def rebuild(self):
        a, n = self.a, len(self.a)
        # nxt[i] = next index holding the same value
        nxt, last = [INF] * n, {}
        for i in range(n - 1, -1, -1):
            nxt[i] = last.get(a[i], INF)
            last[a[i]] = i
```

```
        # reach[i] = earliest repeat from index i onward
        self.reach = [INF] * (n + 1)
        for i in range(n - 1, -1, -1):
            self.reach[i] = min(nxt[i], self.reach[i + 1])

    def unique(self, l, r):
        # no repeat starts in [l, r] and ends inside it
        return self.reach[l] > r

    # FOLLOW-UP 1: the array changes between queries
    def update(self, i, value):
        # reads dominate, so a linear rebuild is fine
        self.a[i] = value
        self.rebuild()


if __name__ == "__main__":
    a = [1, 2, 3, 1, 4]
    q = RangeUnique(a)
    for l in range(len(a)):
        for r in range(l, len(a)):
            plain = unique_with_set(a, l, r)
            assert q.unique(l, r) == plain
    assert q.unique(0, 2) and not q.unique(0, 3)
    assert longest_unique_window(a) == 4
```

```
# an update can create or remove a duplicate
q.update(3, 9)
assert q.unique(0, 4)
q.update(0, 9)
assert not q.unique(0, 3)
```

NOTES FROM CANDIDATES

In one L4 onsite, this was the second question of a round that opened with weighted job scheduling; the interviewer asked specifically for the precomputation that makes each query O(1). The interviewer states the complexity targets explicitly, so the round is judged on hitting them, not on a working brute force. Clarify whether the array can change between queries (one version mentions linear-time updates) and whether ranges are inclusive.

PREPARATION

Precompute, for each index i, the previous index holding the same value (prev[i]) with a single hashmap pass.
Turn that into an O(1) query: a range [l, r] is duplicate-free exactly when every prev[j] < l for j in the range — precompute a running "furthest start that keeps [start, j] unique" array so each query is one comparison.
Explain the cost of an update to one element (which prev entries change) and why it stays linear.

MEDIUM | 1Point3Acres | phone-screen | 18 of 22

Grid Path Count with Diagonal Moves and Checkpoints

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | ALL TIME 48.9 | 30 DAYS 33 | PRIORITY 35/228

ABOUT THIS QUESTION

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #62 Unique Paths

LeetCode match: LC 62 Unique Paths (related)

Frequency: Single report | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/grid-path-count-three-moves-checkpoints

QUESTION

Count paths in an n x m matrix from (n-1, 0) to (n-1, m-1) moving up-right, right, or down-right. Follow-ups: count paths visiting all checkpoints; then checkpoints must be visited in order, with the observation that each move changes column, and optimize space.

QUESTION

Matrix of size n x m. Start at (n-1, 0), end at (n-1, m-1).
Allowed moves from (r, c): up-right (r-1, c+1), right (r, c+1), down-right (r+1, c+1); stay inside the matrix.
Return the total number of distinct paths from start to end.
Follow-up 1: the matrix contains checkpoints; count paths that visit all checkpoints.
Follow-up 2: the checkpoints must be visited in order. Since every move changes the column, each column can contain at most one visited checkpoint. Count the paths and optimize space.

APPROACH

Write the column-by-column DP where dp[c][r] depends only on dp[c-1][r-1..r+1], then reduce it to a single rolling column.
For checkpoints, practice the "segment" approach: sort by column, and between consecutive checkpoints only allow paths that pass through them (zero out other rows in a checkpoint's column).
Test: n = 1 (only straight right), m = 1, a checkpoint at the start or end, and two checkpoints in the same column.

HOW TO THINK ABOUT IT

Every move goes exactly one column to the right, so a path visits each column exactly once and the state is just 'which row am I in at this column'. That is a DP over columns with a single rolling array. A checkpoint (r, c) simply says the path must be in row r at column c, so you zero every other row of that column.

Basis: LeetCode 62, Unique Paths, with up-right / right / down-right moves, and checkpoints the path must visit.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def count(r, c, seen):
    if c == m - 1:
        done = r == n - 1 and checkpoints <= seen
        return 1 if done else 0
    total = 0
    for dr in (-1, 0, 1):
        nr = r + dr
        if 0 <= nr < n:
            nxt = seen | {(nr, c + 1)}
            total += count(nr, c + 1, nxt)
    return total
answer = count(n - 1, 0, {(n - 1, 0)})
```

TIME	SPACE
O(3^m)	O(m)

WHY IT IS SLOW

Up to 3^m paths. Every move advances one column, so a dp over (column, row) with a rolling array counts them in O(n * m).

OPTIMAL SOLUTION · Column DP with a rolling array and checkpoint filters

TIME	SPACE
O(n * m)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Two checkpoints share a column.

No path can visit both, because a path is in exactly one row per column, so the answer is 0 (the code returns it before the DP). Checkpoints at the start or end cell are fine: they only filter rows of the first or last column.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Grid Path Count with Diagonal Moves and
# Checkpoints
#
# Count paths in an n x m matrix from (n-1, 0) to (n-1,
# m-1) moving up-right, right, or down-right. Follow-ups:
# count paths visiting all checkpoints; then checkpoints
# must be visited in order, with the observation that each
# move changes column, and optimize space. Matrix of size
# n x m. Start at (n-1, 0), end at (n-1, m-1). Allowed
# moves from (r, c): up-right (r-1, c+1), right (r, c+1),
# down-right (r+1, c+1); stay inside the matrix. Return
# the total number of distinct paths from start to end.
# Follow-up 1: the matrix contains checkpoints; count
# paths that visit all checkpoints. Follow-up 2: the
# checkpoints must be visited in order. Since every move
# changes the column, each column can contain at most one
# visited checkpoint. Count the paths and optimize space.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Two checkpoints share a column.
#
# INTERVIEW NOTES
# Every move goes exactly one column to the right, so a
# path visits each column exactly once and the state is
# just 'which row am I in at this column'. That is a DP
# over columns with a single rolling array. A checkpoint
# (r, c) simply says the path must be in row r at column
# c, so you zero every other row of that column.
#
# BRUTE FORCE (say it first): Enumerate every path
# Depth-first search over all move sequences, counting
# those that end at the bottom-right with the checkpoints
# visited. Exponential in the number of columns. Time
# O(3^m). Space O(m).


# BRUTE FORCE (say it first): Enumerate every path | time
# O(3^m)
def count_paths_brute(
    n, m, checkpoints=(), ordered=False
):
    cps = [tuple(c) for c in checkpoints]
    total = 0

    def go(r, c, seen):
        nonlocal total
        if (r, c) in cps:
            seen = seen + [(r, c)]
        if c == m - 1:
            if r == n - 1 and (
                seen == cps
                if ordered
                else sorted(seen) == sorted(cps)
            ):
                total += 1
            return
        for dr in (-1, 0, 1):
            if 0 <= r + dr < n:
                go(r + dr, c + 1, seen)

    go(n - 1, 0, [])
    return total


# HELPERS for the base question
def step(ways):
    # next column from the current column only
    n = len(ways)
    return [(ways[r - 1] if r else 0) + ways[r]
            + (ways[r + 1] if r + 1 < n else 0)
            for r in range(n)]


def keep_row(ways, row):
    # a checkpoint forces the path through one row
    return [x if r == row else 0
            for r, x in enumerate(ways)]


# FOLLOW-UP 1: Two checkpoints share a column.
# Answer: No path can visit both, because a path is in
# exactly one row per column, so the answer is 0 (the code
# returns it before the DP). Checkpoints at the start or
# end cell are fine: they only filter rows of the first or
# last column.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def checkpoint_rows(checkpoints, ordered):
    # column -> required row; None if impossible
    rows = {}
    for r, c in checkpoints:
        # two different checkpoints in one column: no path
        if rows.setdefault(c, r) != r:
            return None
    if len(rows) != len(checkpoints):
        return None
    cols = [c for _, c in checkpoints]
    # ordered: they must be listed left to right
    if ordered and cols != sorted(cols):
        return None
    return rows


# BASE QUESTION (optimal): Column DP with a rolling array
# and checkpoint filters
# Idea: ways[r] is the number of paths that are in row r
# at the current column. The next column is ways'[r] =
# ways[r - 1] + ways[r] + ways[r + 1]. At a checkpoint
# column keep only the checkpoint's row.
def count_paths(n, m, checkpoints=(), ordered=False):
    rows = checkpoint_rows(list(checkpoints), ordered)
    if rows is None:
        return 0
    # one rolling column is enough: O(n) space
    ways = [0] * n
    ways[n - 1] = 1
    if 0 in rows:
        ways = keep_row(ways, rows[0])
```

```
    for c in range(1, m):
        ways = step(ways)
        if c in rows:
            ways = keep_row(ways, rows[c])
    return ways[n - 1]


# HELPERS for the checks below
def count_paths_table(n, m):
    # full table: ways[c][r] paths reach row r, column c
    ways = [[0] * n for _ in range(m)]
    ways[0][n - 1] = 1
    for c in range(1, m):
        for r in range(n):
            # arrive from up-right, right or down-right
            for pr in (r - 1, r, r + 1):
                if 0 <= pr < n:
```

```
                    ways[c][r] += ways[c - 1][pr]
    return ways[m - 1][n - 1]


if __name__ == "__main__":
    assert count_paths(3, 3) == 2
    assert count_paths_table(3, 3) == 2
    assert count_paths(1, 5) == 1
    assert count_paths(3, 1) == 1
    assert count_paths_table(4, 6) == count_paths(4, 6)
    # force the middle column through row 1
    assert count_paths(3, 3, [(1, 1)]) == 1
    # two checkpoints in one column: impossible
    assert count_paths(3, 3, [(0, 1), (1, 1)]) == 0
    # ordered checkpoints must go left to right
    assert count_paths(3, 4, [(1, 1), (1, 2)], True) == 1
    assert count_paths(3, 4, [(1, 2), (1, 1)], True) == 0
```

NOTES FROM CANDIDATES

Asked in an L4 phone interview.
Every move advances exactly one column, so the path visits each column exactly once — this is the key structural fact the follow-ups build on.
Clarify for follow-up 1 what happens if two checkpoints share a column (no path can visit both) and whether checkpoints are given as a list or marked in the grid.

PREPARATION

Write the column-by-column DP where dp[c][r] depends only on dp[c-1][r-1..r+1], then reduce it to a single rolling column.
For checkpoints, practice the "segment" approach: sort by column, and between consecutive checkpoints only allow paths that pass through them (zero out other rows in a checkpoint's column).
Test: n = 1 (only straight right), m = 1, a checkpoint at the start or end, and two checkpoints in the same column.

MEDIUM | 1Point3Acres | onsite-coding | 19 of 22

Expression Tree Single-Leaf Mutation

ASKED 2026-09-27 | 23 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 203/228

ABOUT THIS QUESTION

Topic: Expression Parsing and Evaluation

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-27 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/expression-tree-single-leaf-mutation

QUESTION

Evaluate a boolean expression tree, then answer the root value after each single-leaf flip. Current variants include XOR/OR/AND/NOT operators, no parent pointer on the node interface, and an expectation that the candidate cache parent links or paths during the initial traversal.SWEtreedfssimulationmediumstate-machineFrequencyLowLast asked2026-09-27Stageonsite-coding

APPROACH

Practice tree evaluation for boolean expression trees.
Add parent pointers or store root paths during leaf collection to support path-only recomputation.
Drill AND / OR / XOR truth tables so updates are mechanical.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which operators exist (AND, OR, XOR, NOT), whether XOR and others are binary only, whether nodes have parent pointers, and whether leaves are flipped one at a time and then restored or cumulatively.

BRUTE FORCE TO SAY OUT LOUD

For each leaf flip it, evaluate the whole tree, and flip it back. O(n^2).

OPTIMAL IDEA

Evaluate once and cache each node's value. During that traversal record each leaf's path to the root (or parent links). To flip a leaf, recompute only the nodes on its path upward, stopping early when a parent's value does not change because the root value is then the same as before. That is O(depth) per flip.

DRY-RUN THESE CASES

A single leaf tree, a NOT at the root, XOR chains where every flip changes the root, OR with a true sibling (early stop).

LIKELY FOLLOW-UPS

Several flips applied together, or an undo.

PITFALLS

Mutating the cached values without restoring them between independent flips.

HOW TO THINK ABOUT IT

Flipping one leaf can only change the nodes on its path to the root, so cache every node's value and recompute only that path. The path is found through parent links you build during the first traversal. Stop early when a recomputed node keeps its old value, because nothing above it can change.

Basis: Written; incremental evaluation on a tree. Assumed rules (confirm them): leaves hold booleans; internal nodes are AND, OR, XOR (binary) or NOT (unary); report the root value after flipping each leaf on its own (flip, read the root, restore).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(root, leaves):
    answers = []
    for leaf in leaves:
        leaf.value = not leaf.value
        answers.append(evaluate(root))
        leaf.value = not leaf.value  # flip back
    return answers
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

L leaves times an O(n) evaluation: O(n^2). Caching every node's value and recomputing only the path to the root (stopping when a value does not change) costs O(depth) per leaf.

OPTIMAL SOLUTION · Cache values and parents, recompute only the path with early stop

TIME	SPACE
O(n + L * depth)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The node interface has no parent pointer.

Build the path while collecting leaves: carry the list of ancestors with each stack entry, and use that list for the walk up. It costs O(depth) memory per leaf; the parent dict does the same job with O(n) total.

FOLLOW-UP 2 Apply several flips at once, or flips that stay.

The same cached values work: toggle, walk up with early stop, and keep the changes. For many flips between queries, recompute each affected path in turn.

FOLLOW-UP 3 Which leaves change the answer at all?

Compute top-down 'sensitivity': a leaf matters if every ancestor's other input lets it through (AND with true, OR with false, any XOR or NOT). One pass gives all of them in O(n).

FOLLOW-UP 4 Which leaves can change the answer at all?

Propagate a 'matters' flag downward: through AND the other side must be true, through OR the other side must be false, through XOR and NOT it always passes. A leaf matters exactly when the flag reaches it (checked against flipping every leaf). Subtree values are cached in practice, giving O(n).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Expression Tree Single-Leaf Mutation
#
# Evaluate a boolean expression tree, then answer the root
# value after each single-leaf flip. Current variants
# include XOR/OR/AND/NOT operators, no parent pointer on
# the node interface, and an expectation that the
# candidate cache parent links or paths during the initial
# traversal.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The node interface has no parent pointer.
# 2. Apply several flips at once, or flips that stay.
# 3. Which leaves change the answer at all?
# 4. Which leaves can change the answer at all?
#
# INTERVIEW NOTES
# Flipping one leaf can only change the nodes on its path
# to the root, so cache every node's value and recompute
# only that path. The path is found through parent links
# you build during the first traversal. Stop early when a
# recomputed node keeps its old value, because nothing
# above it can change.
#
# BRUTE FORCE (say it first): Flip, re-evaluate the whole
# tree, flip back
# For each of the L leaves, toggle it, evaluate the entire
# tree, and toggle it back. It is the correct baseline and
# the oracle for the cached version. Time O(n^2). Space
# O(h).


# HELPERS for the brute force
def _apply(op, a, b=None):
    return {
        "AND": lambda: a and b,
        "OR": lambda: a or b,
        "XOR": lambda: a != b,
        "NOT": lambda: not a,
    }[op]()


def _eval_full(node):
    if node.op is None:
        return node.val
    if node.op == "NOT":
```

```
        return not _eval_full(node.left)
    return _apply(
        node.op,
```

```
        _eval_full(node.left),
        _eval_full(node.right),
    )


def _leaves(root):
    out, stack = [], [root]
    while stack:
        n = stack.pop()
        if n.op is None:
            out.append(n)
        else:
            if n.right:
                stack.append(n.right)
            stack.append(n.left)
    return out


# BRUTE FORCE (say it first): Flip, re-evaluate the whole
# tree, flip back | time O(n^2)
class XNode:
    def __init__(
        self, op=None, left=None, right=None, val=None
    ):
        self.op, self.left, self.right, self.val = (
            op,
            left,
            right,
            val,
        )


def flips_brute(root):
    out = []
    for leaf in _leaves(root):
        leaf.val = not leaf.val
        out.append(_eval_full(root))
        leaf.val = not leaf.val
    return out


# HELPERS for the base question
```

```
def combine(op, a, b=None):
    # value of one gate from its input values
    if op == "AND":
        return a and b
    if op == "OR":
        return a or b
    if op == "XOR":
        return a != b
    # NOT has one input
    return not a


def right_value(value, node):
    # NOT has no right child
    return value[id(node.right)] if node.right else None


def build_cache(root):
    # one traversal caches parents, values and leaves
    parent, value, leaves = {id(root): None}, {}, []
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                parent[id(child)] = node
                stack.append(child)
    # reversed order visits children before parents
    for node in reversed(order):
        if node.op is None:
            value[id(node)] = node.val
            leaves.append(node)
        else:
            a = value[id(node.left)]
            b = right_value(value, node)
            value[id(node)] = combine(node.op, a, b)
    return parent, value, leaves


# BASE QUESTION (optimal): Cache values and parents,
# recompute only the path with early stop
# Idea: One traversal stores every node's value and its
# parent.
def flip_answers(root):
```

```
    parent, value, leaves = build_cache(root)
    answers = []
    for leaf in leaves:
        undo, node = [], leaf
        new = not value[id(leaf)]
        # walk up only while a value really changes
        while node is not None and value[id(node)] != new:
            undo.append((node, value[id(node)]))
            value[id(node)] = new
            node = parent[id(node)]
            if node is None:
                break
            a = value[id(node.left)]
            b = right_value(value, node)
            new = combine(node.op, a, b)
        answers.append(value[id(root)])
        # put every changed value back for the next leaf
        for node, old in reversed(undo):
            value[id(node)] = old
    return leaves, answers


# FOLLOW-UP 3: Which leaves change the answer at all?
# Answer: Compute top-down 'sensitivity': a leaf matters
# if every ancestor's other input lets it through (AND
# with true, OR with false, any XOR or NOT). One pass
# gives all of them in O(n).
# FOLLOW-UP 4: Which leaves can change the answer at all?
# Answer: Propagate a 'matters' flag downward: through AND
# the other side must be true, through OR the other side
# must be false, through XOR and NOT it always passes.
def sensitive_leaves(root):
    # leaves whose flip can change the root value
    _, value, _ = build_cache(root)
    found, stack = [], [(root, True)]
    while stack:
        node, matters = stack.pop()
        if node.op is None:
            if matters:
                found.append(node)
            continue
        l, r = node.left, node.right
        # the flag passes if the other side lets it
        left_ok = right_ok = matters
        if node.op == "AND":
```

```
            left_ok = matters and value[id(r)]
            right_ok = matters and value[id(l)]
        elif node.op == "OR":
            left_ok = matters and not value[id(r)]
            right_ok = matters and not value[id(l)]
        stack.append((l, left_ok))
        if r is not None:
            stack.append((r, right_ok))
    return found
```

```python
# HELPERS for the checks below
class Node:
    def __init__(self, op=None, left=None,
                 right=None, val=None):
        # op is AND, OR, XOR, NOT, or None for a leaf
        self.op, self.val = op, val
        self.left, self.right = left, right


def evaluate(node):
    # plain recursive evaluation (also the checker)
    if node.op is None:
        return node.val
    a = evaluate(node.left)
    b = evaluate(node.right) if node.right else None
    return combine(node.op, a, b)


if __name__ == "__main__":
    t, f = Node(val=True), Node(val=False)
    tree = Node("XOR", Node("AND", t, Node(val=True)),
                Node("NOT", f))
    leaves, answers = flip_answers(tree)
    # every cached answer matches a full re-evaluation
    for leaf, ans in zip(leaves, answers):
        leaf.val = not leaf.val
        assert evaluate(tree) == ans
        leaf.val = not leaf.val
    # the tree is unchanged afterwards
    assert evaluate(tree) is False
    # sensitivity agrees with really flipping each leaf
    base = evaluate(tree)
    for leaf in leaves:
        leaf.val = not leaf.val
        changes = evaluate(tree) != base
        leaf.val = not leaf.val
        assert changes == (leaf in sensitive_leaves(tree))
```

NOTES FROM CANDIDATES

Brute force is: collect leaves by inorder traversal, flip each leaf, and re-evaluate the whole tree.
The natural optimization is to cache subtree values and only recompute the path from the flipped leaf to the root.
Clarify whether XOR is binary-only and whether the tree is guaranteed valid.
Early-stop optimization is expected: after a leaf flip, recompute upward only until a parent value stays unchanged, then reuse the previous root value.
Settle how intermediate update state will be represented before implementation. In an L4 onsite, uncertainty about that representation caused a promising approach to fall back to brute-force code and contributed to negative coding feedback.

PREPARATION

Practice tree evaluation for boolean expression trees.
Add parent pointers or store root paths during leaf collection to support path-only recomputation.
Drill AND / OR / XOR truth tables so updates are mechanical.

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | phone-screen | 20 of 22

Active Communication Top Users

ASKED 2026-09-22 | 28 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 228/228**

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-22 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/active-communication-top-users

QUESTION

Process a continuous stream of user pairs such as **`A-B`** and **`B-C`**; each pair creates an active communication edge, and the system should output users ordered by current communication count. Follow-ups add self-communication handling, duplicate communication dedupe, and top-K users by count. An additional API variant exposes register and mostActiveUser(), then asks for constant-time most-active lookup and a changing-K ranking extension.SWEhashmapsetsortingtop-kstreamingmediumFrequencyLowLast asked2026-09-22Stagephone-screen

APPROACH

Implement the pair-normalization layer separately from count maintenance.
Practice maintaining user -> count, edge set, and count -> ordered users views without letting updates drift.
Dry-run duplicate edges and self-edges before discussing top-K optimization.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether a pair A-B counts for both users, whether repeated pairs count once, how self communication is treated, how ties order, and whether k changes over time.

BRUTE FORCE TO SAY OUT LOUD

Recount from all edges and sort users on every query. O(E log U).

OPTIMAL IDEA

Normalize pairs (sorted tuple). Keep an edge set for deduplication, a dict of user to count, and either a sorted structure of (count, user) or a bucket map count to ordered users so an update moves a user by one bucket. Top-K reads the highest buckets. Note the edge set can use quadratic space in the number of users; say so.

DRY-RUN THESE CASES

A-B then B-A (duplicate), a self pair, ties on count, a new user, k larger than the number of users.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Constant-time most active user (track the maximum count with buckets), a changing k, or expiry of old edges.

PITFALLS

Claiming linear space while storing all pairs. Counting a duplicate edge twice.

HOW TO THINK ABOUT IT

Normalize the pair first (sorted tuple) so A-B and B-A are one edge, then keep an edge set for dedup and a count per user. The ranking part is a separate question: top-K comes from a heap, and the single most active user can be maintained in O(1) because counts only ever go up. Mention that the edge set uses quadratic space in the worst case.

Basis: Written. Assumed rules (confirm them): a pair 'A-B' is an unordered communication edge; repeated pairs count once; a user's activity is the number of distinct partners; order by activity descending, then name; self pairs (A-A) are ignored unless a flag says otherwise.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
pairs = set()
def add(a, b):
    pairs.add((min(a, b), max(a, b)))
def top_users():
    partners = defaultdict(set)
    for a, b in pairs:
        partners[a].add(b)
        partners[b].add(a)
    def rank(u):
        return (-len(partners[u]), u)
    return sorted(partners, key=rank)
```

TIME	SPACE
O(E + U log U) per query	**O(E)**

WHY IT IS SLOW

Each query rebuilds all counts and sorts, O(E + U log U). Maintaining counts incrementally with count buckets gives O(1) updates and an O(1) most-active lookup.

OPTIMAL SOLUTION · Edge set, counts per user, count buckets for the maximum

TIME	SPACE
O(1) per pair, O(U log k) for top-K, O(1) for the most active user	**O(E)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Self communication (A-A) should now count.

Pass a flag to the class: a self pair is one edge that adds one to that user's count (and only once). The dedup set already handles repeated self pairs.

FOLLOW-UP 2 Top-K with a changing K, or a fixed-size priority queue.

A heap of size K can be wrong after counts change, because an element outside it can overtake one inside. Re-run heapq.nsmallest on the count dict per query (O(U log K)) or maintain a sorted structure keyed by (count, name) updated on every increment.

FOLLOW-UP 3 The space claim.

Distinct pairs can number U^2 / 2 in the worst case, so the edge set is quadratic in the users; the counts alone are linear. If duplicates are not a concern, drop the set and keep O(U).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Active Communication Top Users
#
# Process a continuous stream of user pairs such as A-B
# and B-C; each pair creates an active communication edge,
# and the system should output users ordered by current
# communication count. Follow-ups add self-communication
# handling, duplicate communication dedupe, and top-K
# users by count. An additional API variant exposes
# register and mostActiveUser(), then asks for constant-
# time most-active lookup and a changing-K ranking
# extension.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Self communication (A-A) should now count.
# 2. Top-K with a changing K, or a fixed-size priority
# queue.
# 3. The space claim.
#
# INTERVIEW NOTES
# Normalize the pair first (sorted tuple) so A-B and B-A
# are one edge, then keep an edge set for dedup and a
# count per user. The ranking part is a separate question:
# top-K comes from a heap, and the single most active user
# can be maintained in O(1) because counts only ever go
# up. Mention that the edge set uses quadratic space in
# the worst case.
#
# BRUTE FORCE (say it first): Recompute counts from all
# edges on every query
# Keep every edge and, for each query, rebuild the
# distinct-partner counts and sort the users. Fine as an
# oracle; the work is repeated on each call. Time O(E + U
# log U) per query. Space O(E).


import heapq
from collections import Counter, defaultdict


# BRUTE FORCE (say it first): Recompute counts from all
```

```
# edges on every query | time O(E + U log U) per query
def _norm(pair):
    a, b = pair.split("-")
    return (a, b) if a <= b else (b, a)
```

```
class ActiveBrute:
    def __init__(self, count_self=False):
        self.edges, self.count_self = set(), count_self

    def add(self, pair):
        a, b = _norm(pair)
        if a == b and not self.count_self:
            return
        self.edges.add((a, b))

    def counts(self):
        c = Counter()
        for a, b in self.edges:
            c[a] += 1
            if a != b:
                c[b] += 1
        return c

    def top_k(self, k):
        c = self.counts()
        return sorted(c, key=lambda u: (-c[u], u))[:k]

    def most_active(self):
        c = self.counts()
        if not c:
            return set()
        m = max(c.values())
        return {u for u, v in c.items() if v == m}


# BASE QUESTION (optimal): Edge set, counts per user,
# count buckets for the maximum
# Idea: A new edge increments the count of both users.
# Buckets map each count to the set of users with it, so a
# user moves from bucket c to bucket c + 1.
# FOLLOW-UP 1: Self communication (A-A) should now count.
# Answer: Pass a flag to the class: a self pair is one
# edge that adds one to that user's count (and only once).
# The dedup set already handles repeated self pairs.
class Active:
    def __init__(self, count_self=False):
        # distinct edges seen so far (for the dedupe rule)
        self.edges = set()
        # user -> how many different partners
```

```
        self.count = Counter()
        # count -> users having exactly that count
        self.bucket = defaultdict(set)
        # the highest count right now (it never drops)
        self.top = 0
        self.count_self = count_self

    def bump(self, user):
        c = self.count[user]
        # move the user from bucket c to bucket c + 1
        if c:
            self.bucket[c].discard(user)
        self.count[user] = c + 1
        self.bucket[c + 1].add(user)
        self.top = max(self.top, c + 1)

    def add(self, a, b):
        # A-B and B-A are the same edge
        pair = (min(a, b), max(a, b))
        # follow-ups: skip self pairs (unless counted)
        # and skip a repeated pair
        skip_self = a == b and not self.count_self
```

```
        if skip_self or pair in self.edges:
            return
        self.edges.add(pair)
        self.bump(a)
        # a self pair counts once, not twice
        if a != b:
            self.bump(b)

    def most_active(self):
        # O(1): read the top bucket
        if not self.top:
            return set()
        return set(self.bucket[self.top])

    def top_k(self, k):
        # k users by count, ties broken by name
        def rank(u):
            return (-self.count[u], u)
        return heapq.nsmallest(k, self.count, key=rank)


if __name__ == "__main__":
    a = Active()
```

```
    for x, y in [("A", "B"), ("B", "C"), ("B", "A")]:
        a.add(x, y)
    # B talks to A and C; the repeat B-A is ignored
    assert a.most_active() == {"B"}
    assert a.count["B"] == 2 and a.count["A"] == 1
    assert a.top_k(2) == ["B", "A"]
    # self pairs are ignored by default
    a.add("C", "C")
    assert a.count["C"] == 1
    # ... but can be switched on, counting once
    s = Active(count_self=True)
    s.add("C", "C")
    s.add("C", "C")
    assert s.count["C"] == 1
    assert Active().most_active() == set()
```

NOTES FROM CANDIDATES

A Java solution used an ordered map / tree structure to keep counts sortable.
The interviewer asked many clarification questions and expected dry runs.
The first version did not require self-communication handling; it was added as a coded follow-up.
Account for the per-user contact sets when explaining space use: the distinct relationship state can take quadratic space in the number of users. The interviewer explicitly challenged an initial linear-space claim.
A fixed-size priority-queue proposal did not finish the changing-K discussion; the interviewer asked for further optimization.
Maintaining the API variant
With registrations only and any tied maximum acceptable, normalize undirected pairs and store the distinct edges. For a new edge, increment each distinct endpoint once; an accepted self-edge changes one count, while an ignored self-edge changes none. Clarify that choice and the empty-state response.
Cache a user with the largest count. Counts never decrease, so only the endpoints of a new edge can displace the cached winner; duplicate edges do not change it. Reading this cache makes mostActiveUser() O(1), with expected O(1) registration work using

hash-based edge membership and counters under unit-cost user identifiers. A stricter tie rule requires separate handling.
For changing K, one option is a balanced ordered index of all users keyed by count and an agreed tie-breaker. Remove and reinsert the affected users when counts change. Updates cost O(log U); starting at the highest-ranked user and walking entries in descending-count order costs O(log U + K) for at most U returned users. This supports different K values without rebuilding a fixed-capacity heap and is a preparation option, not a specified interviewer solution.
Counters, the maximum cache, and the ranking index use O(U) space. Distinct relationship storage adds O(E), giving O(U + E) overall and O(U^2) in a dense graph.

PREPARATION

Implement the pair-normalization layer separately from count maintenance.
Practice maintaining user -> count, edge set, and count -> ordered users views without letting updates drift.
Dry-run duplicate edges and self-edges before discussing top-K optimization.

PRACHUB

1 questions | Hard 0, Medium 1, Easy 0 | posted in the last 30 days: 1, last 90 days: 1 (grouped by recency, newest first) | topics: BFS/DFS and trees first

POSTED: LAST 30 DAYS (1)

Medium (1)

MEDIUM | IN APP LC 305 | PracHub | Technical Screen | 21 of 22

Count Islands in a Grid and Track the Count as Land Cells Are Added

POSTED 2026-10-02 | 18 DAYS AGO | LAST 30 DAYS

STUDY: TALK ALL TIME 36 30 DAYS 0 PRIORITY 104/228

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #305 Number of Islands II

LeetCode match: LC 305 Number of Islands II

Posted on PracHub: 2026-10-02 (updated 2026-10-03)

Original: https://prachub.com/coding-questions/count-islands-in-a-grid-and-track-the-count-as-land-cells-are-added

QUESTION

A map is a grid with **m** rows and **n** columns in which every cell is land (**1**) or water (**0**). An island is a maximal group of land cells connected horizontally or vertically. Cells that touch only at a corner are not connected. You are given the initial map and a sequence of additions, each of which turns one cell into land. Report the number of islands before any addition, and again after each addition.

Function Signature

```
def island_counts(grid: list[list[int]], additions: list[tuple[int, int]]) -> list[int]:
```

Rules

- **grid[r][c]** is **1** for land and **0** for water, where **r** is the row index and **c** is the column index.
- An addition **(r, c)** turns cell **(r, c)** into land. If that cell is already land, either initially or because of an earlier addition, the map does not change.
- Additions are applied in the given order. Land never turns back into water.
- Return a list of length **len(additions) + 1**. Element **0** is the number of islands in the initial grid. For **i >= 1**, element **i** is the number of islands after the first **i** additions have been applied.

Constraints

- **1 <= m <= 1000**, **1 <= n <= 1000** and **m * n <= 200000**
- Every row of **grid** has exactly **n** entries, each **0** or **1**.
- **0 <= len(additions) <= 200000**
- Every addition **(r, c)** satisfies **0 <= r < m** and **0 <= c < n**. The same cell may appear more than once.
- Every count is at most **m * n**, so all values fit in a 32-bit signed integer.

Examples

Example 1

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = []
Output: [3]
```

The islands are {(0, 0), (0, 1)}, {(1, 3), (2, 3)} and {(2, 0)}. With no additions, the only element is the initial count.

Example 2

```
Input: grid = [[0, 0, 0], [0, 0, 0], [0, 0, 0]], additions = [(0, 0), (0, 1), (1, 2), (2, 1), (1, 1), (1, 1)]
Output: [0, 1, 1, 2, 3, 1, 1]
```

The grid starts with no land. (0, 0) creates an island and (0, 1) joins it. (1, 2) and (2, 1) each touch no land, so each creates a new island. (1, 1) touches all three islands and merges them into one. The second (1, 1) is already land, so nothing changes.

Example 3

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = [(0, 2), (0, 3), (1, 0)]
Output: [3, 3, 2, 1]
```

(0, 2) joins the top-left island. It touches (1, 3) only at a corner, so the count stays 3. (0, 3) connects the top-left island to the island at (1, 3) and (2, 3), giving 2. (1, 0) connects the merged island to (2, 0), giving 1.

APPROACH

Union-find over cells: start with initial islands, then each addition creates a component and unions with the four neighbors; track the component count. Ignore repeated cells. O((mn + k) alpha).

HOW TO THINK ABOUT IT

Re-running a flood fill after every addition costs O(mn) each time. Land only ever appears, so islands only merge: Union-Find fits exactly. A new land cell starts as a new island (count + 1) and every union with a neighbouring land cell that belongs to a different island removes one island (count - 1).

Basis: LeetCode 305, Number of Islands II. Report the count before any addition and after each; adding a cell that is already land changes nothing.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def add_land(r, c):
    grid[r][c] = 1
    return count_islands(grid)  # full flood fill
answers = [add_land(r, c) for r, c in additions]
```

TIME	SPACE
O(A * m * n)	O(m * n)

WHY IT IS SLOW

Each addition costs a pass over the whole grid, O(A * m * n). Union-Find merges the new cell with its land neighbours and updates the count in near-constant time.

OPTIMAL SOLUTION · Union-Find updated cell by cell

TIME	SPACE
O(m * n + A * alpha(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Land can also be removed (or turned back to water).

Union-Find cannot split sets. Either recompute the affected island with a local flood fill, or process all operations offline with a segment tree over time and a rollback Union-Find.

FOLLOW-UP 2 The grid is 10^9 x 10^9 with a few additions.

Keep the Union-Find on a dictionary keyed by coordinates, as the code does; memory depends on the number of land cells only, never on the grid size. **Cost:** O(a * alpha) for a additions and O(a) memory, independent of the grid size.

FOLLOW-UP 3 Report the size of the largest island after each addition.

Store a size per root in the Union-Find (union by size) and keep a running maximum, which only grows when land is added; each step stays near-constant time. Checked against recomputing the largest island from scratch. **Cost:** O(alpha) per addition with size tracking; the maximum is one running variable.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Islands in a Grid and Track the
# Count as Land Cells Are Added
#
# A map is a grid with m rows and n columns in which every
# cell is land (1) or water (0). An island is a maximal
# group of land cells connected horizontally or
# vertically. Cells that touch only at a corner are not
# connected. You are given the initial map and a sequence
# of additions, each of which turns one cell into land.
# Report the number of islands before any addition, and
# again after each addition. ### Function Signature python
# def island_counts(grid: list[list[int]], additions:
# list[tuple[int, int]]) -> list[int]: ### Rules -
# grid[r][c] is 1 for land and 0 for water, where r is the
# row index and c is the column index. - An addition (r,
# c) turns cell (r, c) into land. If that cell is already
# land, either initially or because of an earlier
# addition, the map does not change. - Additions are
# applied in the given order. Land never turns back into
# water.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Land can also be removed (or turned back to water).
# 2. The grid is 10^9 x 10^9 with a few additions.
# 3. Report the size of the largest island after each
# addition.
#
# INTERVIEW NOTES
# Re-running a flood fill after every addition costs O(mn)
# each time. Land only ever appears, so islands only
# merge: Union-Find fits exactly. A new land cell starts
# as a new island (count + 1) and every union with a
# neighbouring land cell that belongs to a different
# island removes one island (count - 1).
#
# BRUTE FORCE (say it first): Recount all islands after
# every addition
# Apply the addition and run a full flood-fill count
# again. Correct and the oracle; every addition costs a
# pass over the entire grid. Time O(A * m * n). Space O(m
# * n).


# BRUTE FORCE (say it first): Recount all islands after
# every addition | time O(A * m * n)
```

```
def _count_islands(grid):
    m, n = len(grid), len(grid[0]) if grid else 0
    seen, count = set(), 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] == 1 and (r, c) not in seen:
                count += 1
                stack = [(r, c)]
                seen.add((r, c))
                while stack:
                    x, y = stack.pop()
                    for nx, ny in (
                        (x + 1, y),
                        (x - 1, y),
                        (x, y + 1),
                        (x, y - 1),
                    ):
                        if (
                            0 <= nx < m
                            and 0 <= ny < n
                            and grid[nx][ny] == 1
                            and (nx, ny) not in seen
                        ):
                            seen.add((nx, ny))
                            stack.append((nx, ny))
    return count


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def island_counts_recount(grid, additions):
```

```
    g = [row[:] for row in grid]
    out = [_count_islands(g)]
    for r, c in additions:
        g[r][c] = 1
        out.append(_count_islands(g))
    return out


# BASE QUESTION (optimal): Union-Find updated cell by cell
# Idea: Build the initial islands with unions (count
# them), then for each addition: ignore it if the cell is
# already land; otherwise make it a new set (count + 1)
```

```
# and union it with each land neighbour, subtracting one
# from the count every time two different sets merge.
class Islands:
    def __init__(self):
        # land cell -> parent cell, and sizes of the roots
        self.parent, self.size = {}, {}
        self.count, self.largest = 0, 0

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add(self, r, c):
        if (r, c) in self.parent:
            return
        self.parent[(r, c)] = (r, c)
        self.size[(r, c)] = 1
        self.count += 1
        self.largest = max(self.largest, 1)
        # join with every land neighbour
        for dr, dc in STEPS:
            nb = (r + dr, c + dc)
            if nb in self.parent:
                self.union((r, c), nb)

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return
        # union by size: the small island joins the big
        # one
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
        self.count -= 1
        self.largest = max(self.largest, self.size[a])


# BASE QUESTION (optimal): Union-Find updated cell by cell
# Idea: Build the initial islands with unions (count
# them), then for each addition: ignore it if the cell is
```

```
# already land; otherwise make it a new set (count + 1)
# and union it with each land neighbour, subtracting one
# from the count every time two different sets merge.
def island_counts(grid, additions):
    # counts before any addition, then after each one
    isl = Islands()
    for r, row in enumerate(grid):
        for c, v in enumerate(row):
            if v == 1:
                isl.add(r, c)
    out = [isl.count]
    for r, c in additions:
        isl.add(r, c)
        out.append(isl.count)
    return out


# FOLLOW-UP 3: Report the size of the largest island after
# each addition.
# Answer: Store a size per root in the Union-Find (union
# by size) and keep a running maximum, which only grows
# when land is added; each step stays near-constant time.
# Checked against recomputing the largest island from
# scratch.
def largest_after_each(grid, additions):
    # the follow-up: size of the biggest island each time
    isl = Islands()
    for r, row in enumerate(grid):
        for c, v in enumerate(row):
            if v == 1:
                isl.add(r, c)
    out = [isl.largest]
    for r, c in additions:
        isl.add(r, c)
        out.append(isl.largest)
    return out


if __name__ == "__main__":
    grid = [[1, 0, 0], [0, 0, 0], [0, 0, 1]]
    adds = [(0, 1), (1, 1), (2, 1), (0, 1)]
    # 2 islands; the adds join them into one; a repeat is
    # ignored
    assert island_counts(grid, adds) == [2, 2, 2, 1, 1]
    sizes = largest_after_each(grid, adds)
```

```
    assert sizes == [1, 2, 3, 5, 5]
    # corners do not connect cells
    assert island_counts([[1, 0], [0, 1]], []) == [2]
    # the grid may be huge: only land cells are stored
    big = Islands()
    big.add(10 ** 9, 10 ** 9)
    big.add(10 ** 9, 10 ** 9 + 1)
    assert big.count == 1 and big.largest == 2
```

FUNCTION AND TEST CASES

Signature

```
def island_counts(grid: list[list[int]], additions: list[l
ist[int]]) -> list[int]:
```

Sample call

```
island_counts([[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]],
[])
```

Test cases

Input	Expected
`([[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], [])`	`[3]`
`([[0, 0, 0], [0, 0, 0], [0, 0, 0]], [[0, 0], [0, 1], [1, 2], [2, 1], [1, 1], [1, 1]])`	`[0, 1, 1, 2, 3, 1, 1]`

EXPONENT

1 questions | Hard 0, Medium 1, Easy 0 | last asked (approx) in the last 30 days: 1, last 90 days: 1 (grouped by recency, newest first) | topics: BFS/DFS and trees first

ASKED (ROUGH): LAST 30 DAYS (1)

Medium (1)

MEDIUM | **Exponent | 22 of 22**

Find the shortest path in a graph.

ASKED (ANY CO.) ~11 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | **ALL TIME 37.9** | **30 DAYS 33** | **PRIORITY 62/228**

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #743 Network Delay Time, LeetCode #1091 Shortest Path in Binary Matrix

LeetCode match: No direct match; related LC 743 Network Delay Time, LC 1091 (inferred from short statement)

Last asked (any company): 11 days ago, about 2026-09 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/2094/find-shortest-path-graph

QUESTION

Find the shortest path in a graph.

APPROACH

Unweighted: BFS; weighted non-negative: Dijkstra with a heap; negative edges: Bellman-Ford. Clarify the graph type first.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Clarify the graph first: directed or undirected, weighted, whether weights can be negative, the size, and whether you need the path itself or only its length.

BRUTE FORCE TO SAY OUT LOUD

Enumerate all paths with DFS and take the shortest. Exponential.

OPTIMAL IDEA

Unweighted: BFS with a parent map, O(V + E). Non-negative weights: Dijkstra with a min-heap, O((V + E) log V). Negative weights without a negative cycle: Bellman-Ford, O(V * E). All pairs on a small graph: Floyd-Warshall. A weights of 0 and 1 graph: 0-1 BFS. Reconstruct the path from the parent pointers.

DRY-RUN THESE CASES

Source equals destination, no path, a graph with a cycle, parallel edges, a negative edge (shows why Dijkstra fails).

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

A* with a heuristic, bidirectional BFS, or the k shortest paths.

PITFALLS

Using BFS on a weighted graph. Not stating the assumption about edge weights.

HOW TO THINK ABOUT IT

Ask the graph questions first: directed or undirected, weighted or not, can weights be negative, and do you want the length or the path. That choice picks the algorithm: unweighted means breadth-first search, non-negative weights mean Dijkstra with a heap, negative weights mean Bellman-Ford. Remember parent pointers if the path itself is needed.

Basis: The shortest-path family: BFS (unweighted), Dijkstra (non-negative weights, LeetCode 743), Bellman-Ford (negative weights).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, source, target):
    dist = [INF] * n
    dist[source] = 0
    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    return dist[target]
```

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

Each round relaxes every edge: O(V * E). It is the right tool for negative weights; for non-negative weights Dijkstra's heap is O((V + E) log V), and for unweighted graphs BFS is O(V + E).

OPTIMAL SOLUTION · Dijkstra with a heap (non-negative weights), returning the path

TIME	SPACE
O((V + E) log V)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The graph is unweighted.

BFS from the source with a parent map gives a shortest path in O(V + E); Dijkstra would be correct but needlessly slow. Stop as soon as you pop the target.

FOLLOW-UP 2 Weights can be negative.

Dijkstra can give wrong answers. Use Bellman-Ford (O(V * E)); one more pass that still improves a distance proves a negative cycle reachable from the source, where shortest paths are undefined.

FOLLOW-UP 3 Weights are only 0 or 1, or the graph is huge and you know the target.

0-1 BFS uses a deque in O(V + E). With a good distance estimate use A* (Dijkstra ordered by distance plus a heuristic that never overestimates); bidirectional search roughly squares the work saved on large graphs.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find the shortest path in a graph.
#
# Find the shortest path in a graph.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The graph is unweighted.
# 2. Weights can be negative.
# 3. Weights are only 0 or 1, or the graph is huge and you
# know the target.
#
# INTERVIEW NOTES
# Ask the graph questions first: directed or undirected,
# weighted or not, can weights be negative, and do you
# want the length or the path. That choice picks the
# algorithm: unweighted means breadth-first search, non-
# negative weights mean Dijkstra with a heap, negative
# weights mean Bellman-Ford. Remember parent pointers if
# the path itself is needed.
#
# BRUTE FORCE (say it first): Bellman-Ford: relax every
# edge V - 1 times
# Start with distance 0 at the source and repeatedly try
# to improve every edge's endpoint. It works with negative
# weights as well and is a good oracle for Dijkstra; it
# simply repeats full passes over all edges. Time O(V *
# E). Space O(V).


import heapq
from collections import defaultdict, deque


# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first): Bellman-Ford: relax every
# edge V - 1 times | time O(V * E)
def shortest_bellman_ford(n, edges, s, t):
    INF = float("inf")
    dist = [INF] * n
    dist[s] = 0
    for _ in range(max(0, n - 1)):
        changed = False
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v], changed = dist[u] + w, True
        if not changed:
            break
    return None if dist[t] == INF else dist[t]


# BASE QUESTION (optimal): Dijkstra with a heap (non-
# negative weights), returning the path
# Idea: Keep the best known distance to each node and a
# min-heap of (distance, node). Pop the closest unsettled
# node, relax its edges, and remember the parent of every
# improvement.
def dijkstra_path(edges, s, t):
    # weighted, non-negative edges (u, v, w)
    graph = defaultdict(list)
    for u, v, w in edges:
        graph[u].append((v, w))
    dist, parent, heap = {s: 0}, {s: None}, [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        if u == t:
            # the first pop of t is final: rebuild the
            # path
            path = []
            while u is not None:
                path.append(u)
                u = parent[u]
            return d, path[::-1]
        for v, w in graph[u]:
            if d + w < dist.get(v, INF):
                dist[v], parent[v] = d + w, u
                heapq.heappush(heap, (d + w, v))
    return None


# FOLLOW-UP 1: The graph is unweighted.
# Answer: BFS from the source with a parent map gives a
# shortest path in O(V + E); Dijkstra would be correct but
# needlessly slow. Stop as soon as you pop the target.
def bfs_path(edges, s, t):
    # unweighted: BFS is enough, O(V + E)
    graph = defaultdict(list)
    for u, v, *_ in edges:
        graph[u].append(v)
    parent, queue = {s: None}, deque([s])
    while queue:
        u = queue.popleft()
        if u == t:
            path = []
            while u is not None:
                path.append(u)
                u = parent[u]
            return len(path) - 1, path[::-1]
        for v in graph[u]:
            if v not in parent:
                parent[v] = u
                queue.append(v)
    return None


# FOLLOW-UP 2: Weights can be negative.
# Answer: Dijkstra can give wrong answers. Use Bellman-
# Ford (O(V * E)); one more pass that still improves a
# distance proves a negative cycle reachable from the
# source, where shortest paths are undefined.
def bellman_ford(n, edges, s):
    # negative weights allowed: n - 1 rounds of relaxing
    # every
    # edge; one more improving round proves a negative
    # cycle
```

```
    dist = [INF] * n
    dist[s] = 0
    for _ in range(n - 1):
        changed = False
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                changed = True
        if not changed:
            break
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            raise ValueError("negative cycle")
    return dist


# FOLLOW-UP 3: Weights are only 0 or 1, or the graph is
```

```
# huge and you know the target.
# Answer: 0-1 BFS uses a deque in O(V + E). With a good
# distance estimate use A* (Dijkstra ordered by distance
# plus a heuristic that never overestimates);
# bidirectional search roughly squares the work saved on
# large graphs.
def zero_one_bfs(n, edges, s):
    # weights are only 0 or 1: a deque keeps the order
    graph = defaultdict(list)
    for u, v, w in edges:
        graph[u].append((v, w))
    dist = [INF] * n
    dist[s] = 0
    dq = deque([s])
    while dq:
        u = dq.popleft()
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                # a free edge goes to the front, a cost-1
                # edge back
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    return dist


if __name__ == "__main__":
    edges = [(0, 1, 4), (0, 2, 1), (2, 1, 2), (1, 3, 1)]
    assert dijkstra_path(edges, 0, 3) == (4, [0, 2, 1, 3])
    assert dijkstra_path(edges, 3, 0) is None
    assert dijkstra_path(edges, 0, 0) == (0, [0])
    assert bfs_path(edges, 0, 3) == (2, [0, 1, 3])
    neg = [(0, 1, 5), (1, 2, -3)]
    assert bellman_ford(4, neg, 0) == [0, 5, 2, INF]
    try:
        bellman_ford(2, [(0, 1, 1), (1, 0, -3)], 0)
        raise AssertionError("negative cycle not found")
    except ValueError:
        pass
    cheap = [(0, 1, 1), (1, 2, 0), (0, 2, 1)]
    assert zero_one_bfs(3, cheap, 0) == [0, 1, 1]
```